

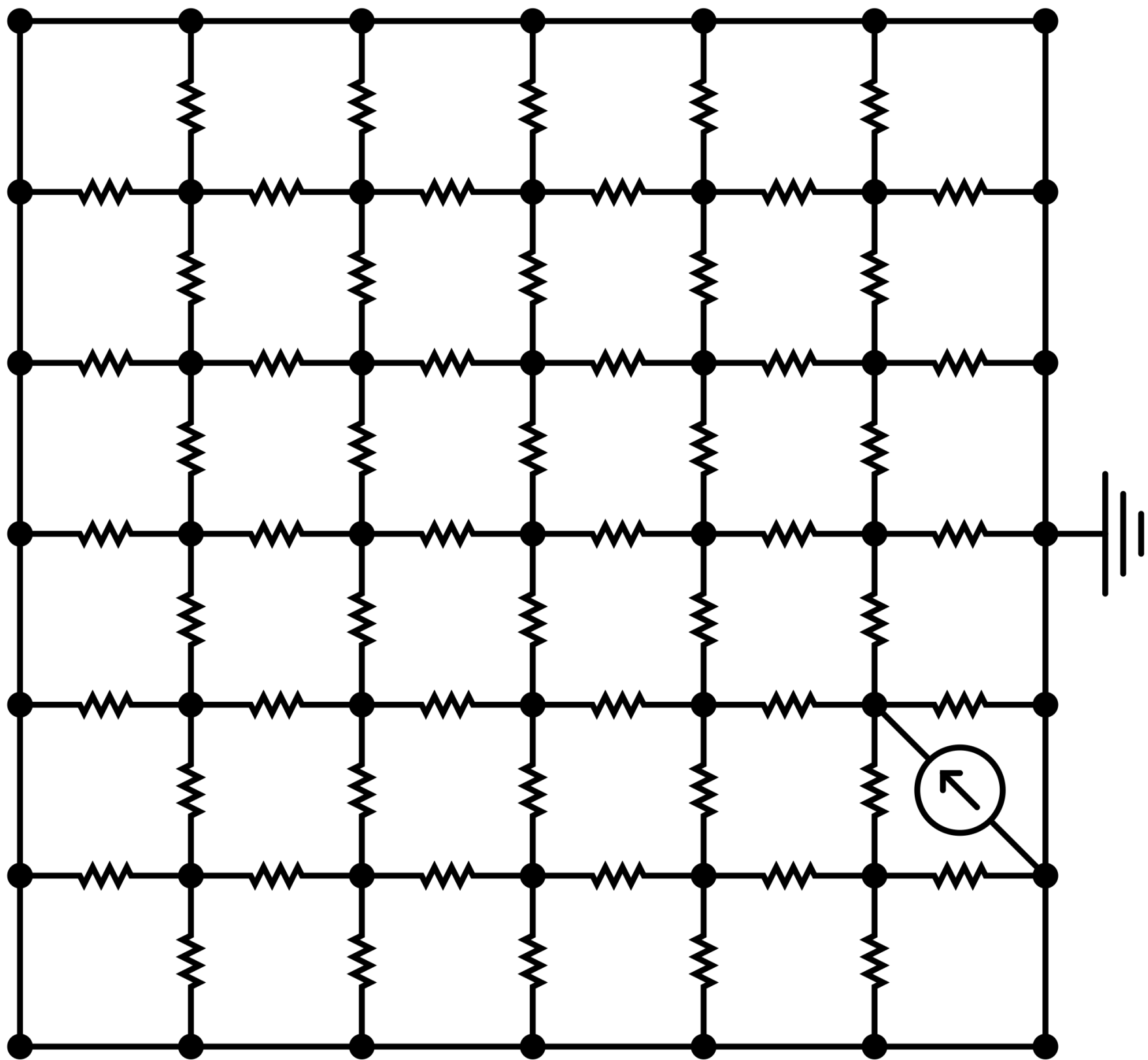
CUDA C++ By Example

Session 5: Optimizations II

From Last Time: Poisson's Equation

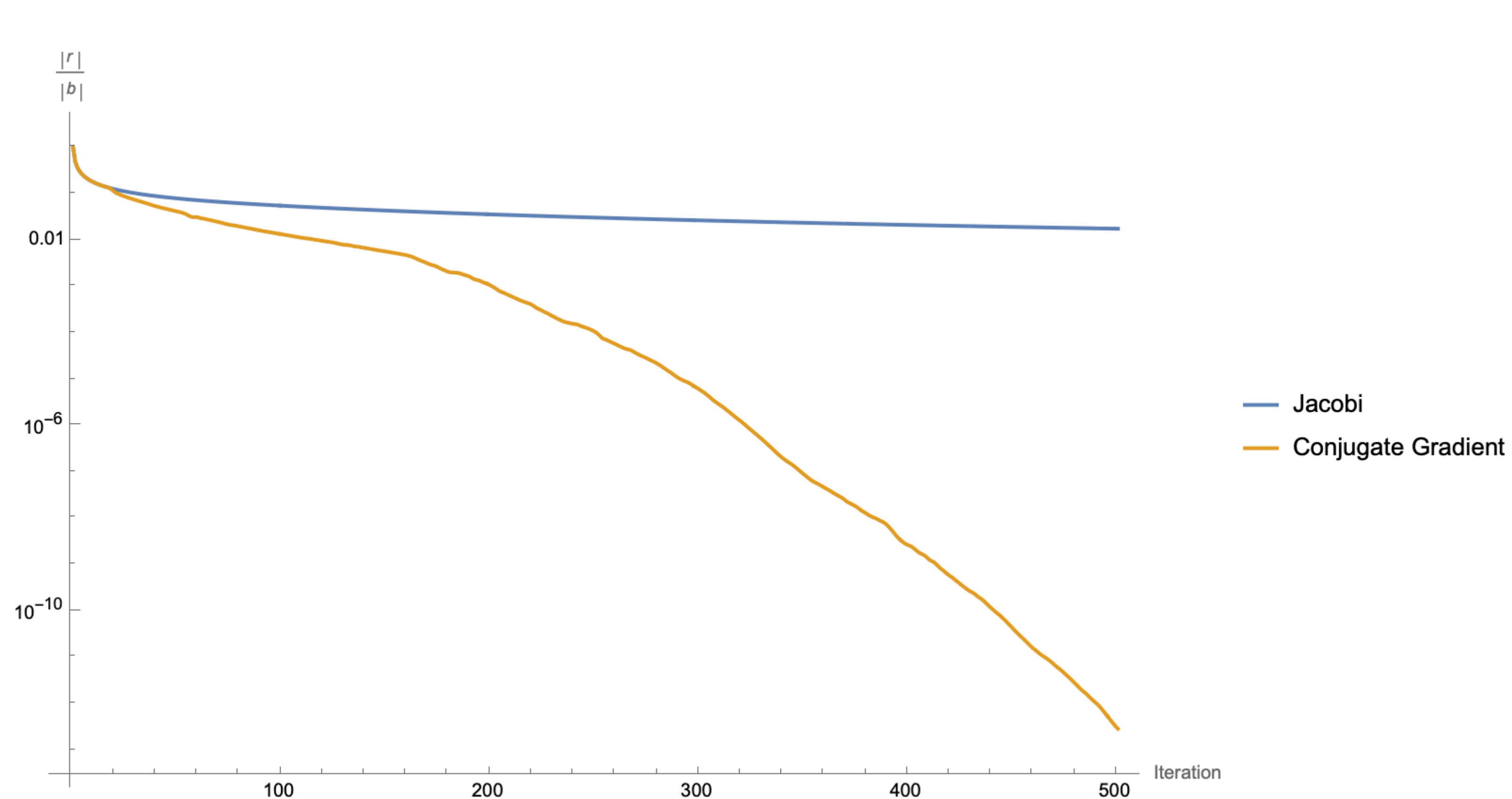
$$\nabla^2 u = f, \quad u = 0 \text{ on } \partial\Gamma$$

Which, after discretization is isomorphic to



```
~/code/Intro-to-CUDA/build$ ./poisson_0
0 0.408248
1 0.263523
2 0.199665
3 0.16319
4 0.139255
5 0.122182
6 0.109311
7 0.0992158
8 0.091058
9 0.0843111
10 0.0786259
...
163 0.0105274
164 0.0104795
165 0.0104322
166 0.0103854
167 0.0103391
168 0.0102932
169 0.0102479
170 0.0102029
171 0.0101585
172 0.0101145
173 0.0100709
174 0.0100278
175 0.00998508
finished in 3478.52 ms
```

Optimization #1: Algorithm



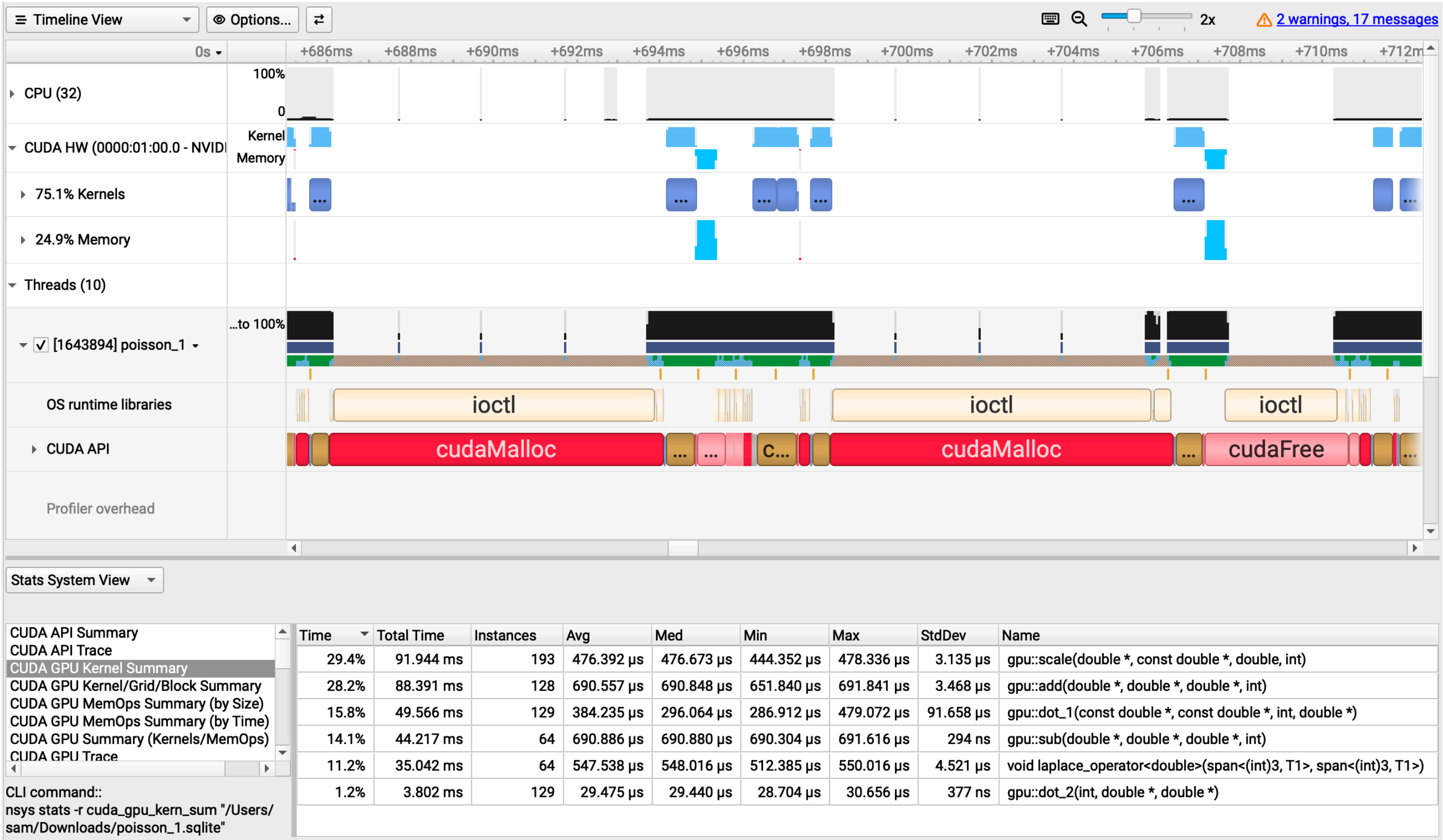
```
~/code/Intro-to-CUDA/build$ ./poisson_1
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 1516.11 ms
```


Optimization #2: Memory Allocation

These are **synchronous**, and run on the **default** stream:

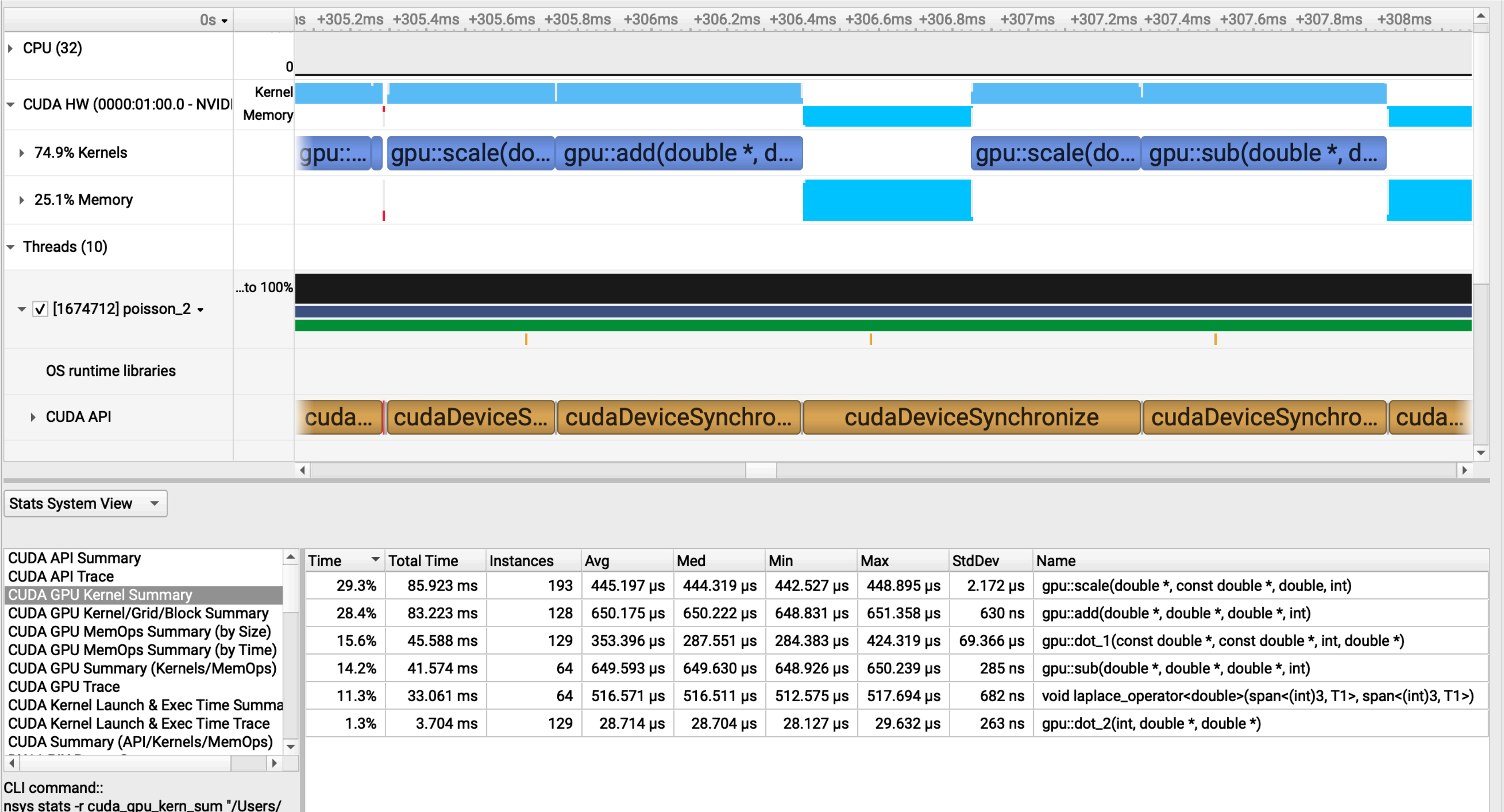
```
cudaMalloc(void ** ptr, size_t nbytes)
```

```
cudaFree(void * ptr);
```



Optimization #2: Memory Allocation

cudaMallocAsync/cudaFreeAsync can also be combined with
cudaMemPool_t to make memory allocation inside a loop less expensive:



```
~/code/Intro-to-CUDA/build$ ./poisson_2
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 383.364 ms
```

Optimization #3: Kernel Fusion

```
for (int i = 0; i < n; i++) {  
    tmp1[i] = alpha * d[i];  
}  
  
for (int i = 0; i < n; i++) {  
    tmp2[i] = x[i] + tmp1[i];  
}  
  
for (int i = 0; i < n; i++) {  
    x[i] = tmp2[i];  
}
```

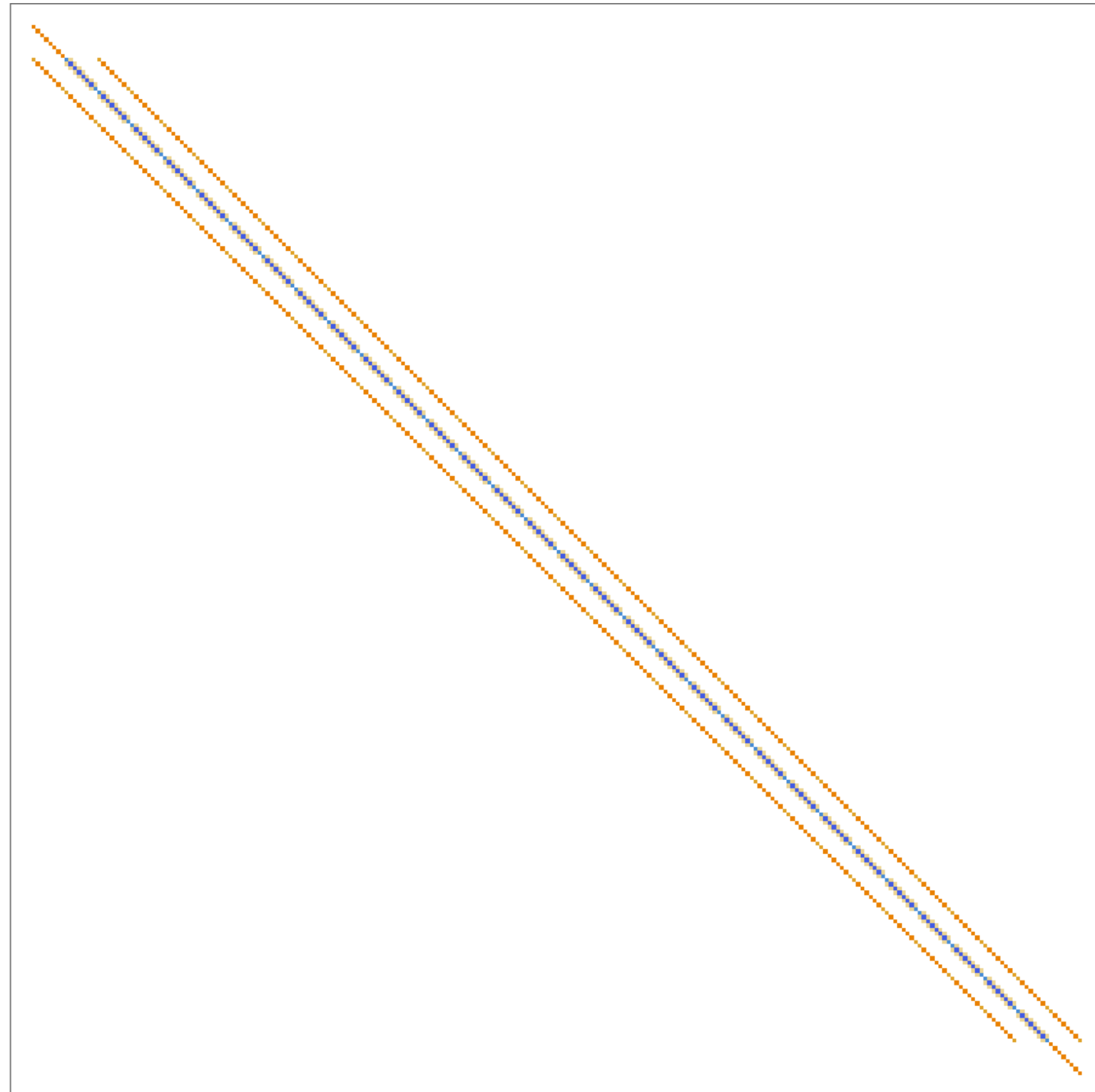
4n values read
3n values written
3 kernel launches*

```
for (int i = 0; i < n; i++) {  
    x[i] = x[i] + alpha * d[i];  
}
```

2n values read
1n values written
1 kernel launch*

```
~/code/Intro-to-CUDA/build$ ./poisson_3  
0 0.166667  
1 0.06  
2 0.032124  
3 0.0195303  
4 0.0131258  
5 0.00953437  
6 0.00713249  
7 0.00559492  
8 0.00448975  
9 0.00367153  
10 0.00307865  
...  
53 0.000138708  
54 0.00013376  
55 0.000129046  
56 0.00012465  
57 0.000120391  
58 0.000116405  
59 0.000112602  
60 0.000108948  
61 0.00010553  
62 0.000102211  
63 9.90787e-05  
finished in 212.451 ms
```

Optimization #4: SpMV



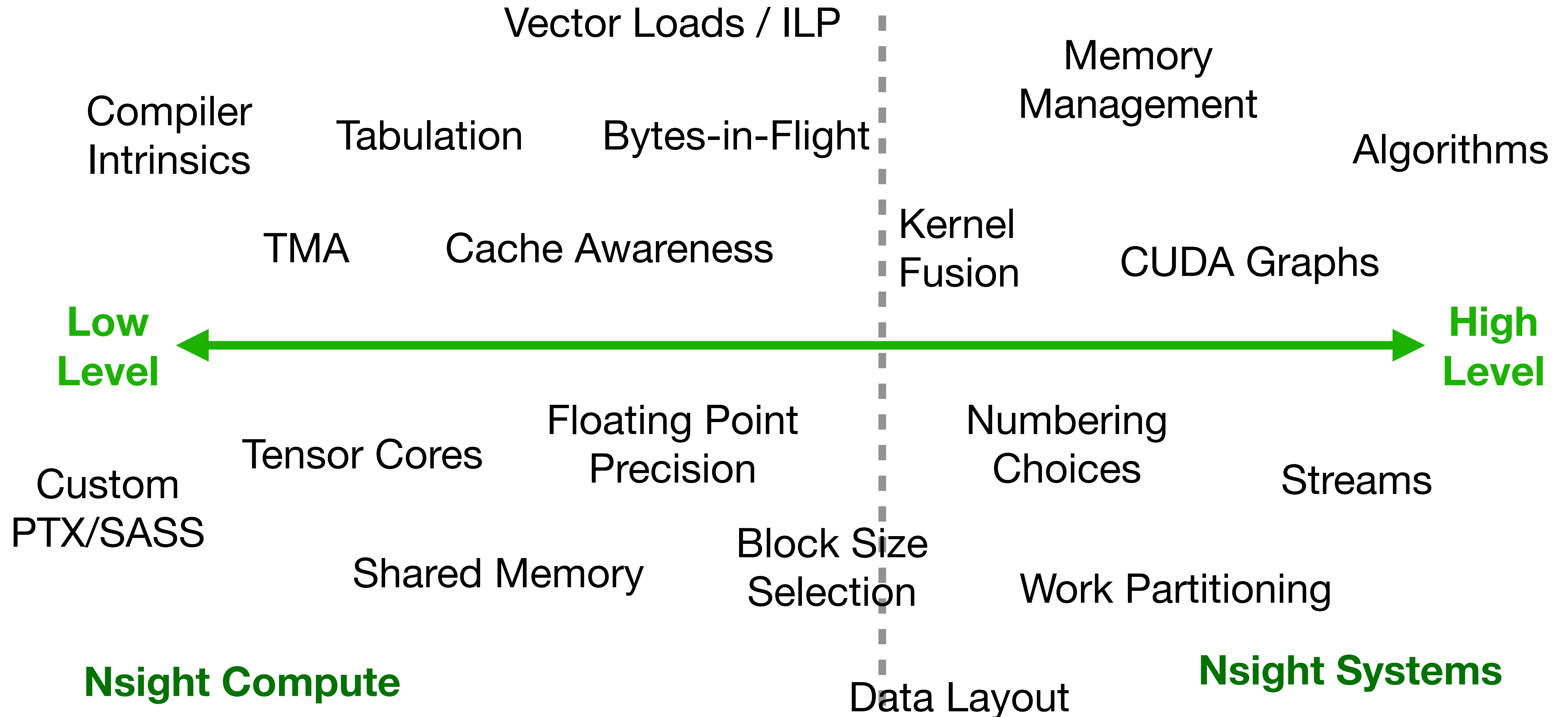
represent laplace operator w/
CSR matrix in cuSparse

```
~/code/Intro-to-CUDA/build$ ./poisson_4
ndof: 16777216, nnz: 115099600
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 396.666 ms
```

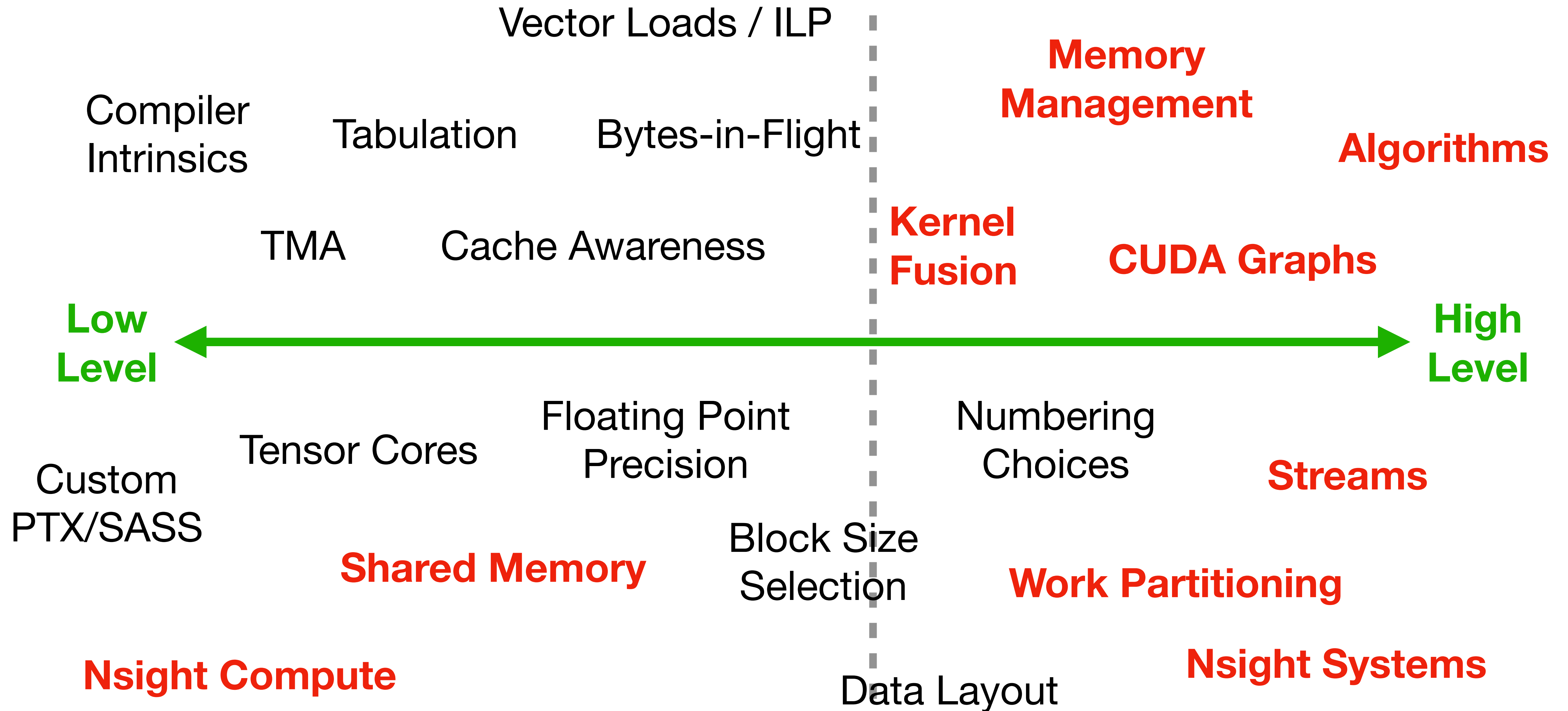
Why is SpMV slower here?

Arithmetic Intensity

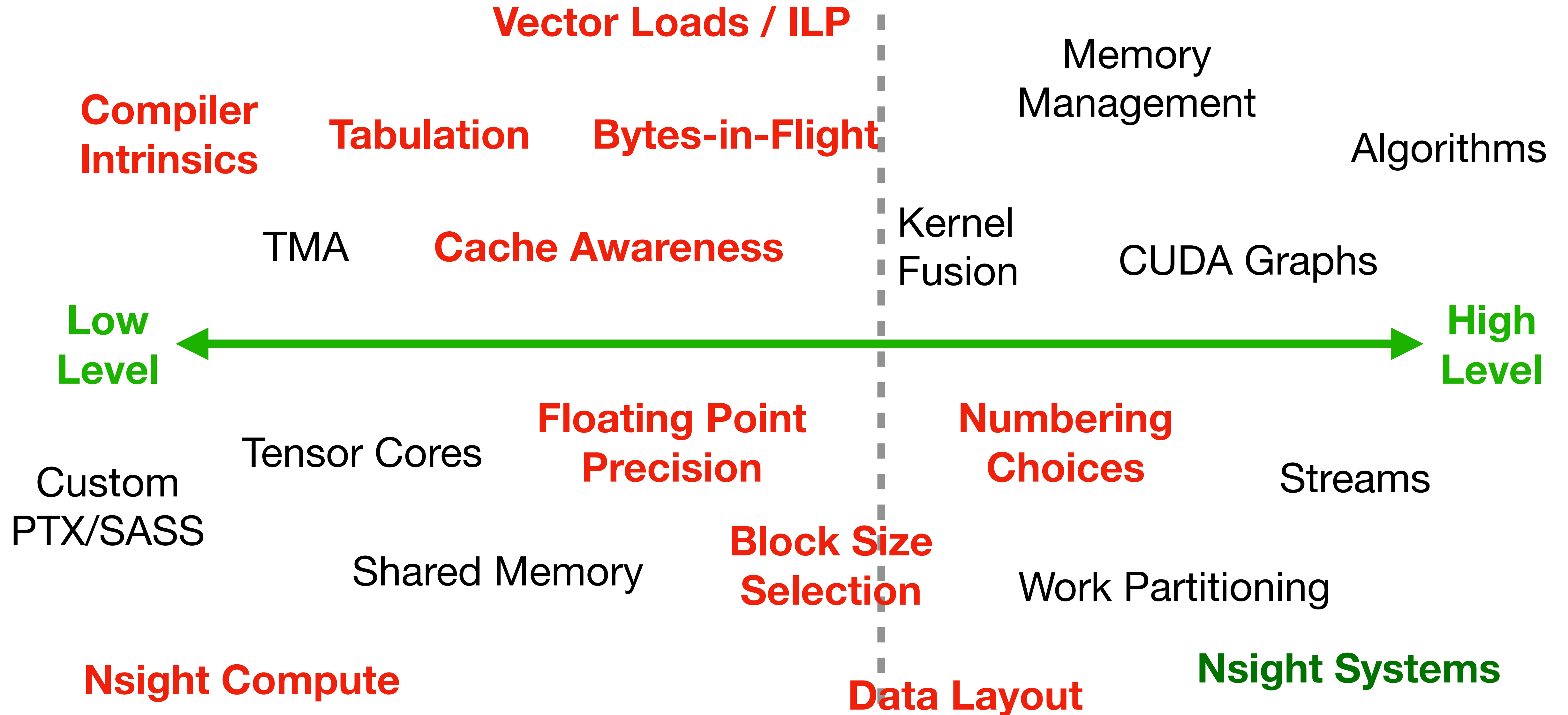
Application Optimizations



Discussed Already



This Session



Optimization #5: Blocksize Tuning

See: [06_profiling/poisson_5/laplace_tuning.cu](#)

Try running the laplace operator kernel with the following block sizes:

- 8x8x8
- 16x8x4, 4x8x16
- 32x8x2, 2x8x32
- 64x8x1, 1x8x64
- 128x4x1, 1x4x128

How does the performance vary with block size?
What might explain the differences?

Optimization #5: Blocksize Tuning

```
$ ./laplace_tuning  
8x8x8: 0.590658 ms  
4x8x16: 0.51185 ms  
16x8x4: 0.553713 ms  
32x8x2: 0.529425 ms  
2x8x32: 0.927768 ms  
64x8x1: 0.470374 ms  
1x8x64: 1.87671 ms  
128x4x1: 0.465897 ms  
1x4x128: 0.00371967 ms
```

on V100

```
$ ./laplace_tuning  
8x8x8: 0.207814 ms  
4x8x16: 0.188475 ms  
16x8x4: 0.170217 ms  
32x8x2: 0.169571 ms  
2x8x32: 0.244135 ms  
64x8x1: 0.168906 ms  
1x8x64: 0.421257 ms  
128x4x1: 0.172216 ms  
1x4x128: 0.004181 ms
```

on RTX 5090

Optimization #5: Blocksize Tuning

Let's apply the best-performing block size to poisson_5/poisson.cu

```
~/code/Intro-to-CUDA/build$ ./poisson_3
```

```
0 0.166667
```

```
1 0.06
```

```
2 0.032124
```

```
3 0.0195303
```

```
4 0.0131258
```

```
5 0.00953437
```

```
6 0.00713249
```

```
7 0.00559492
```

```
8 0.00448975
```

```
9 0.00367153
```

```
10 0.00307865
```

```
...
```

```
53 0.000138708
```

```
54 0.00013376
```

```
55 0.000129046
```

```
56 0.00012465
```

```
57 0.000120391
```

```
58 0.000116405
```

```
59 0.000112602
```

```
60 0.000108948
```

```
61 0.00010553
```

```
62 0.000102211
```

```
63 9.90787e-05
```

```
finished in 212.451 ms
```

```
~/code/Intro-to-CUDA/build$ ./poisson_5
```

```
0 0.166667
```

```
1 0.06
```

```
2 0.032124
```

```
3 0.0195303
```

```
4 0.0131258
```

```
5 0.00953437
```

```
6 0.00713249
```

```
7 0.00559492
```

```
8 0.00448975
```

```
9 0.00367153
```

```
10 0.00307865
```

```
...
```

```
53 0.000138708
```

```
54 0.00013376
```

```
55 0.000129046
```

```
56 0.00012465
```

```
57 0.000120391
```

```
58 0.000116405
```

```
59 0.000112602
```

```
60 0.000108948
```

```
61 0.00010553
```

```
62 0.000102211
```

```
63 9.90787e-05
```

```
finished in 207.397 ms
```

Optimization #5: Blocksize Tuning

Let's apply the best-performing block size to poisson_5/poisson.cu

```
~/code/Intro-to-CUDA/build$ ./poisson_3
```

```
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
```

```
finished in 212.451 ms
```

```
~/code/Intro-to-CUDA/build$ ./poisson_5
```

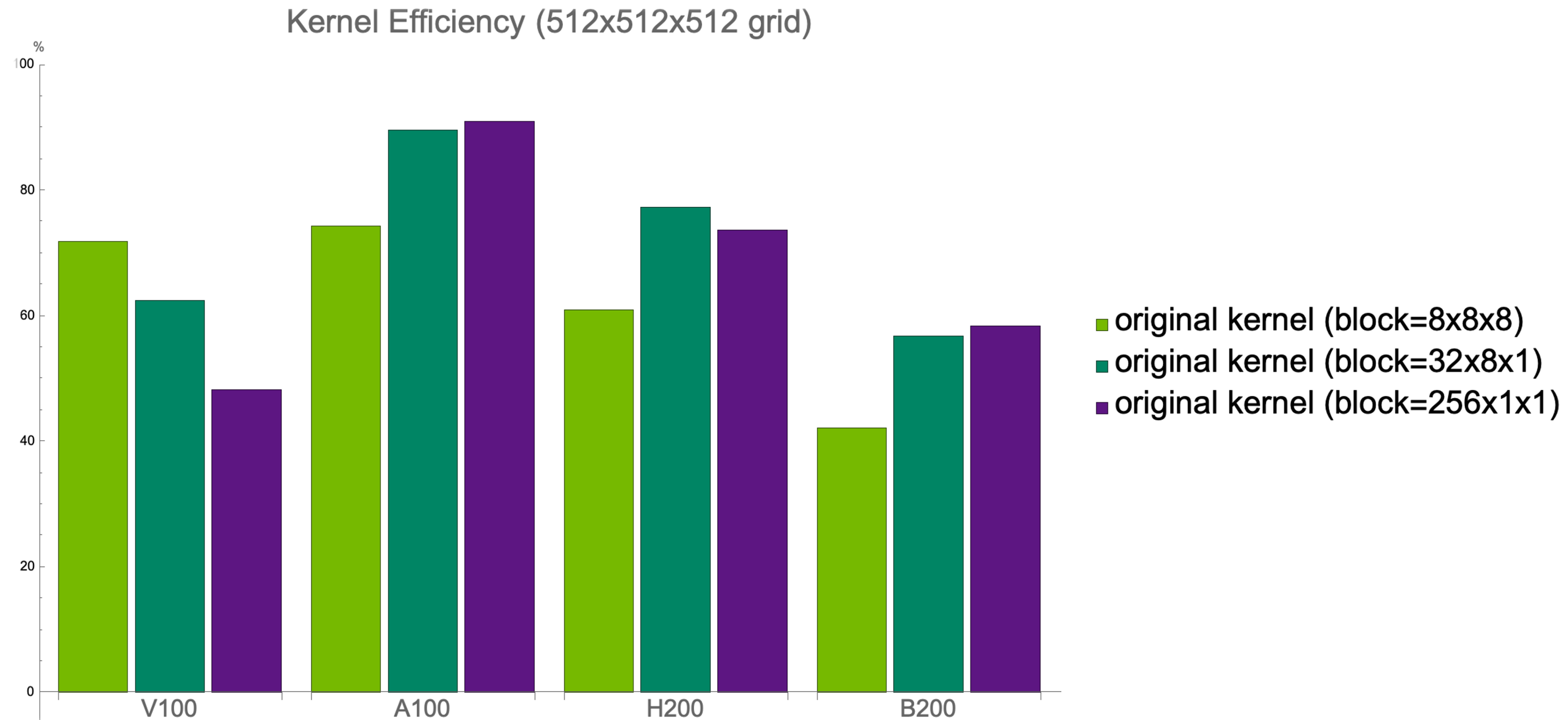
```
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
```

```
finished in 207.397 ms
```

The laplace kernel may be
25% faster with the new
block size, but the overall
improvement is only ~2-3%.

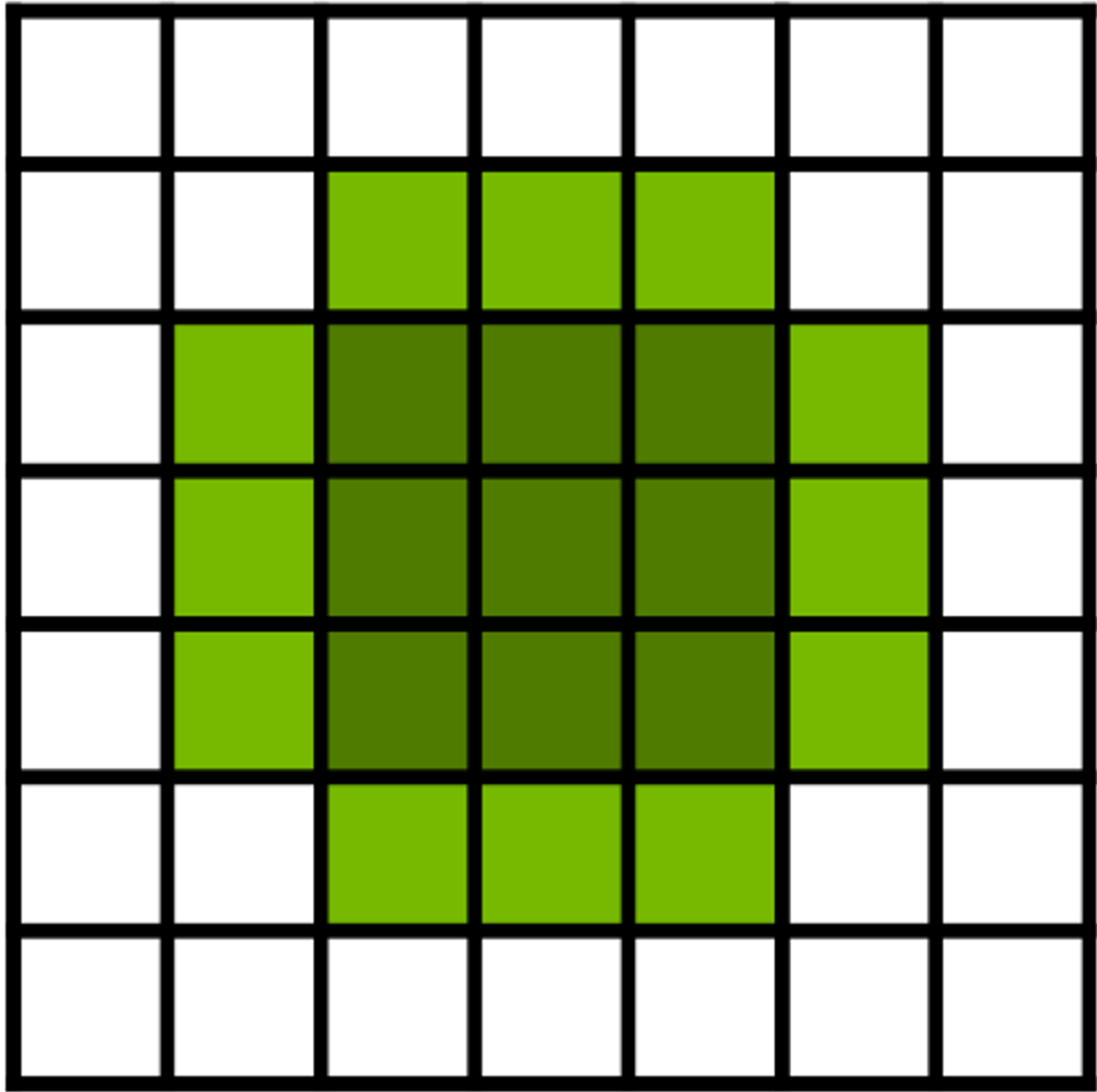
Amdahl's law in action!

Optimization #5: Blocksize Tuning



V100 prefers 8x8x8 block, but not newer architectures. Why?

Optimization #5: Blocksize Tuning



■ Read

■ Read + Write

Block Size

Bytes Moved

8x8x8

7168

16x8x4

7680

32x8x2

9472

64x8x1

13440

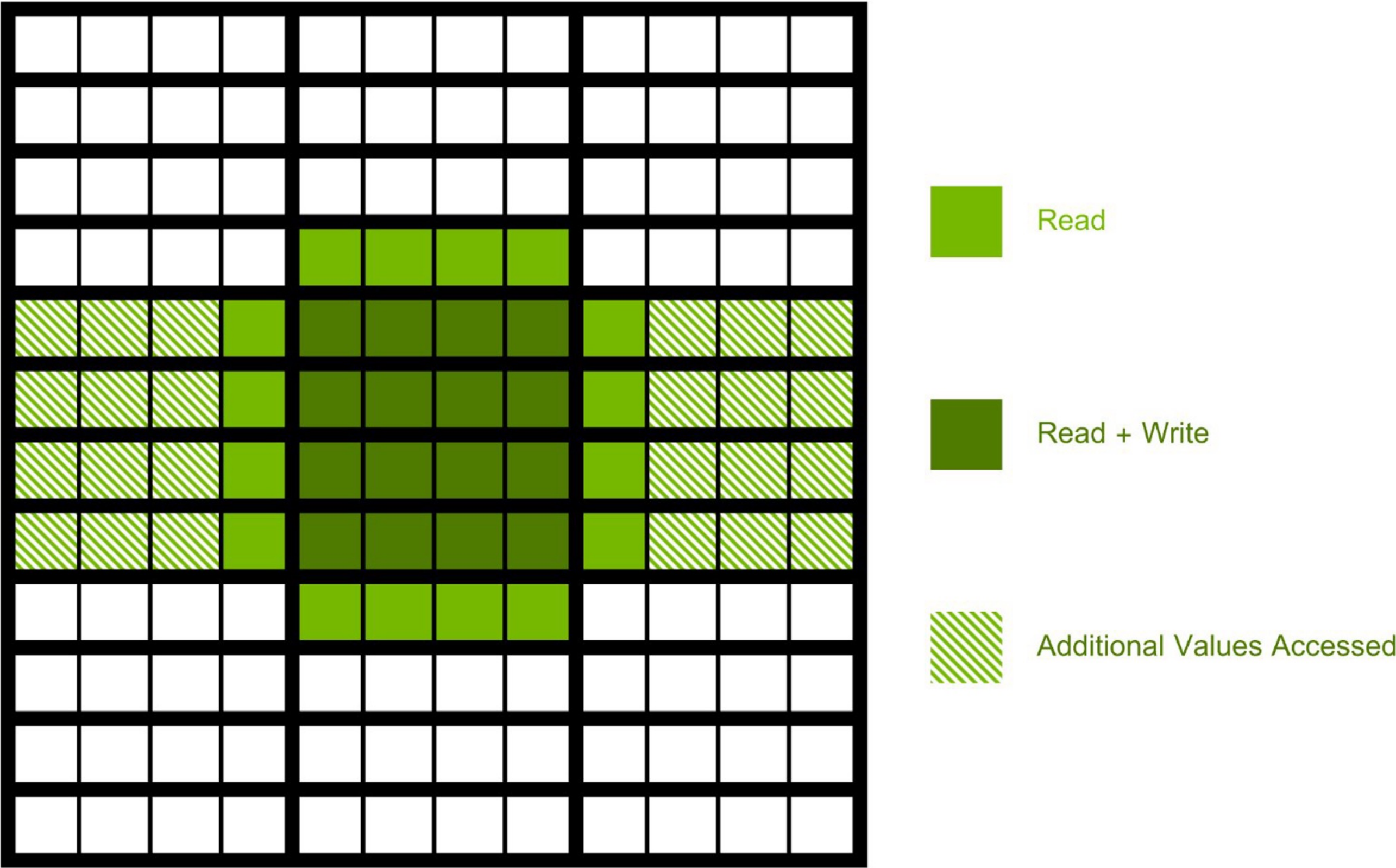
128x4x1

14400

256x2x1

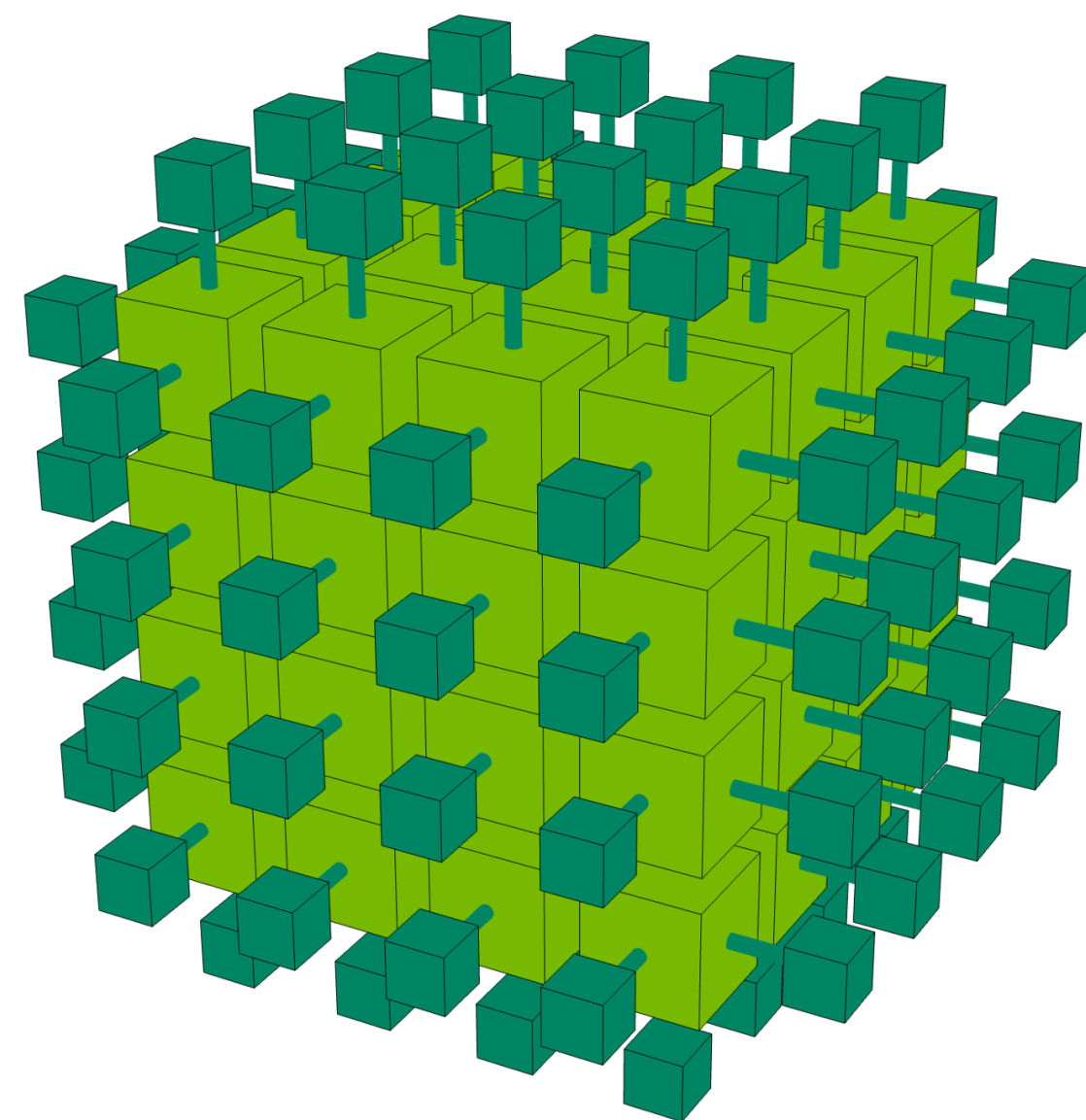
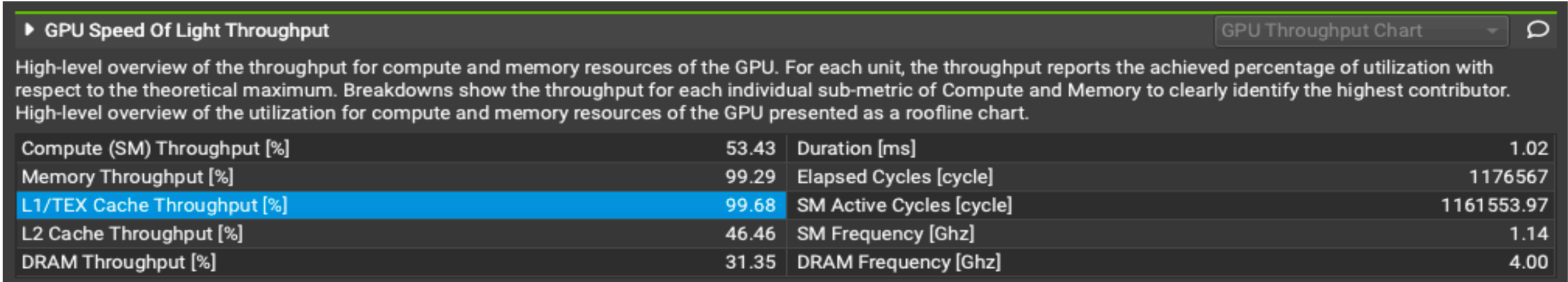
16416

Optimization #5: Blocksize Tuning

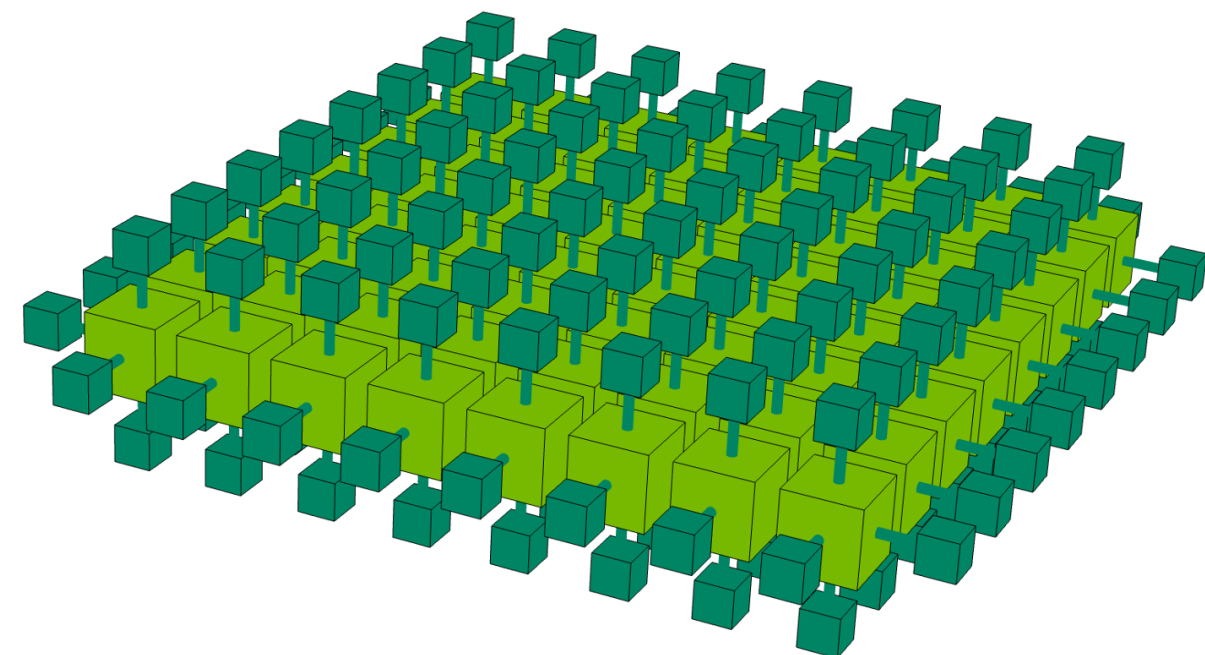


Block Size	Bytes Moved
8x8x8	22528
16x8x4	15360
32x8x2	13312
64x8x1	15360
128x4x1	15360
256x2x1	16896

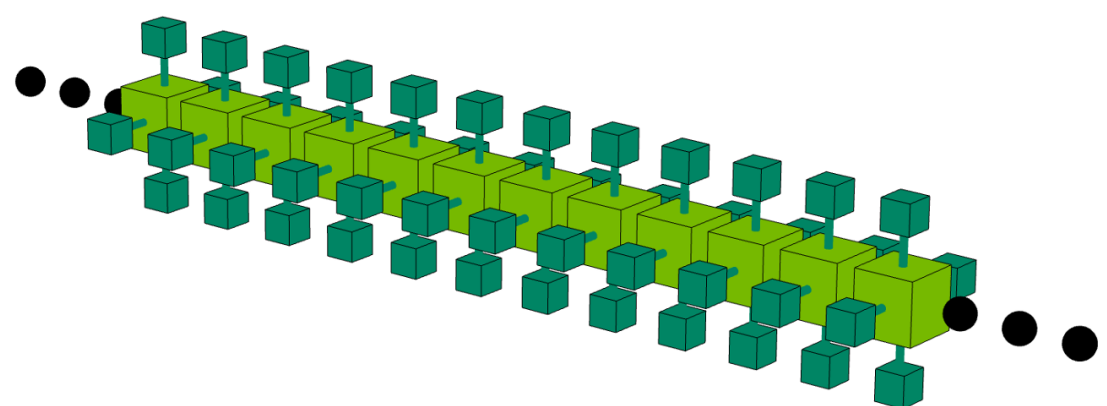
Optimization #5: Blocksize Tuning



4x4x4 Block
96 Halo Values



8x8x1 Block
160 Halo Values



64x1x1 Block
258 Halo Values

Optimization #6: AXPY / Dot Kernels

Let's look more closely at the AXPY / dot kernels.

Try building and running:

- 06_profiling/bytes_in_flight/axpy.cu
- 06_profiling/bytes_in_flight/dot_product.cu

```
$ ./axpy
```

```
...
```

```
$ ./dot_product
```

```
...
```

Optimization #6: AXPY / Dot Kernels

Note: runs below were performed on a Titan V (~650 GB/s memory throughput)

```
$ ./axpy
```

```
fp32: 0.476106 ms, 422.861 GB/s
```

```
fp64: 0.653148 ms, 616.481 GB/s
```

```
u8: 0.475577 ms, 105.833 GB/s
```

```
u16: 0.475801 ms, 211.566 GB/s
```

```
u32: 0.476103 ms, 422.864 GB/s
```

```
u64: 0.652940 ms, 616.677 GB/s
```

```
$ ./dot_product
```

```
fp32: 0.350763 ms 382.645 GB/s
```

```
fp64: 0.424643 ms 632.144 GB/s
```

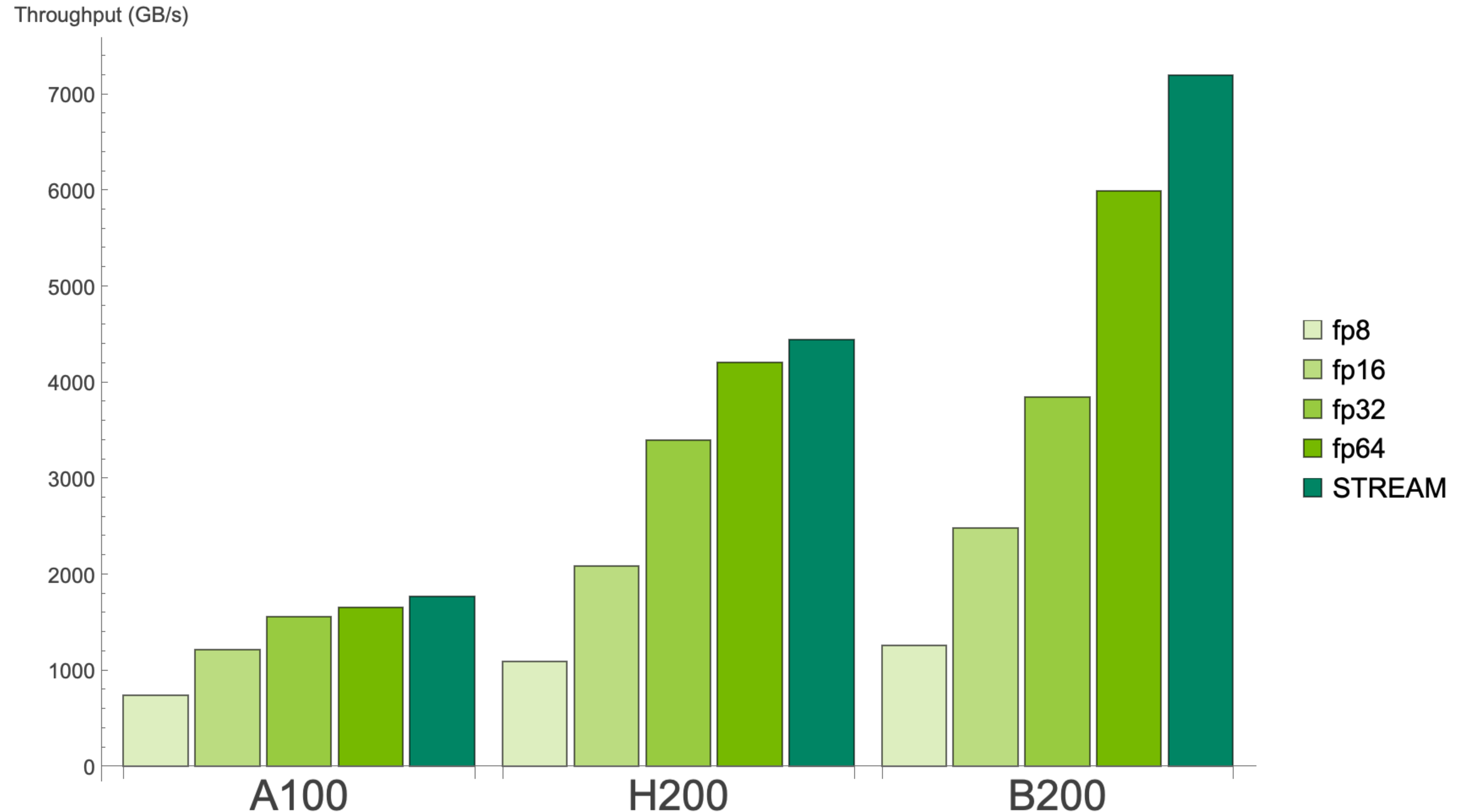
```
u8: 0.350187 ms 95.8186 GB/s
```

```
u16: 0.349897 ms 191.796 GB/s
```

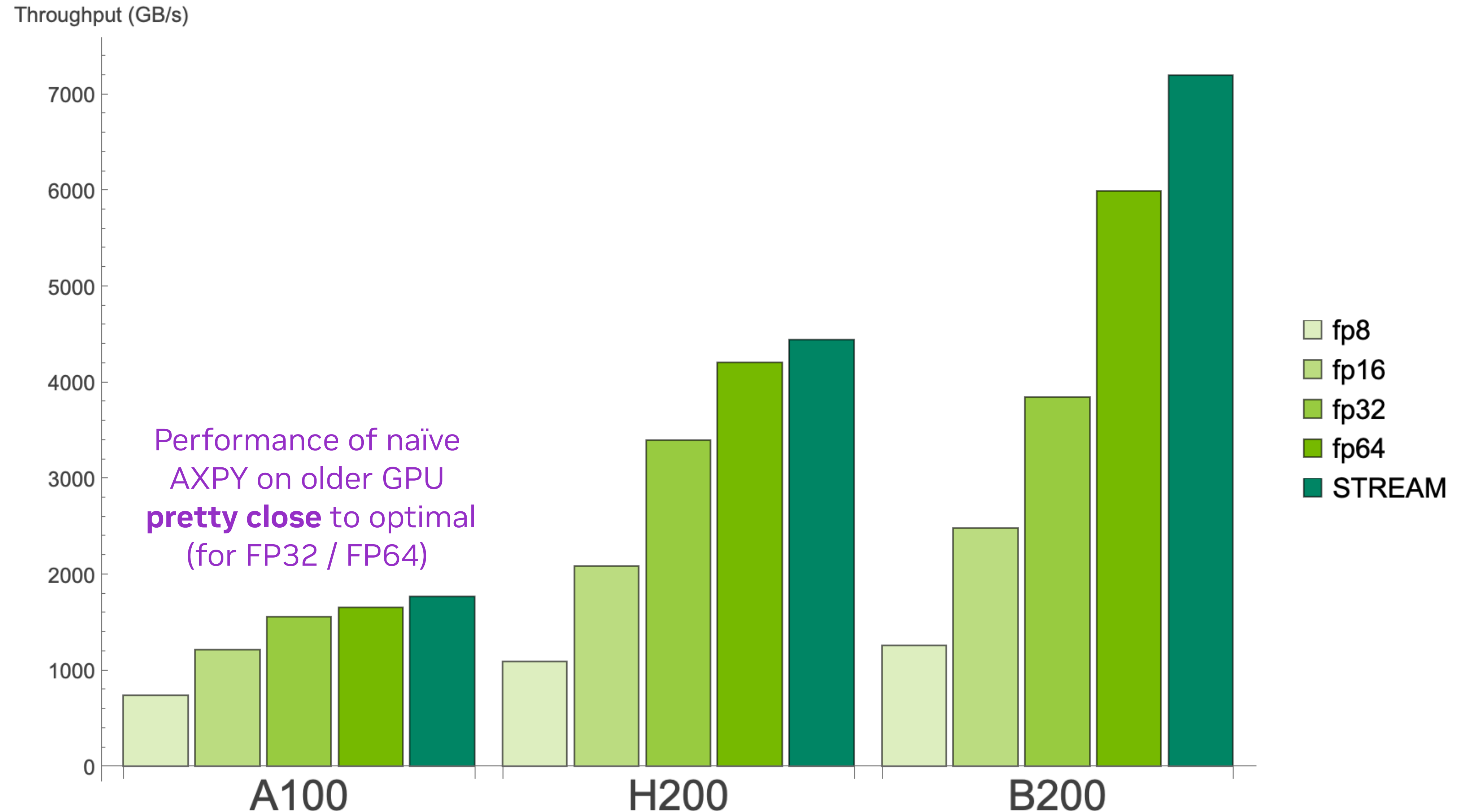
```
u32: 0.350116 ms 383.352 GB/s
```

```
u64: 0.423438 ms 633.943 GB/s
```

Optimization #6: AXPY / Dot Kernels

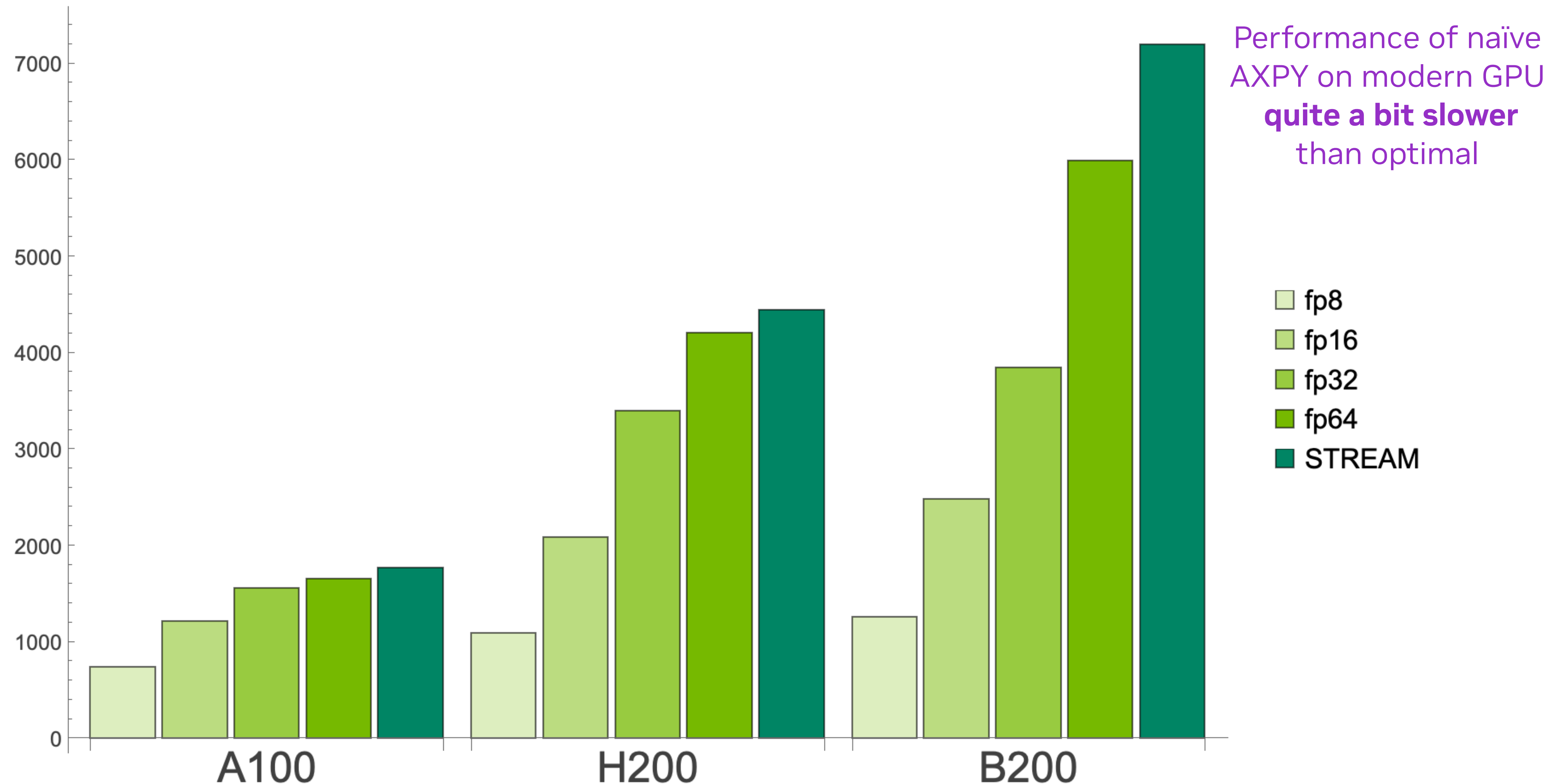


Optimization #6: AXPY / Dot Kernels

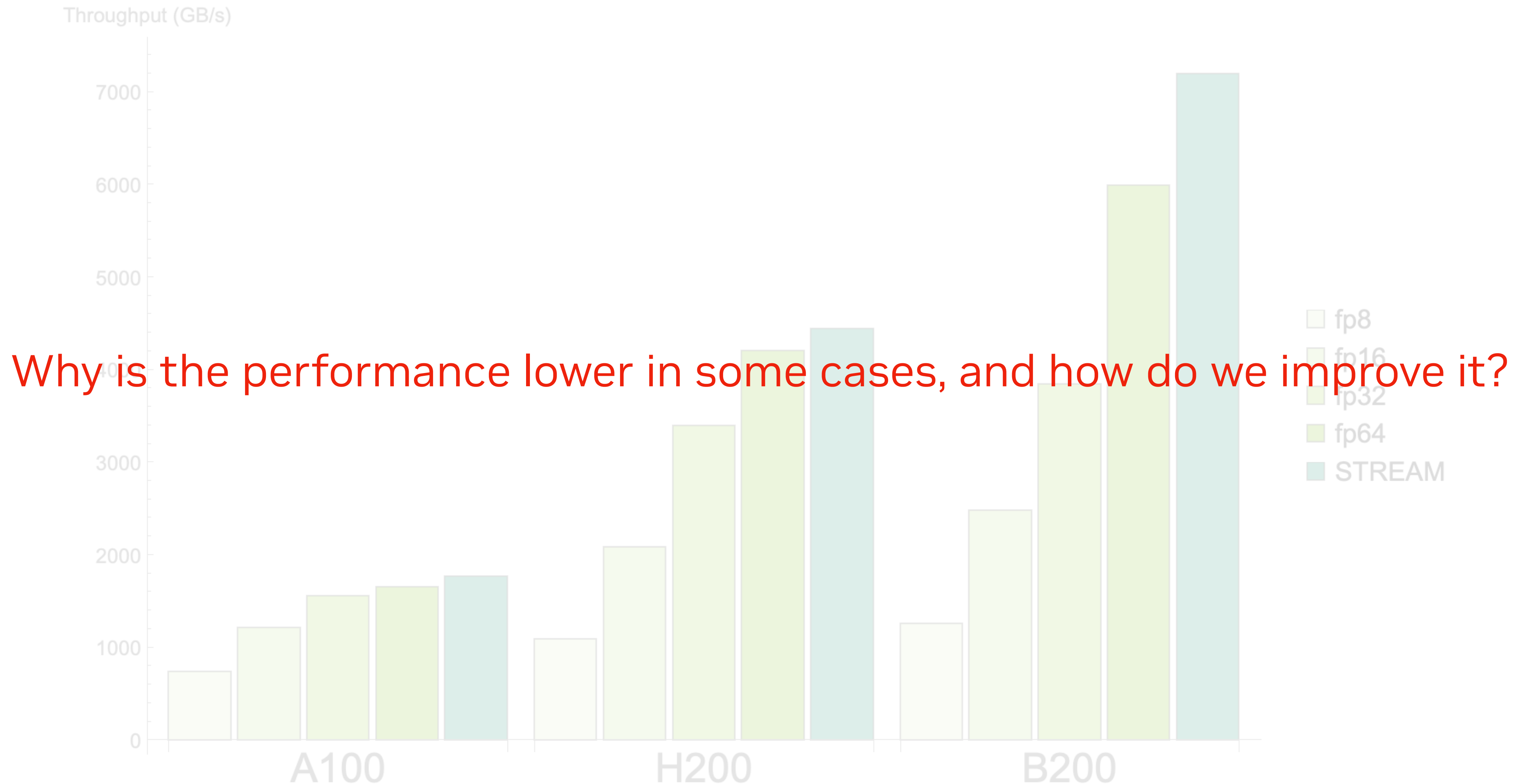


Optimization #6: AXPY / Dot Kernels

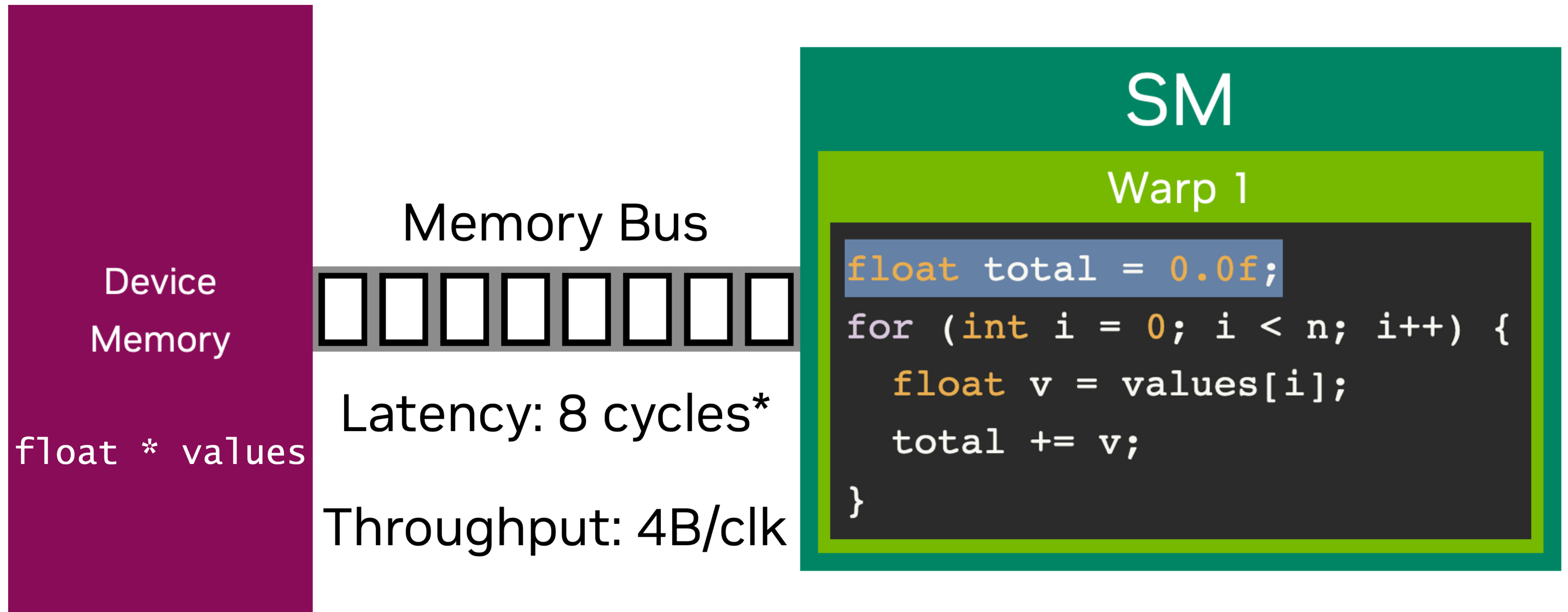
Throughput (GB/s)



Optimization #6: AXPY / Dot Kernels

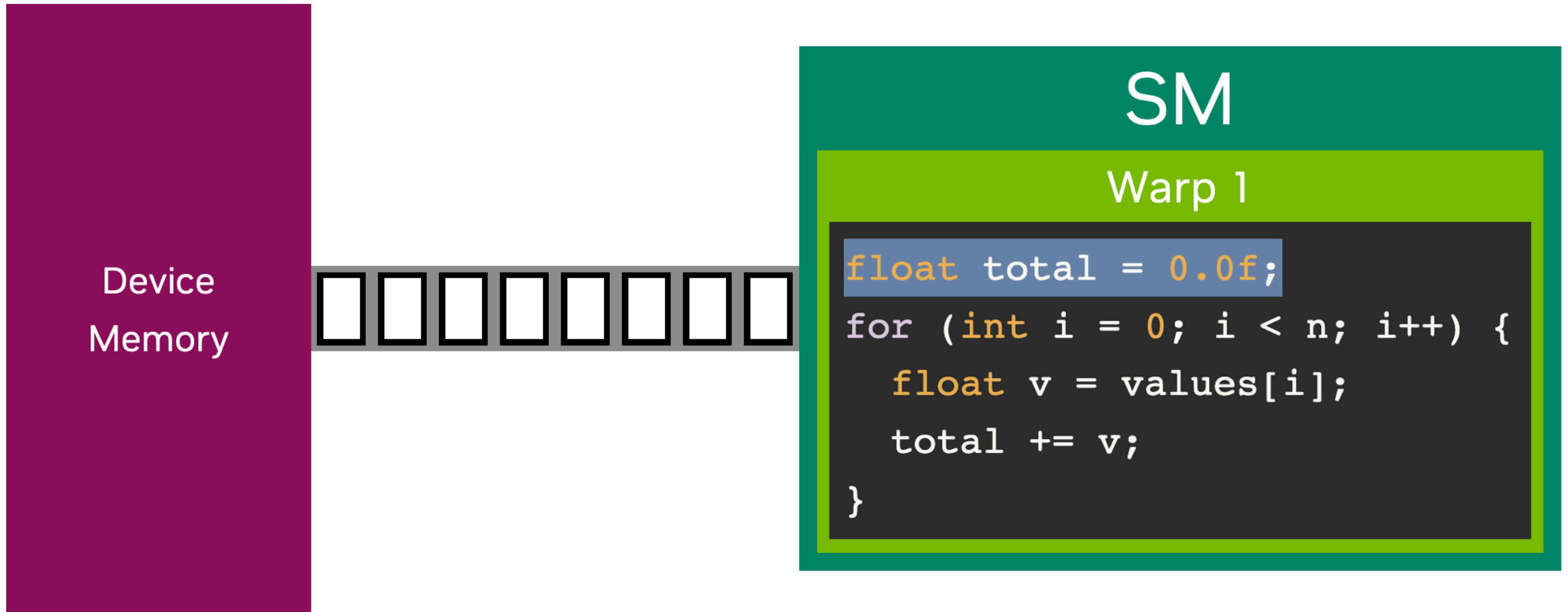


Bytes In Flight

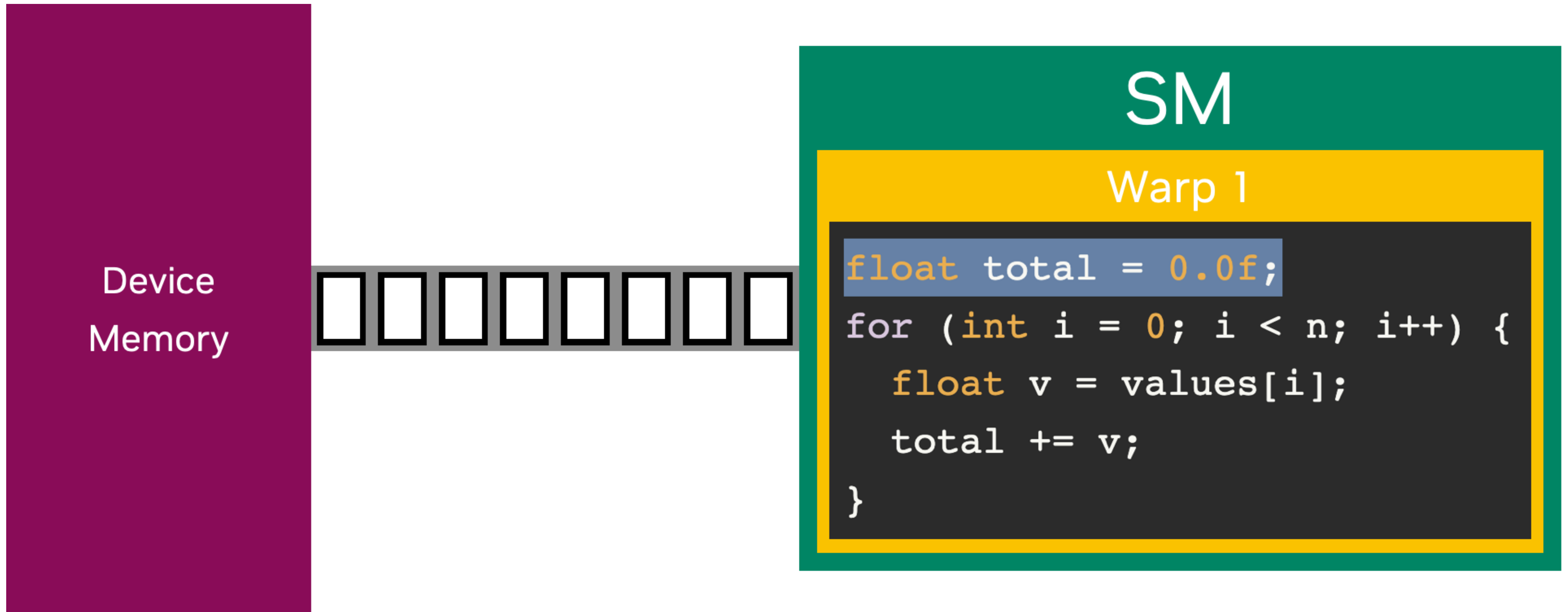


*on actual GPUs, this value is much larger (roughly 500 - 1000 cycles)

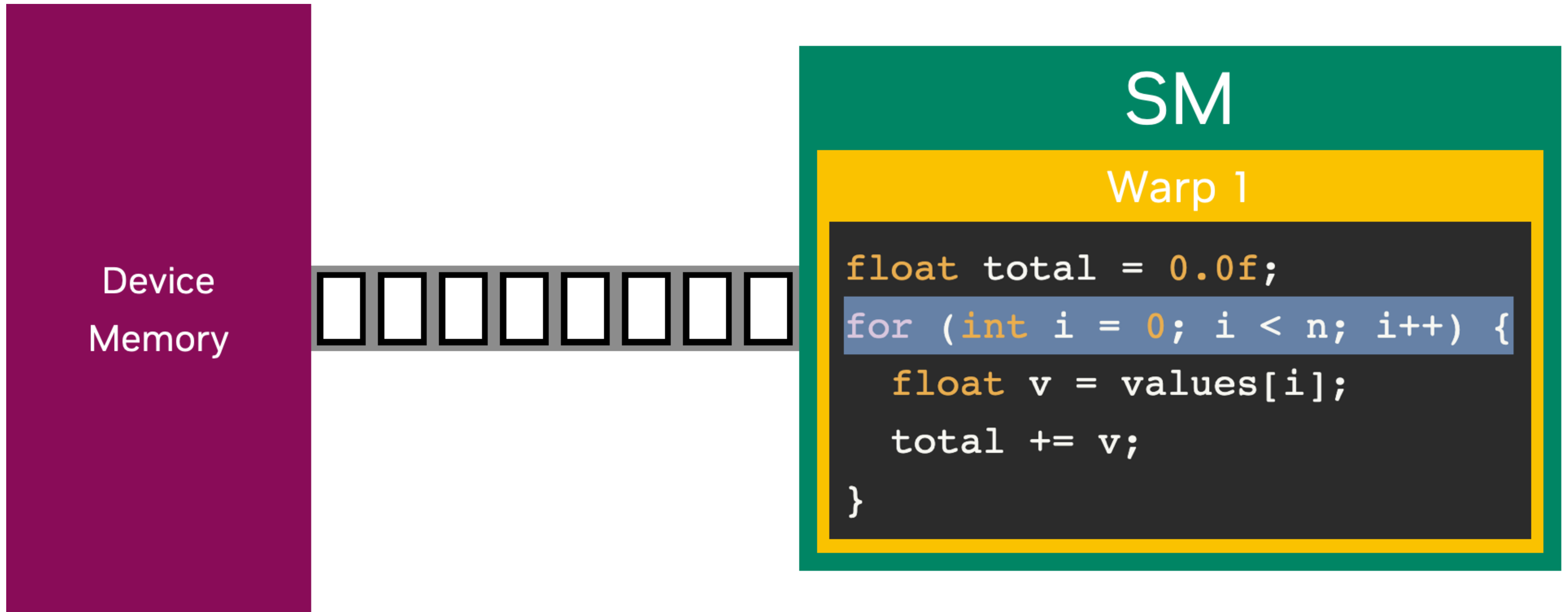
Bytes In Flight



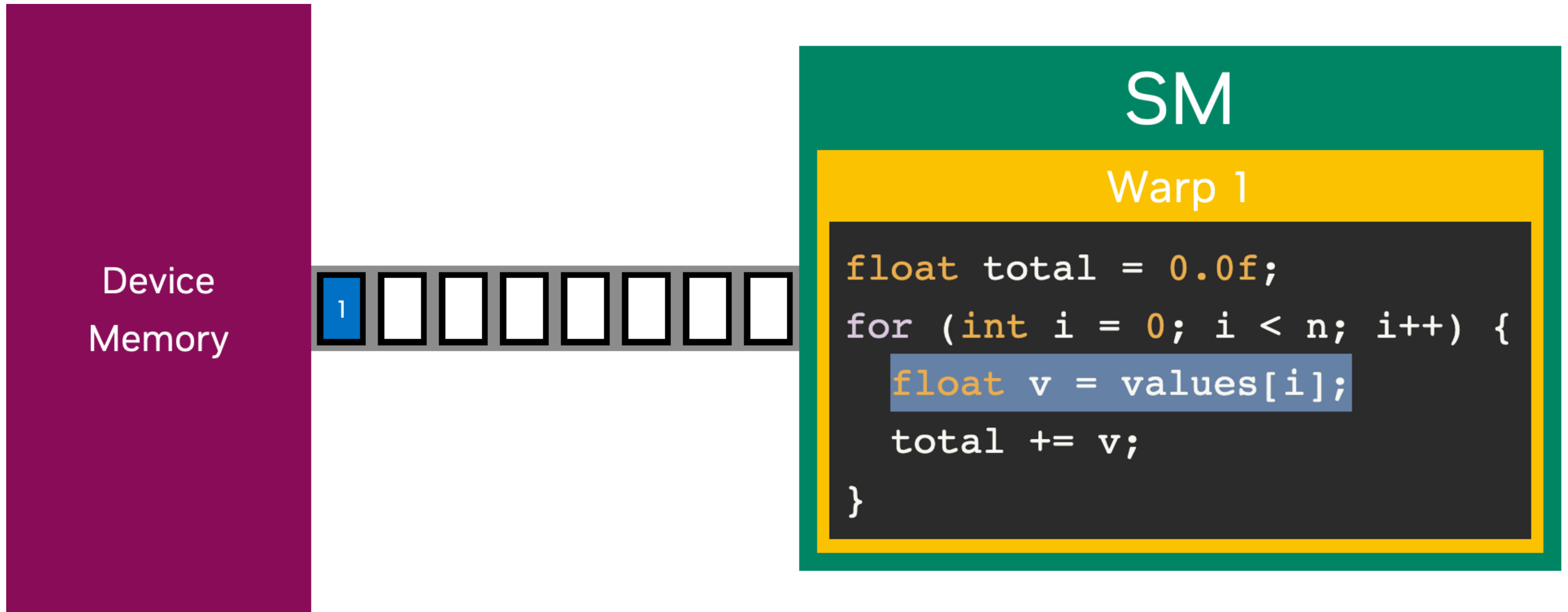
Bytes In Flight



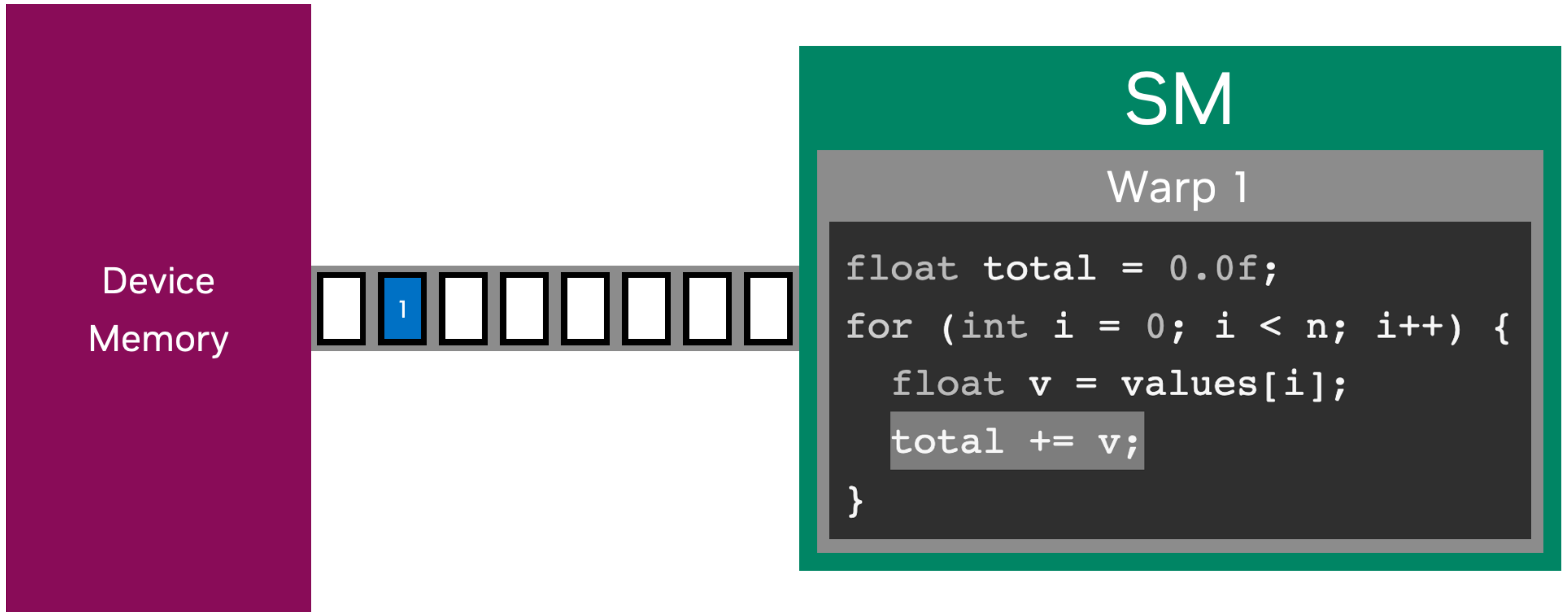
Bytes In Flight



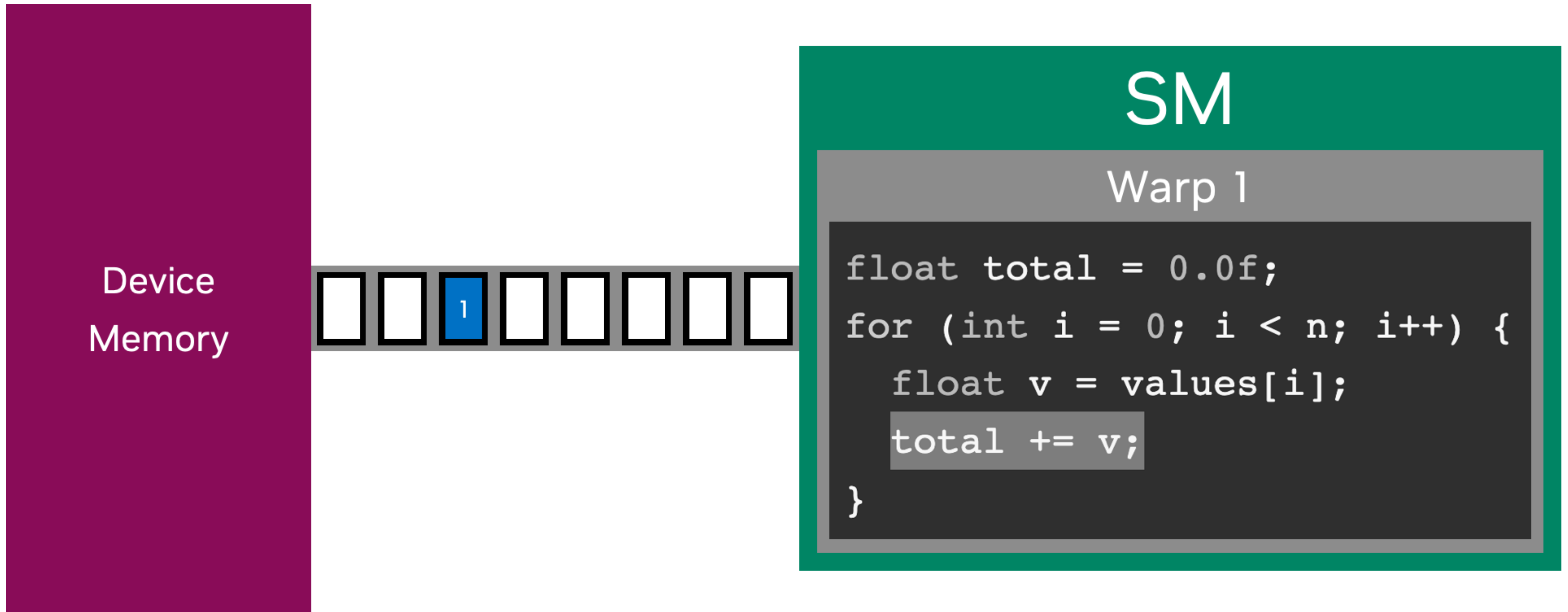
Bytes In Flight



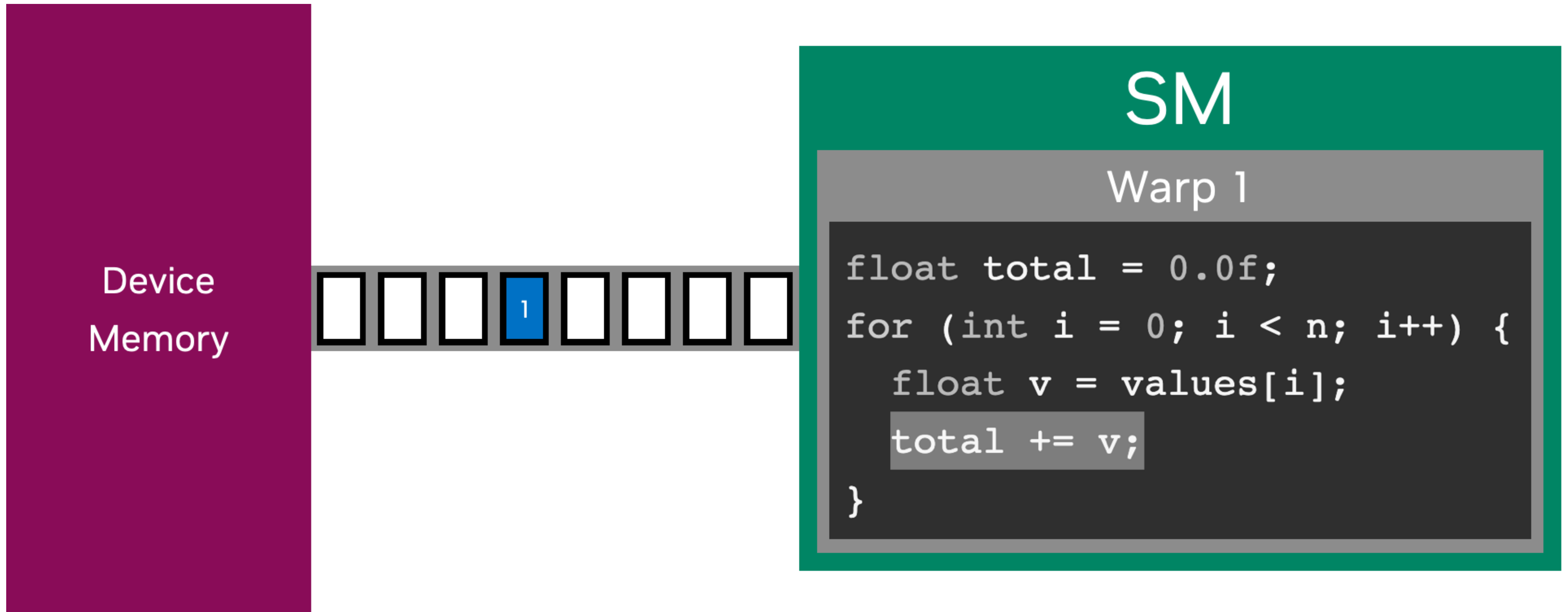
Bytes In Flight



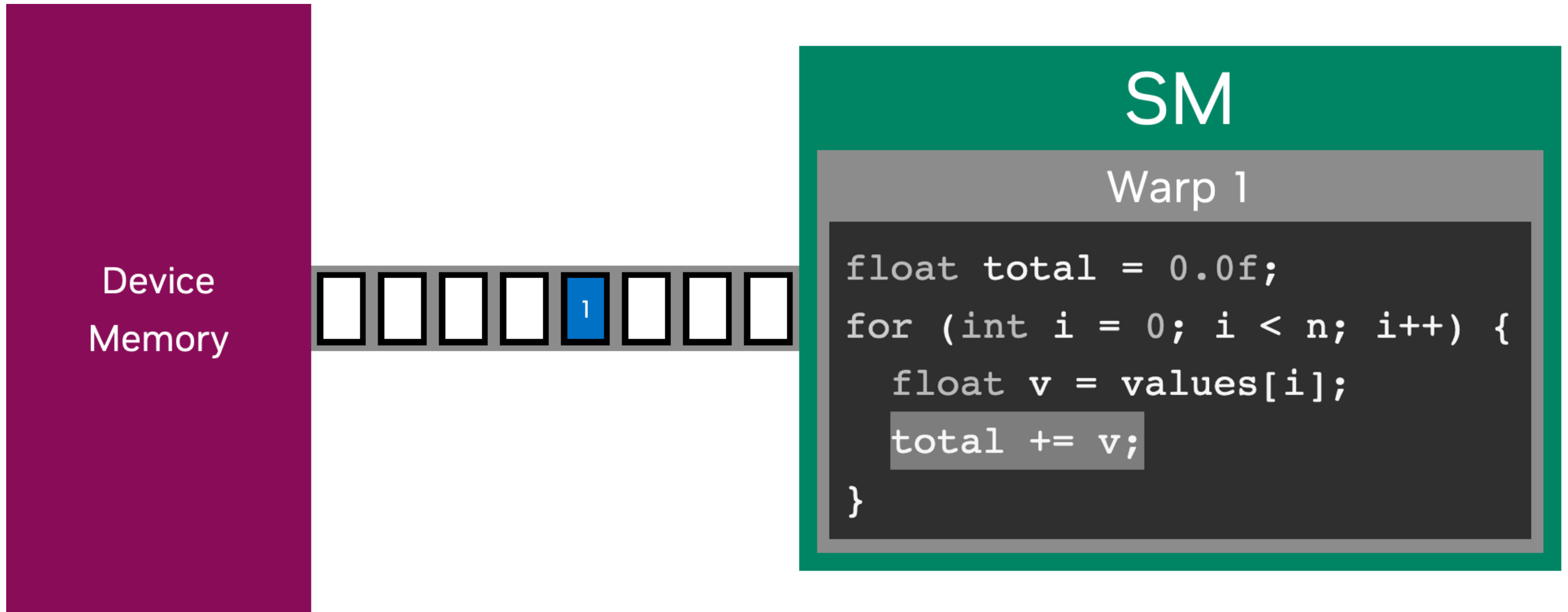
Bytes In Flight



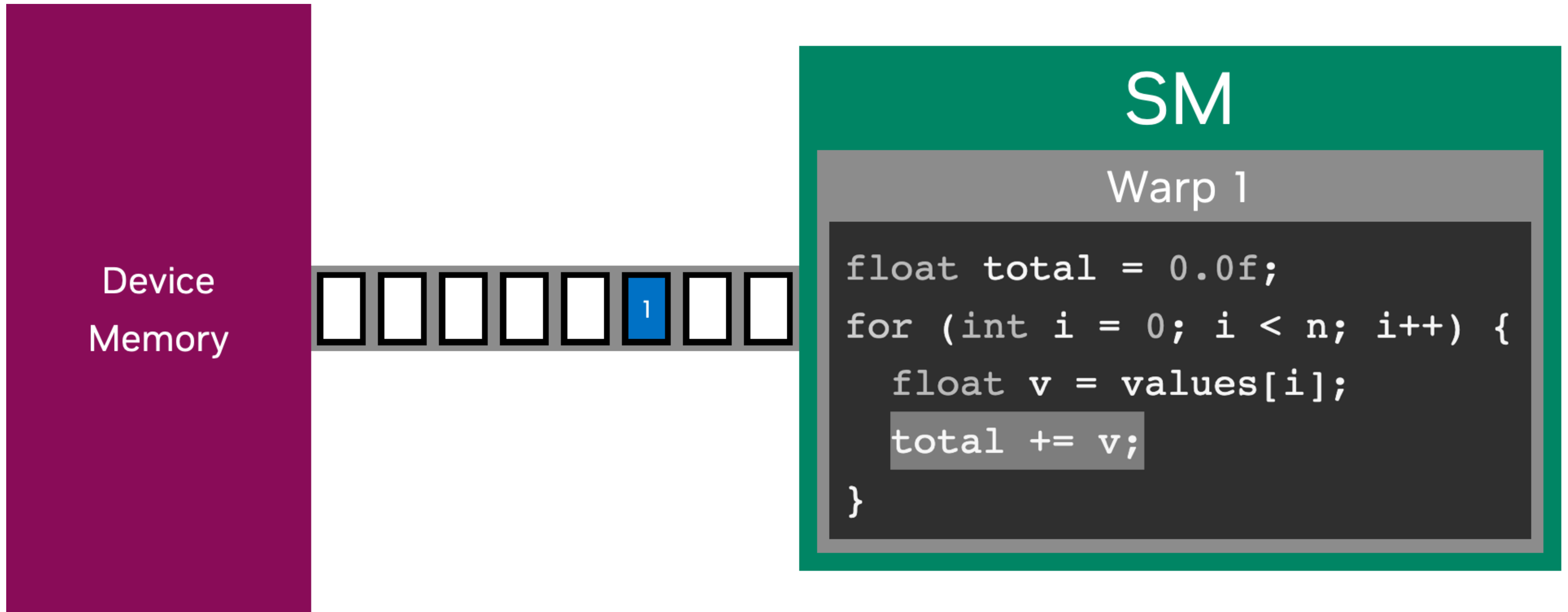
Bytes In Flight



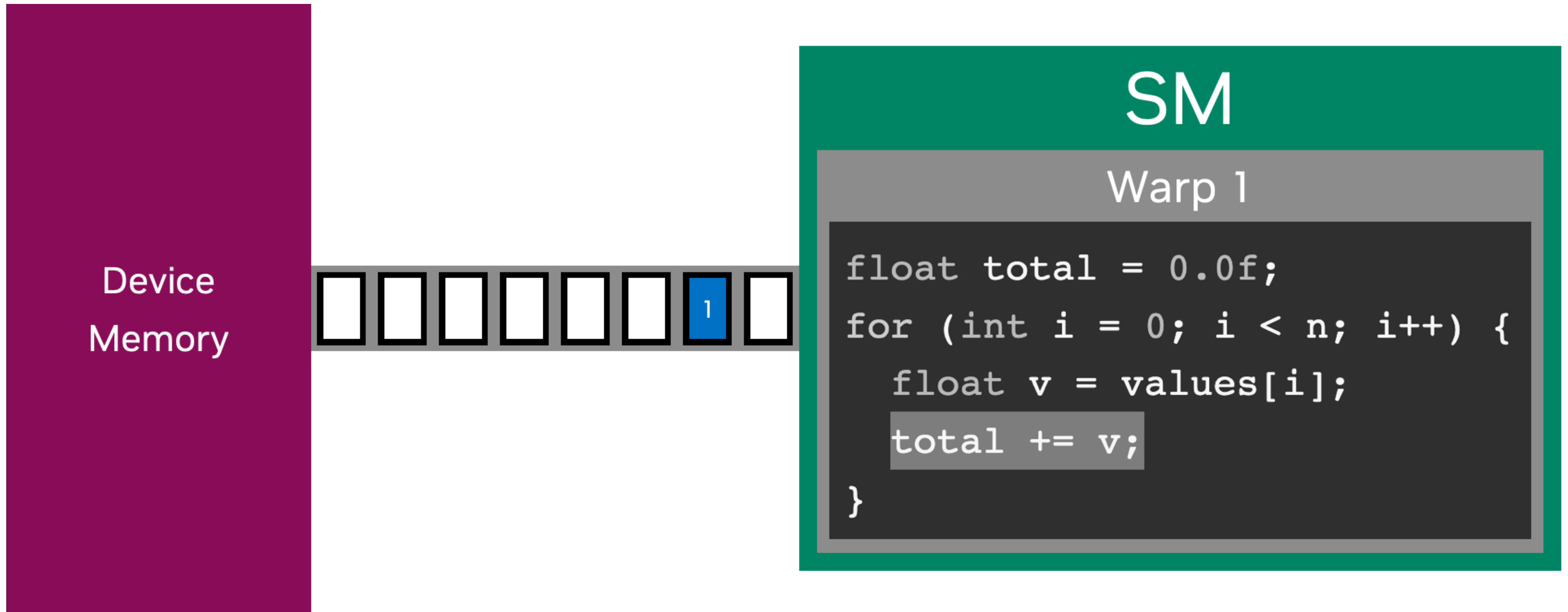
Bytes In Flight



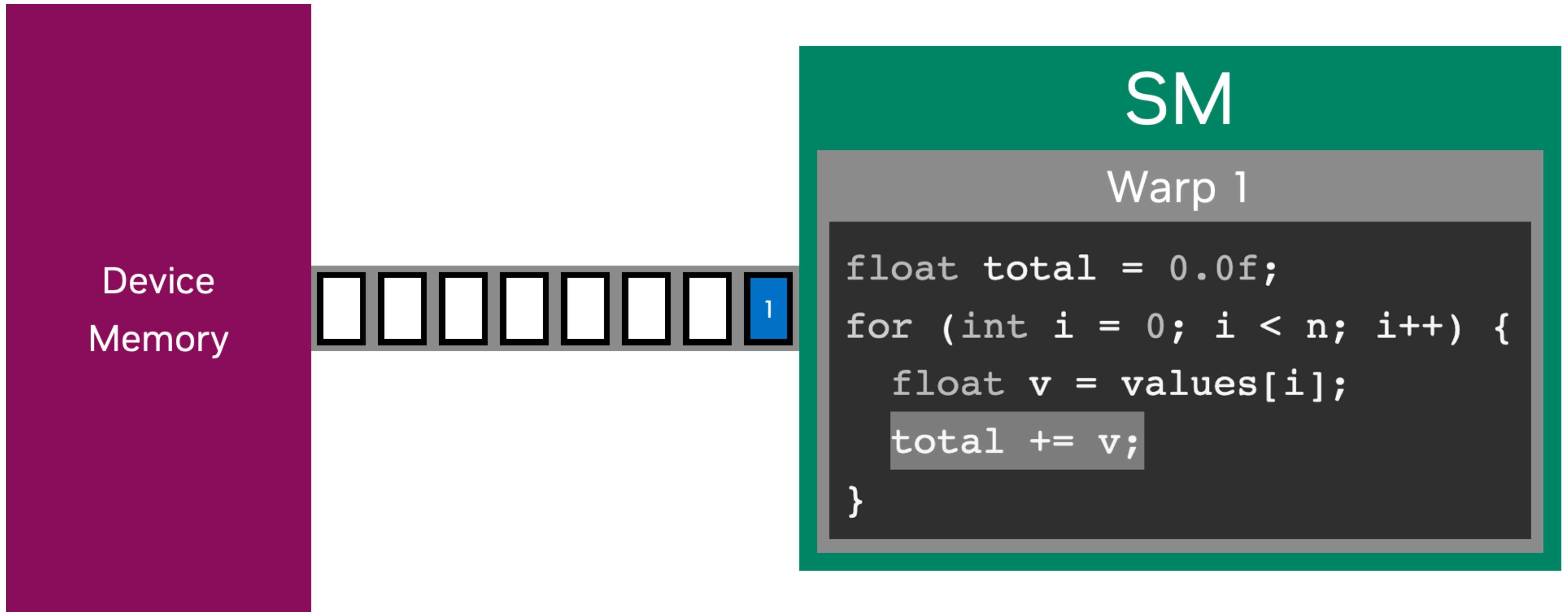
Bytes In Flight



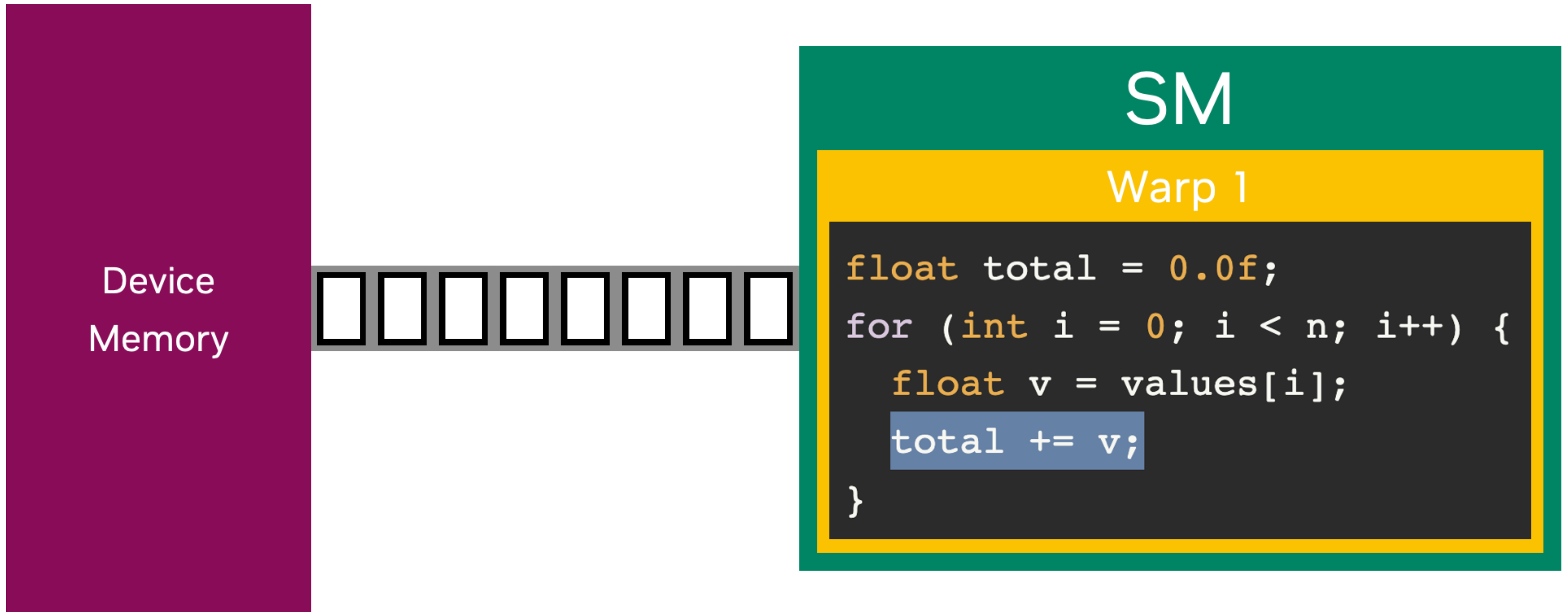
Bytes In Flight



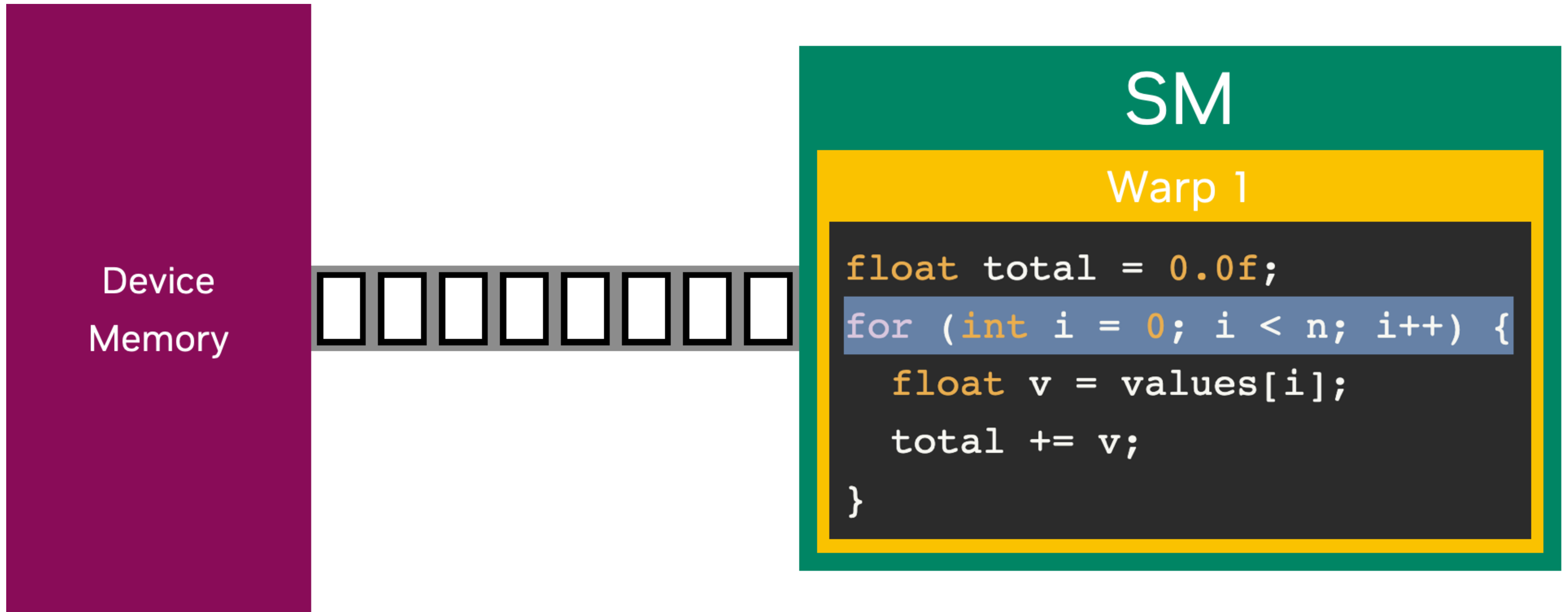
Bytes In Flight



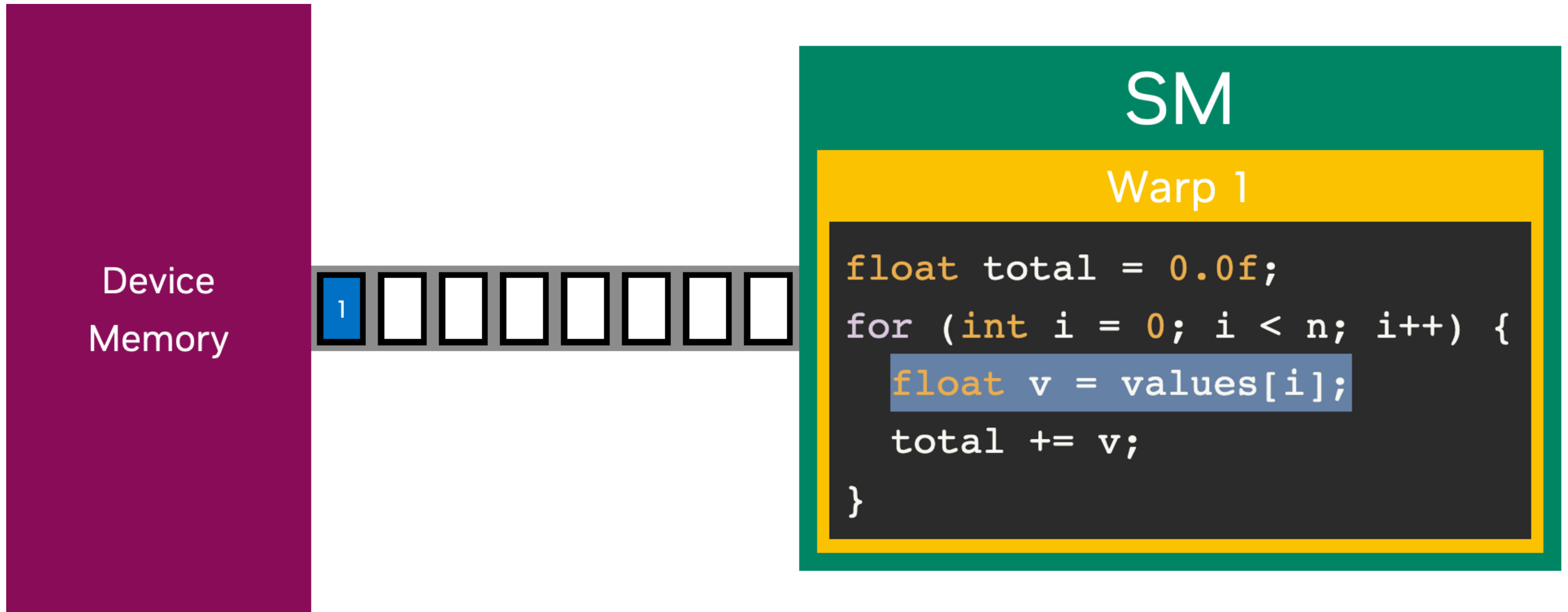
Bytes In Flight



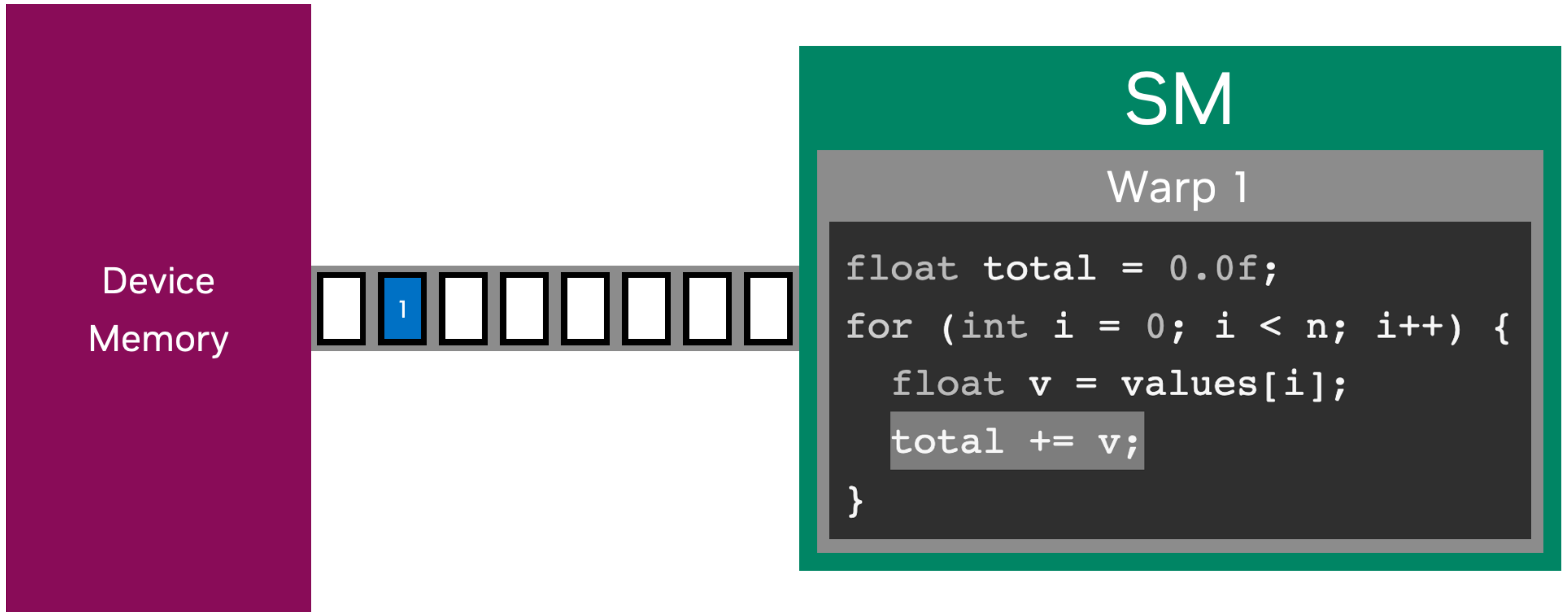
Bytes In Flight



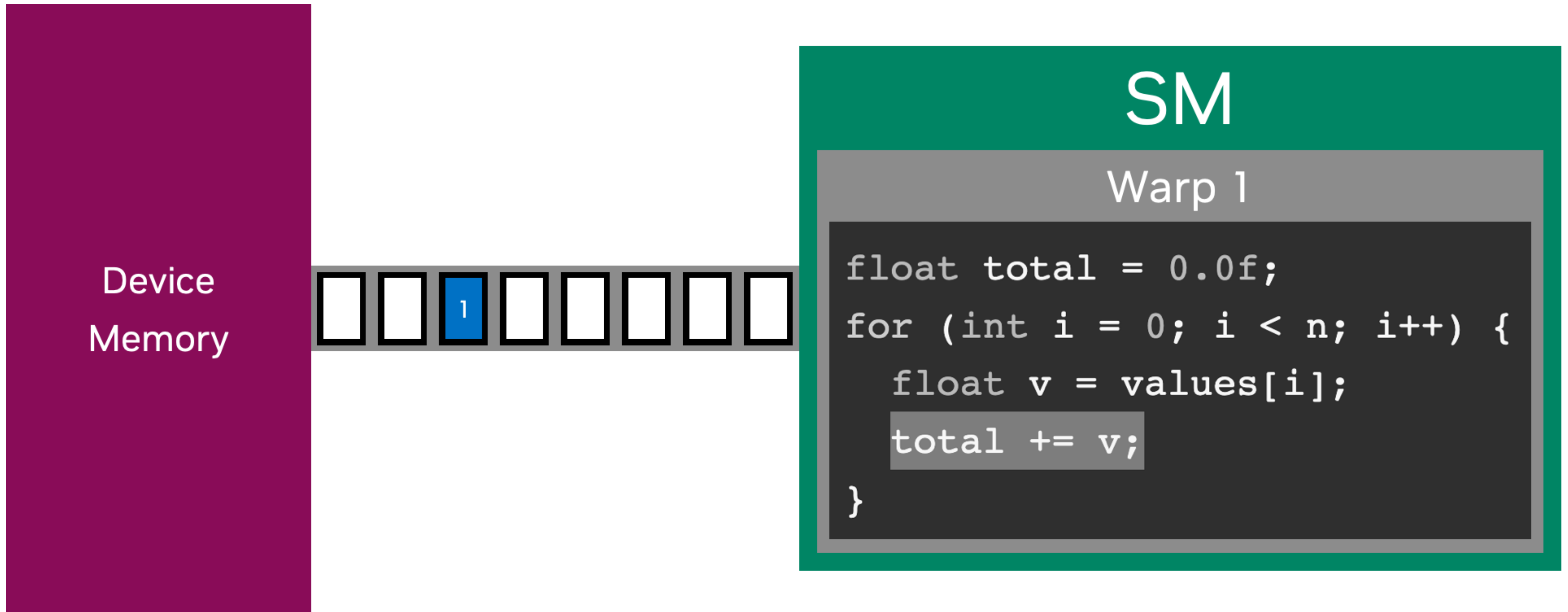
Bytes In Flight



Bytes In Flight



Bytes In Flight



Bytes In Flight

And so on ...

This example has **low** bandwidth:

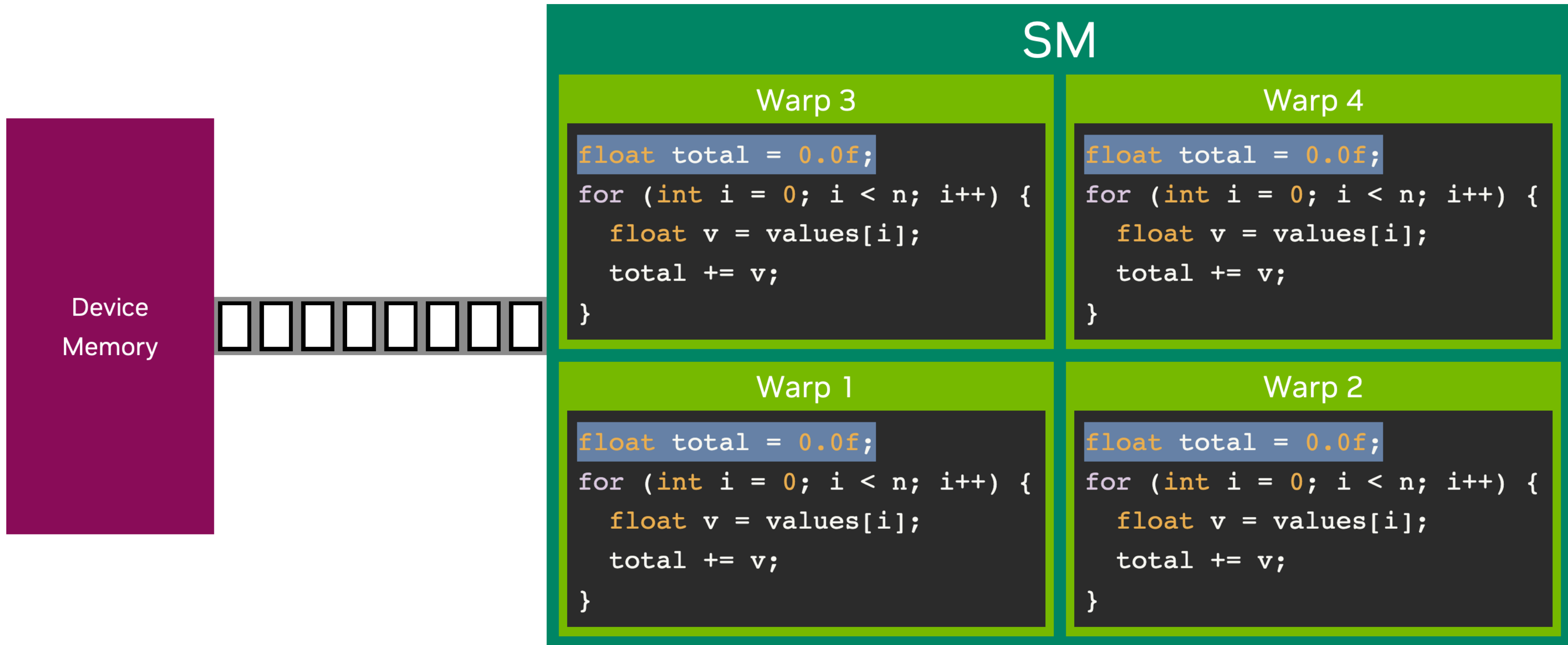
Device
Memory

- **Low** occupancy (few warps / SM)
- Memory bus is **almost empty**

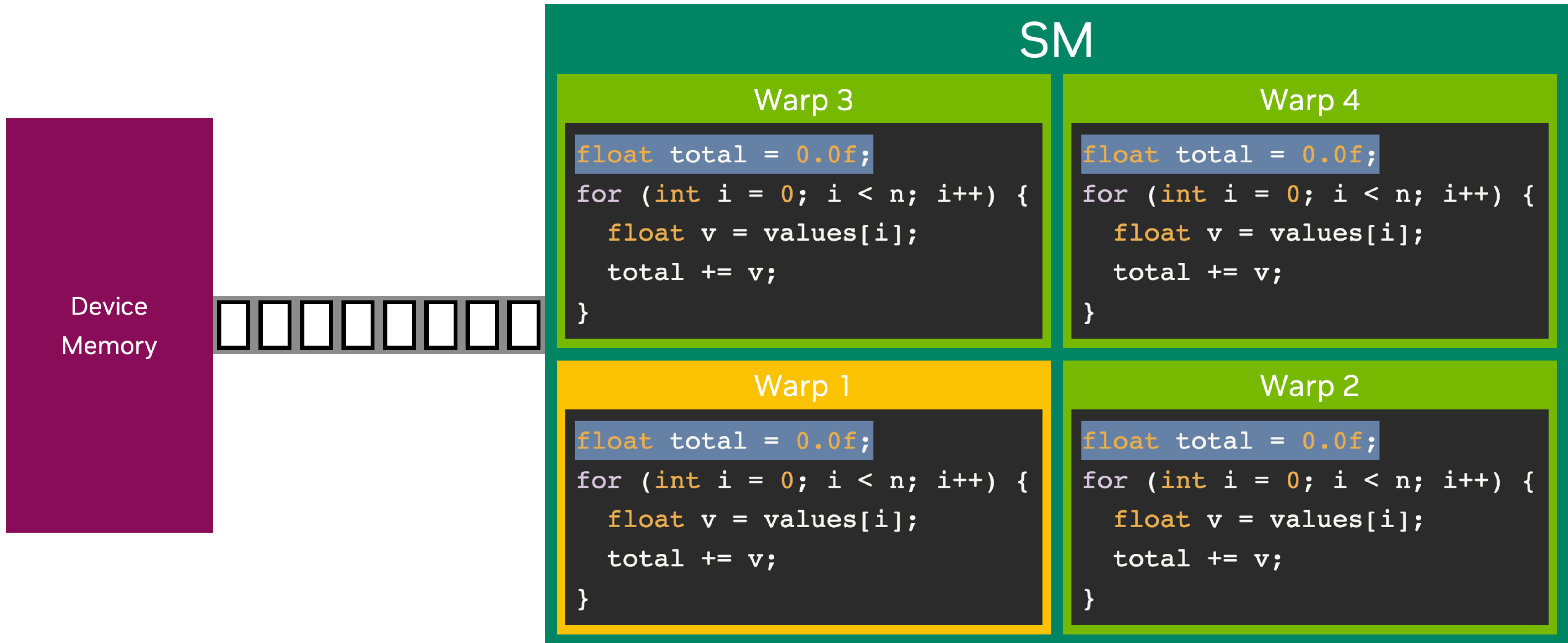
SM

```
float total = 0.0f;
for (int i = 0; i < values.size(); i++) {
    float v = values[i];
    total += v;
}
```

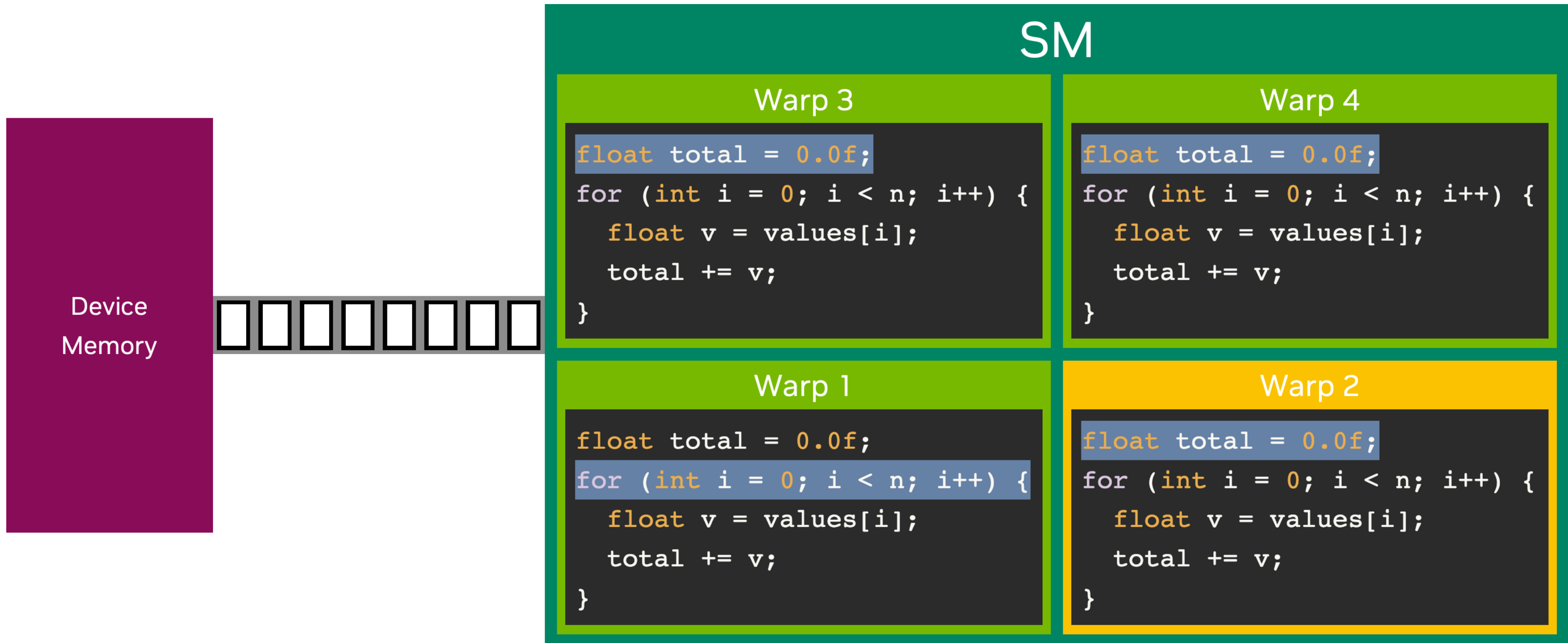
Bytes In Flight



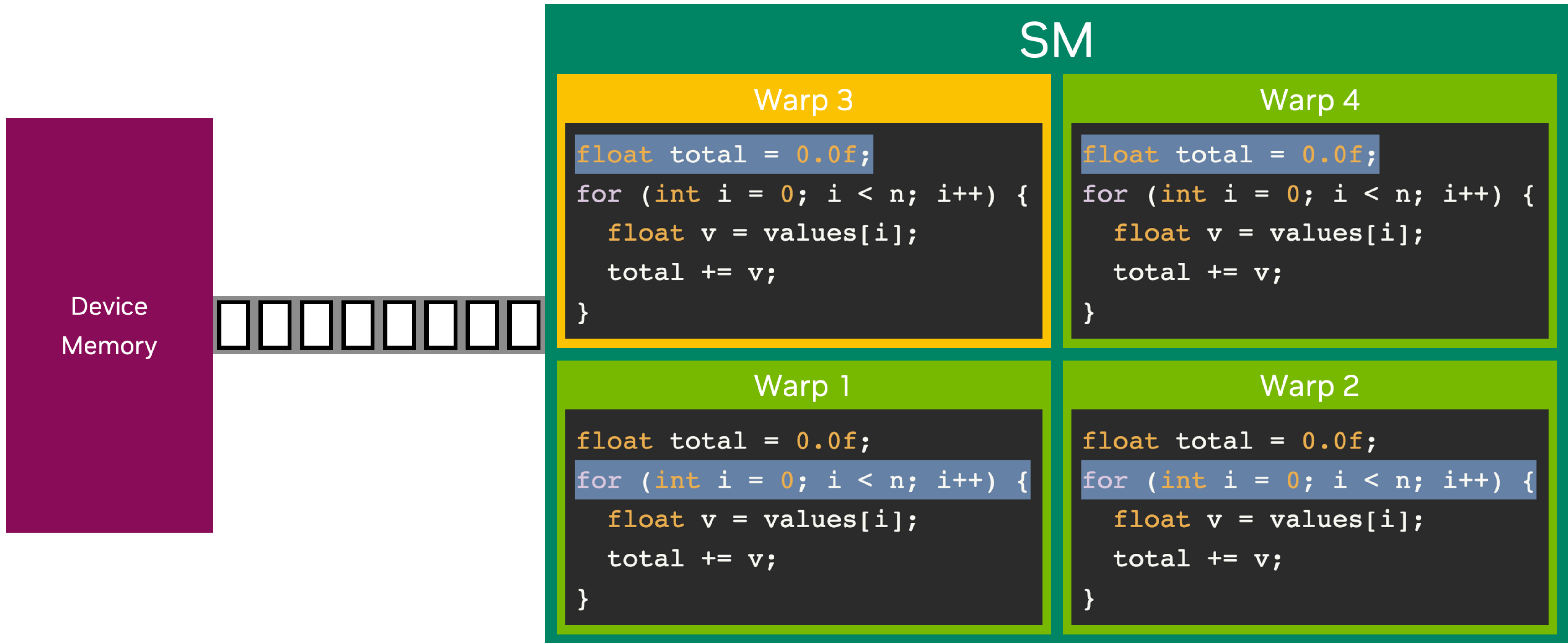
Bytes In Flight



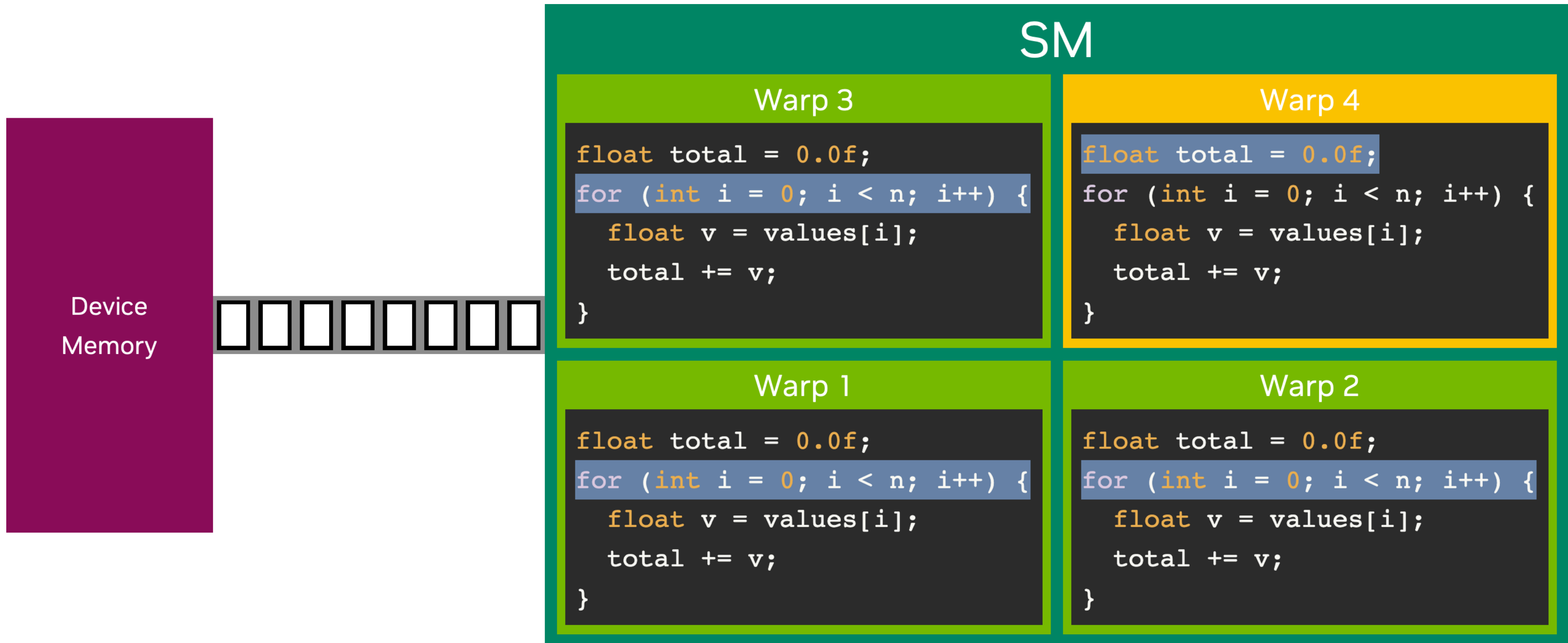
Bytes In Flight



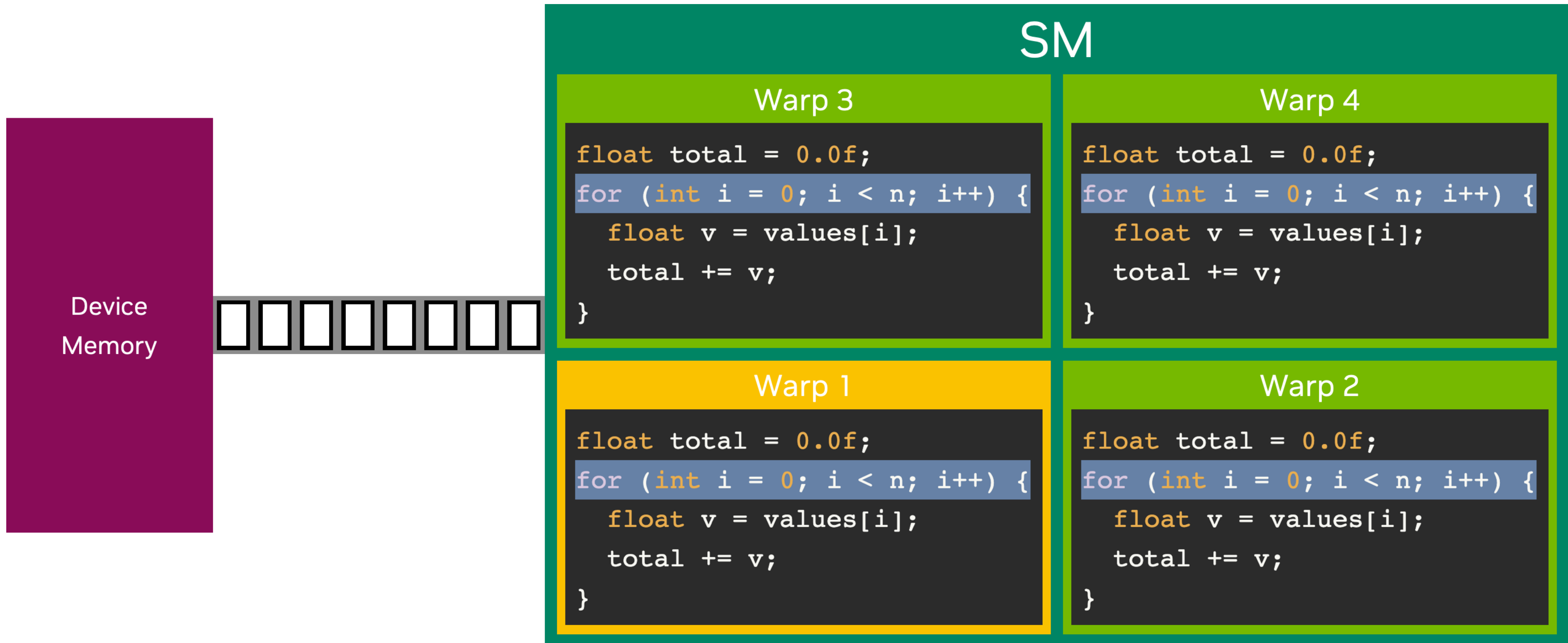
Bytes In Flight



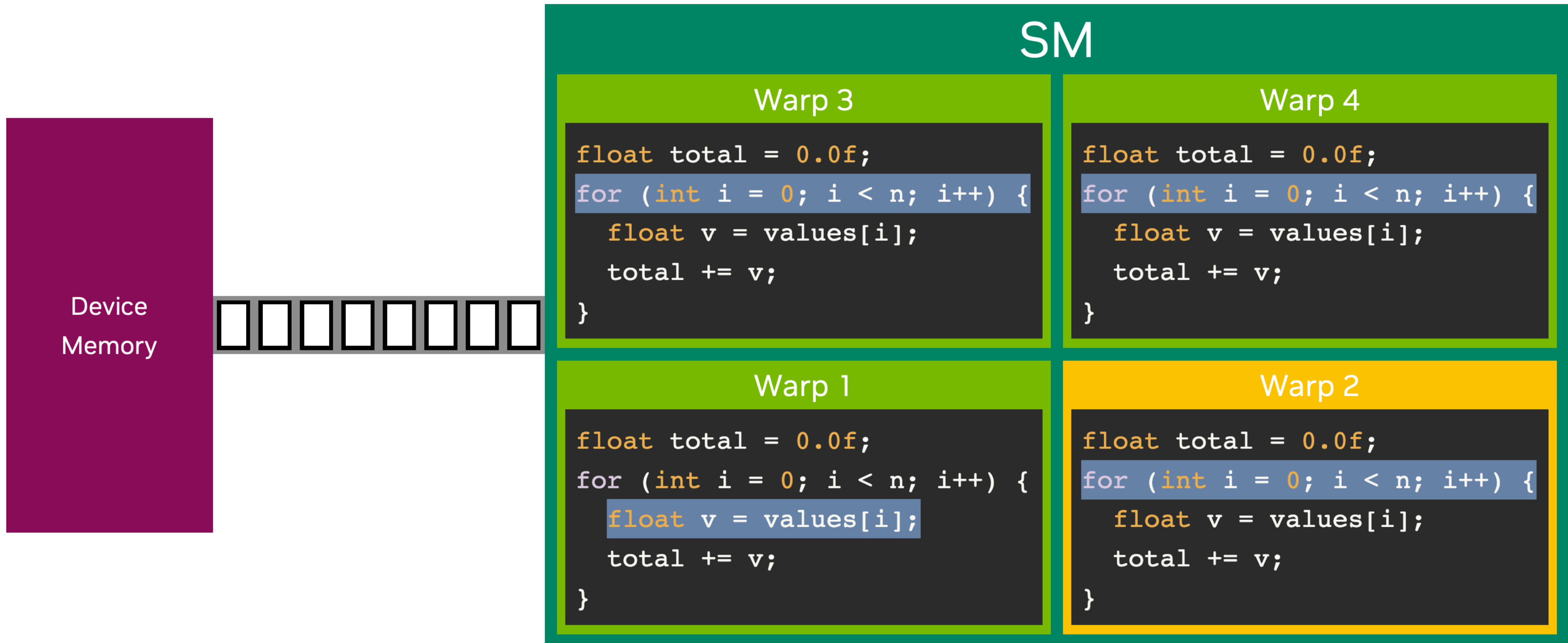
Bytes In Flight



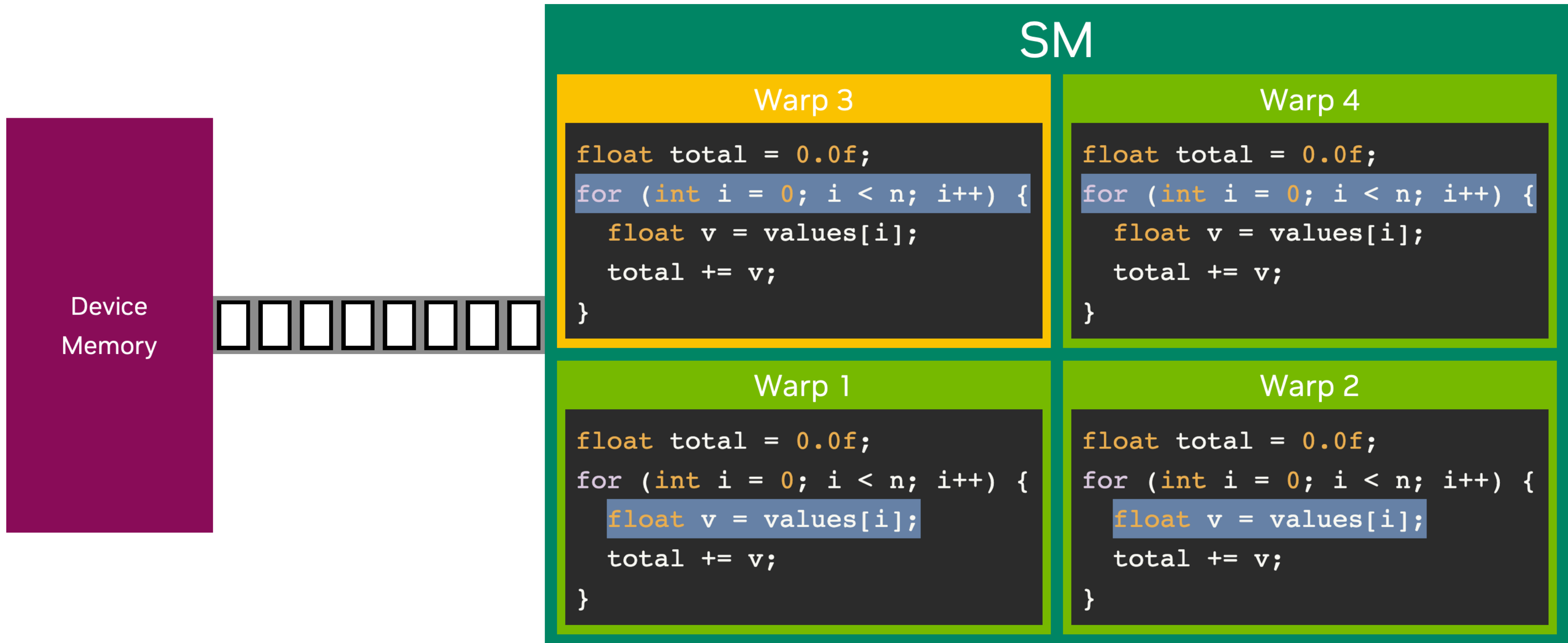
Bytes In Flight



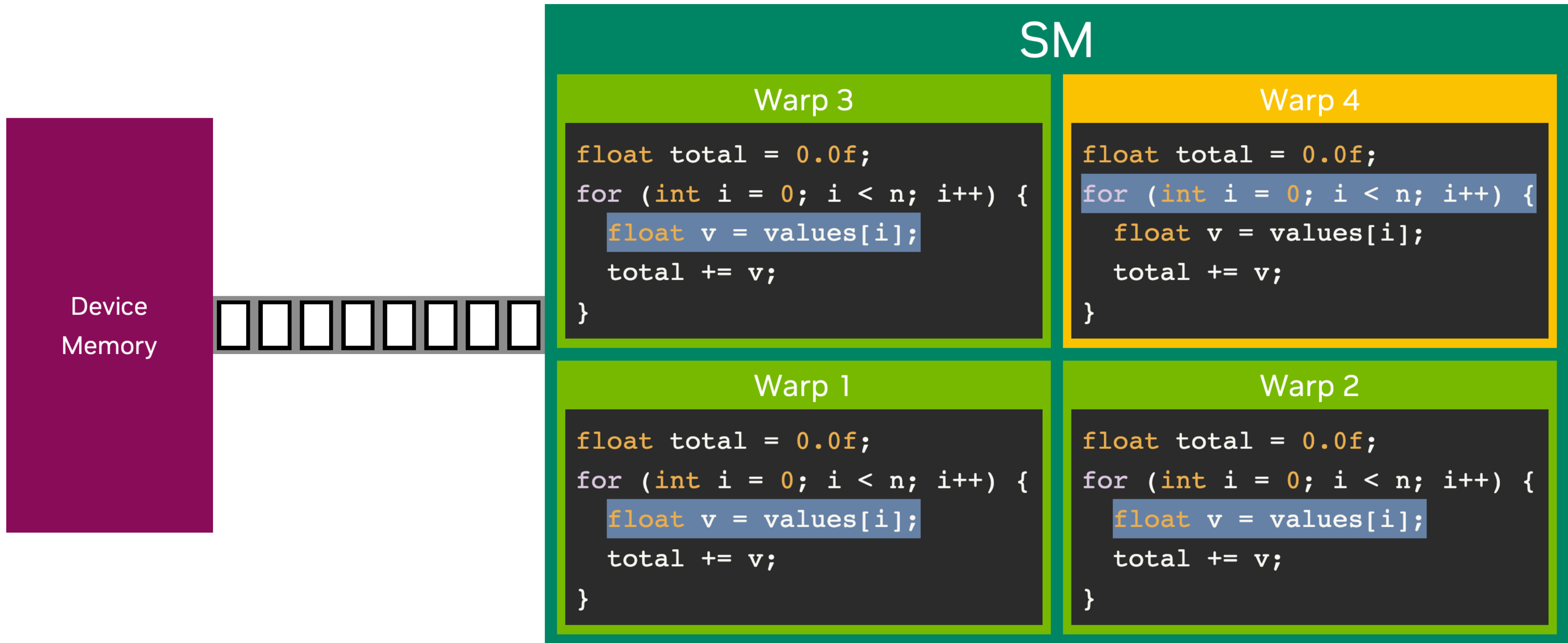
Bytes In Flight



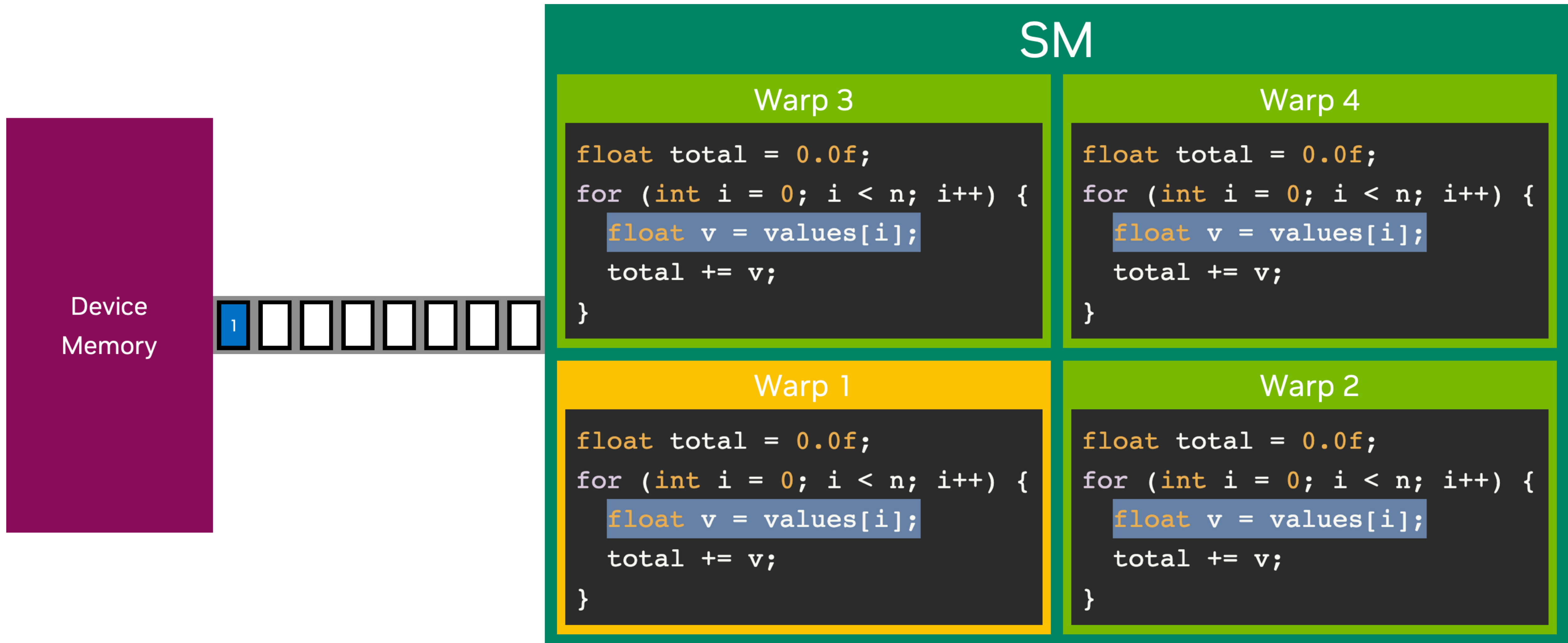
Bytes In Flight



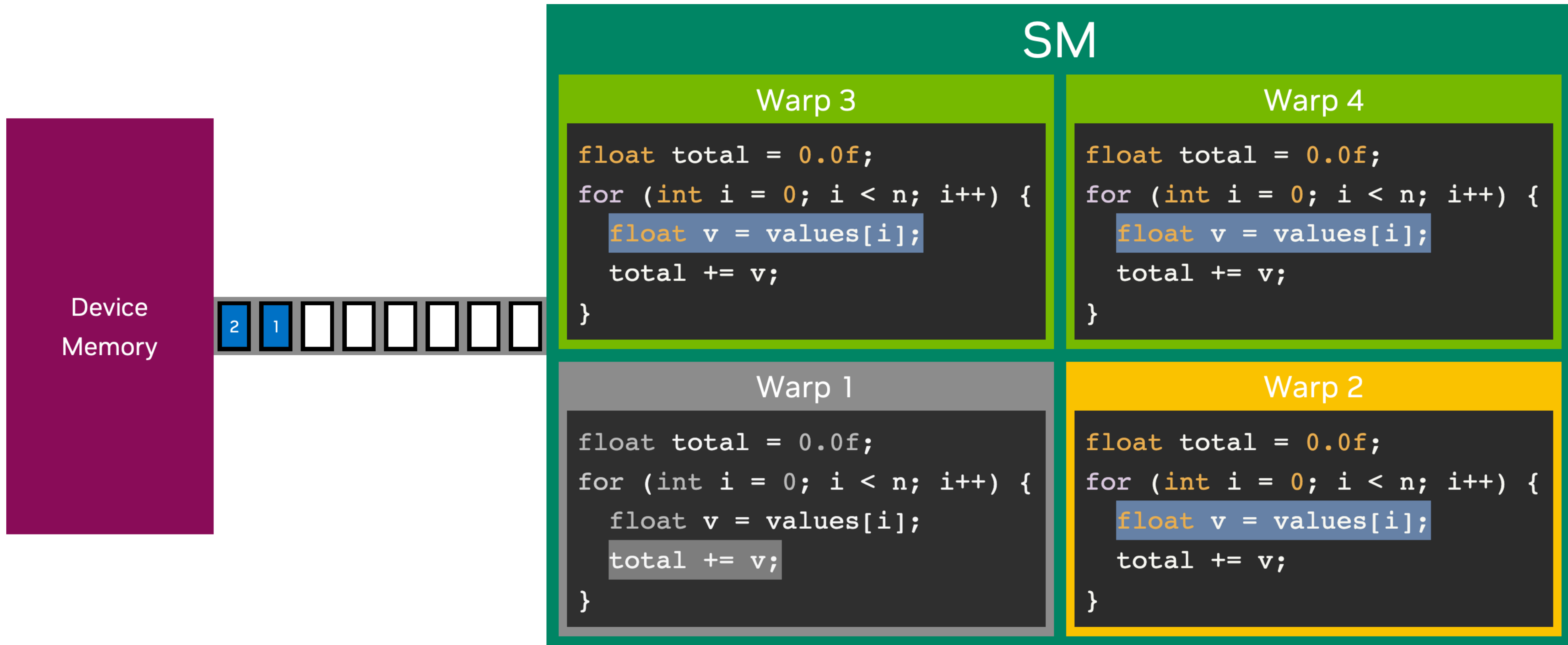
Bytes In Flight



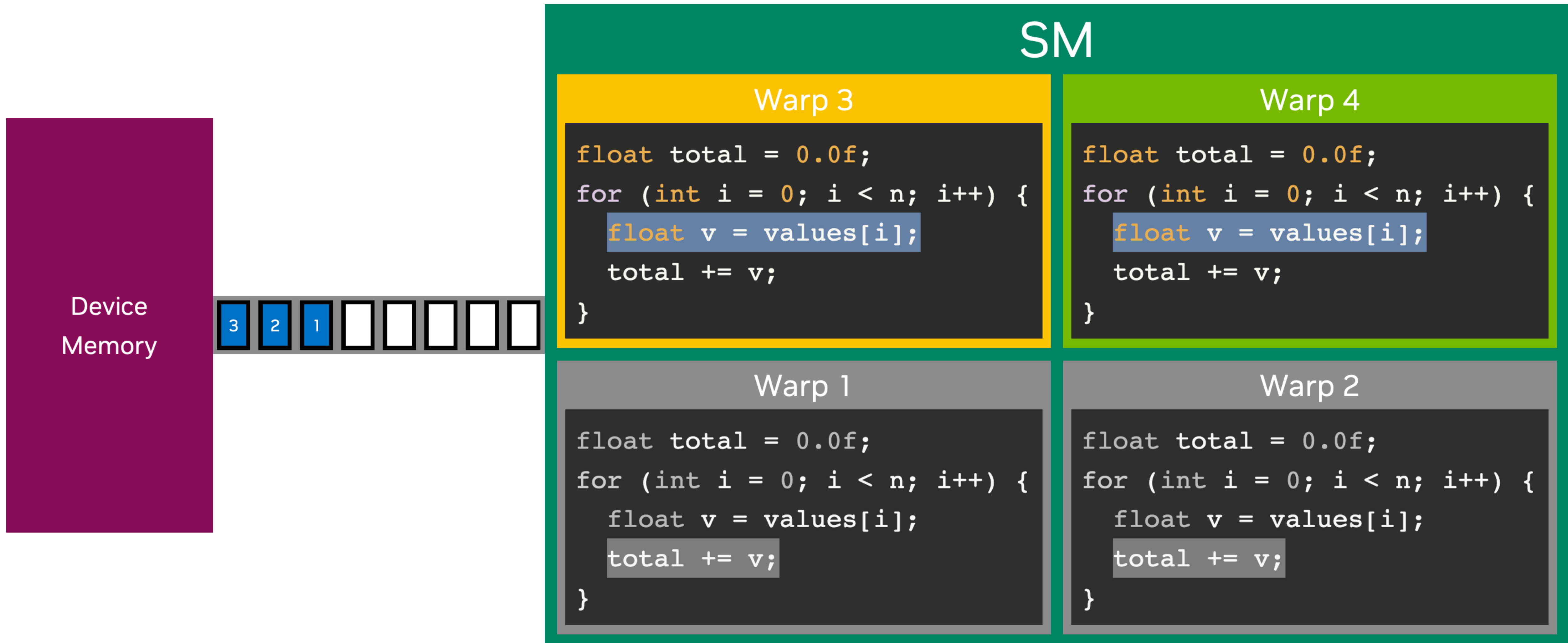
Bytes In Flight



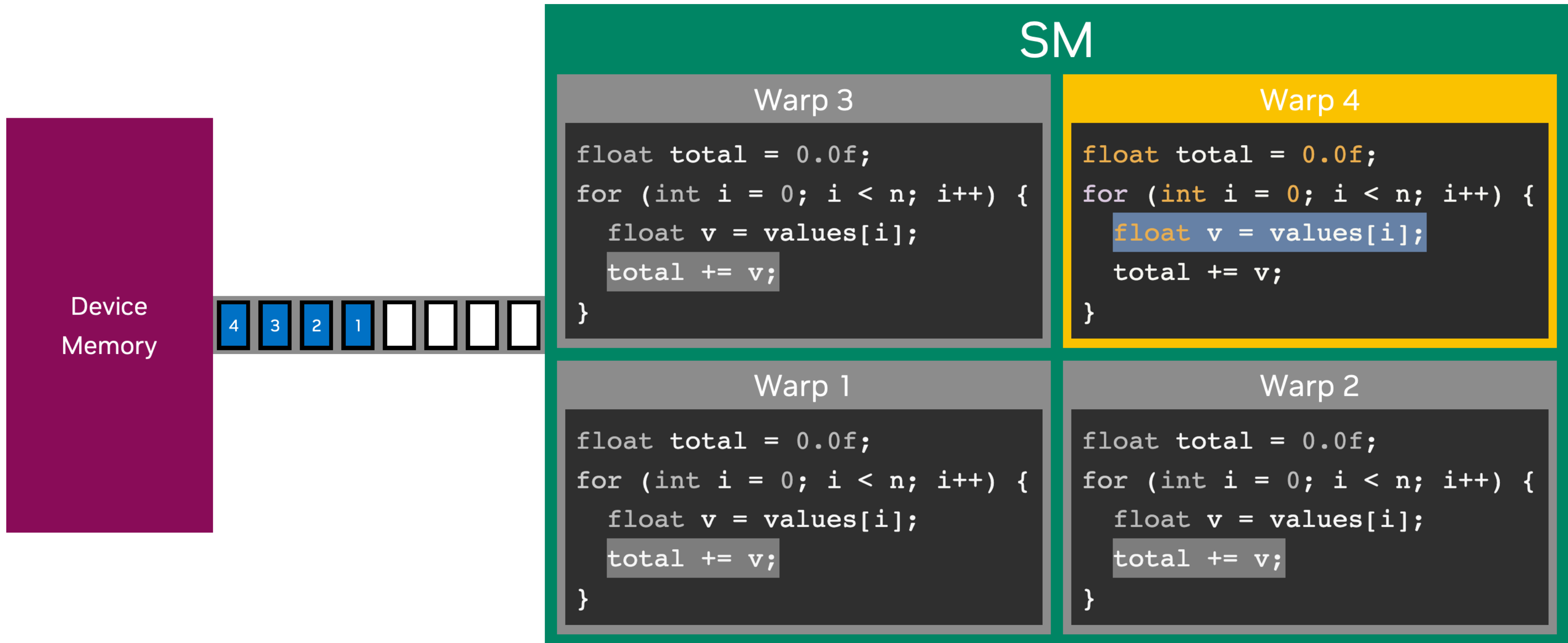
Bytes In Flight



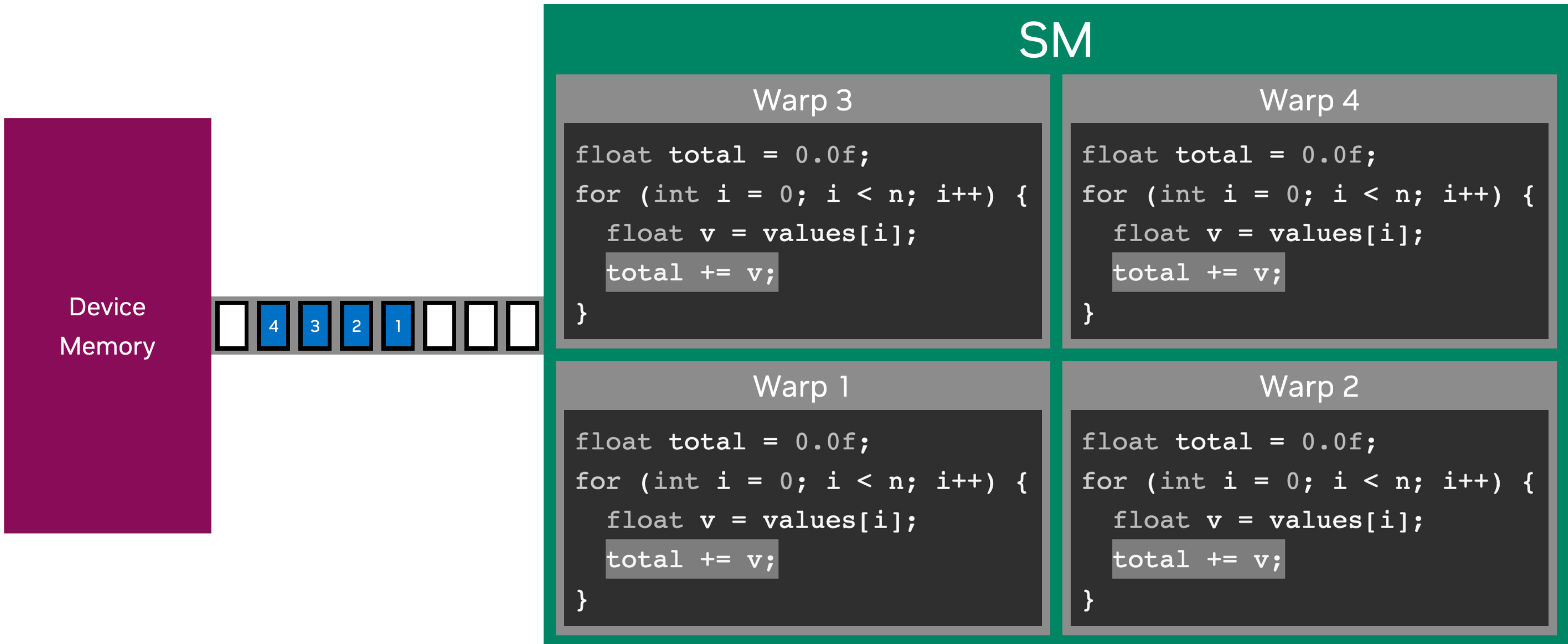
Bytes In Flight



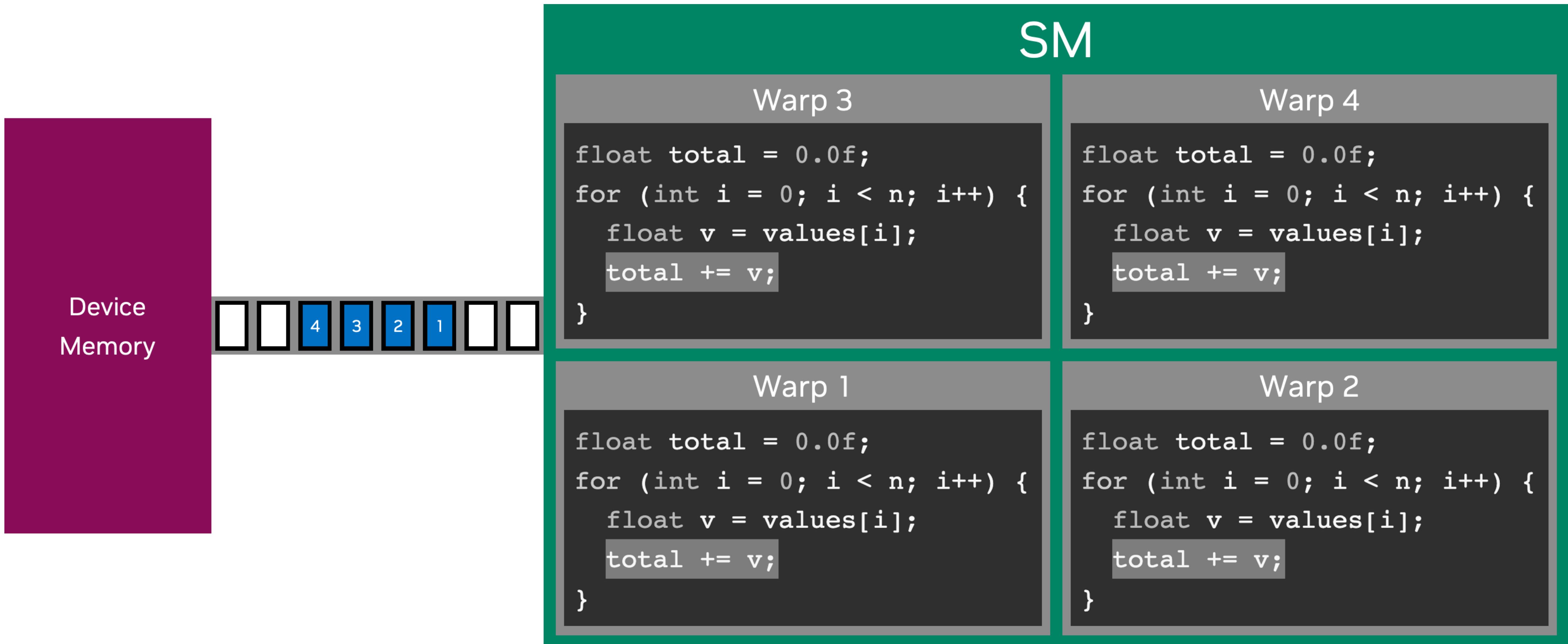
Bytes In Flight



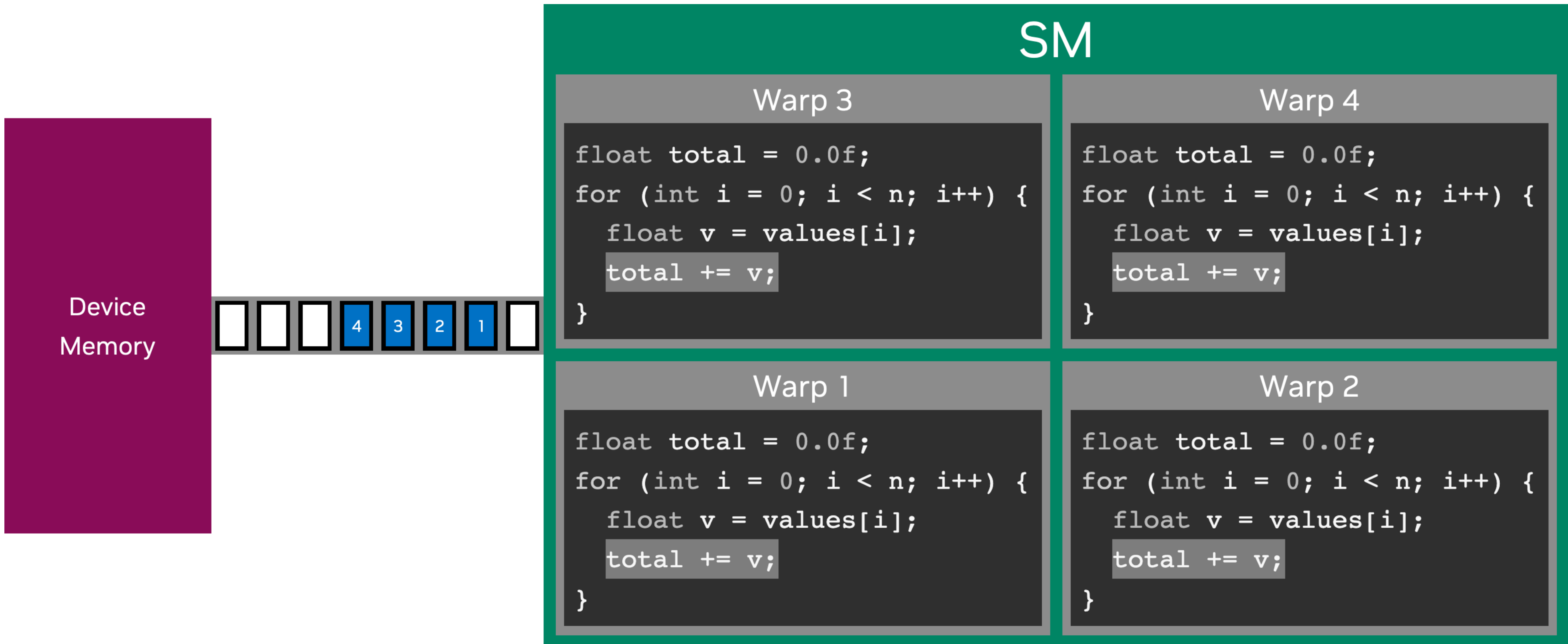
Bytes In Flight



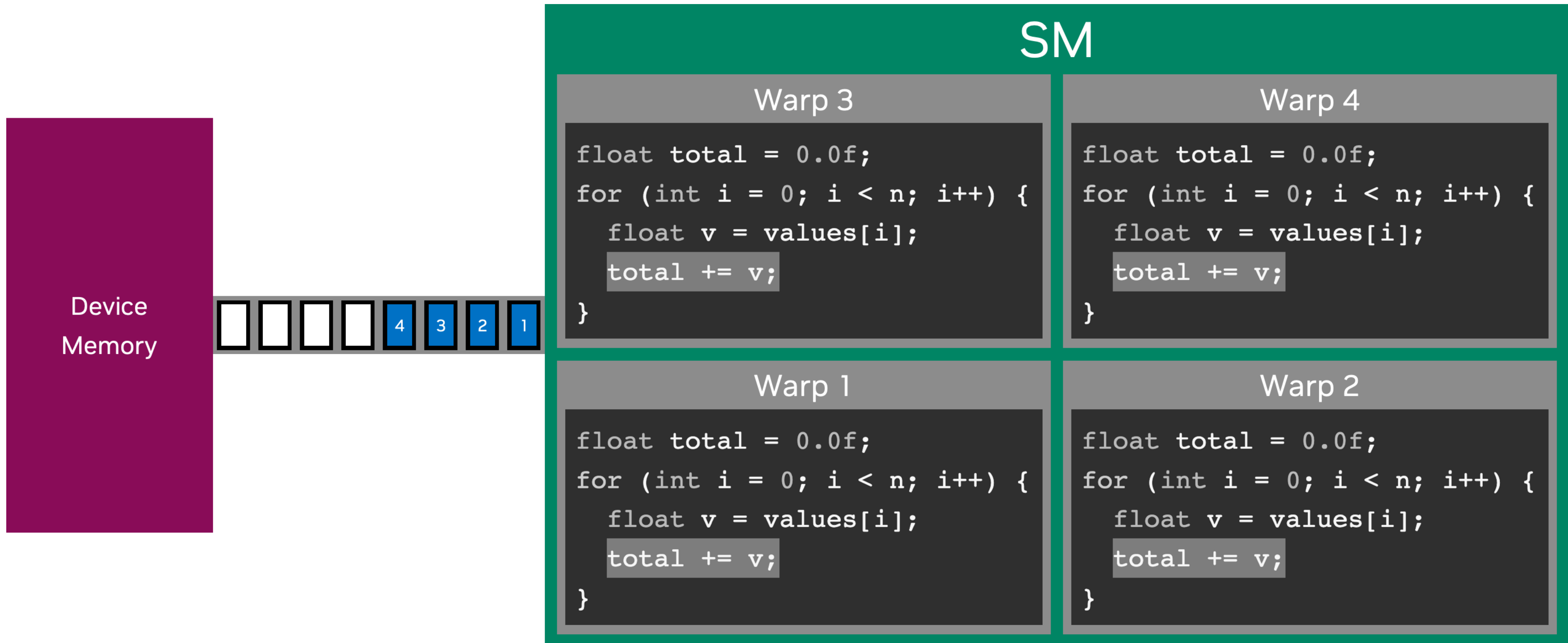
Bytes In Flight



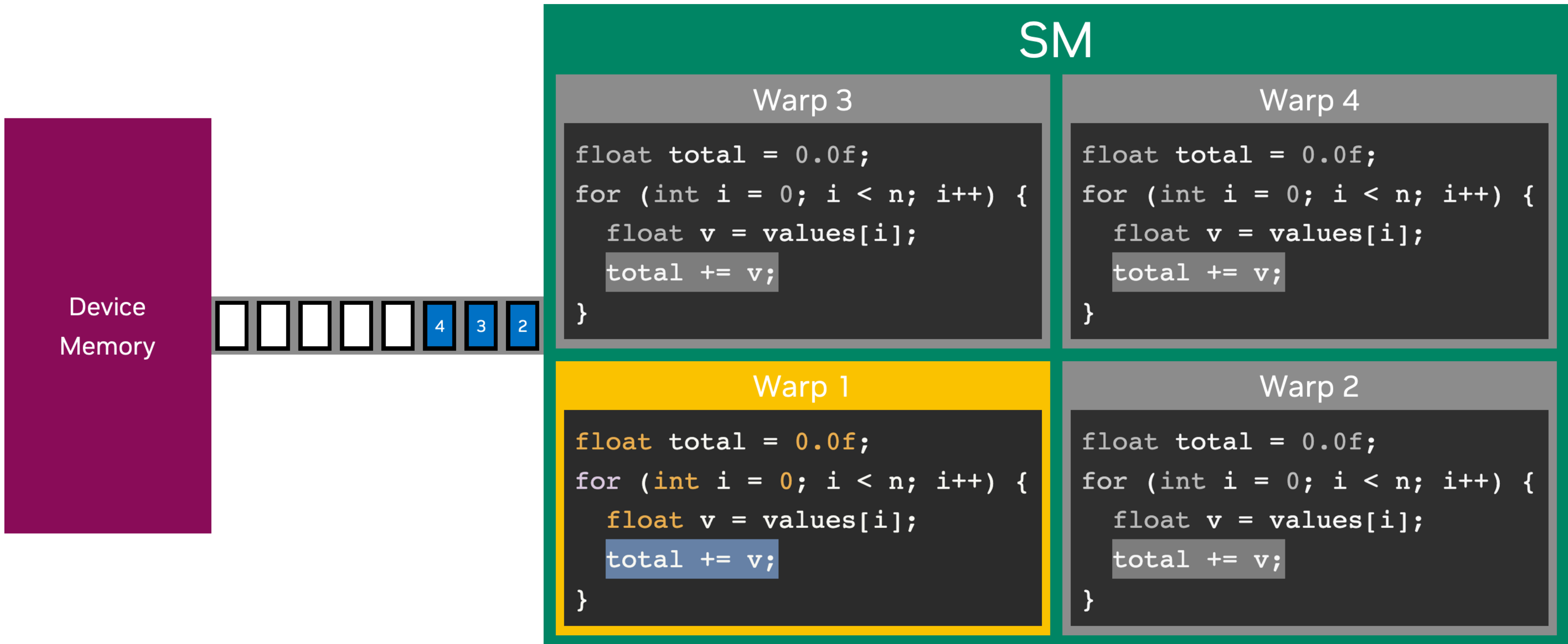
Bytes In Flight



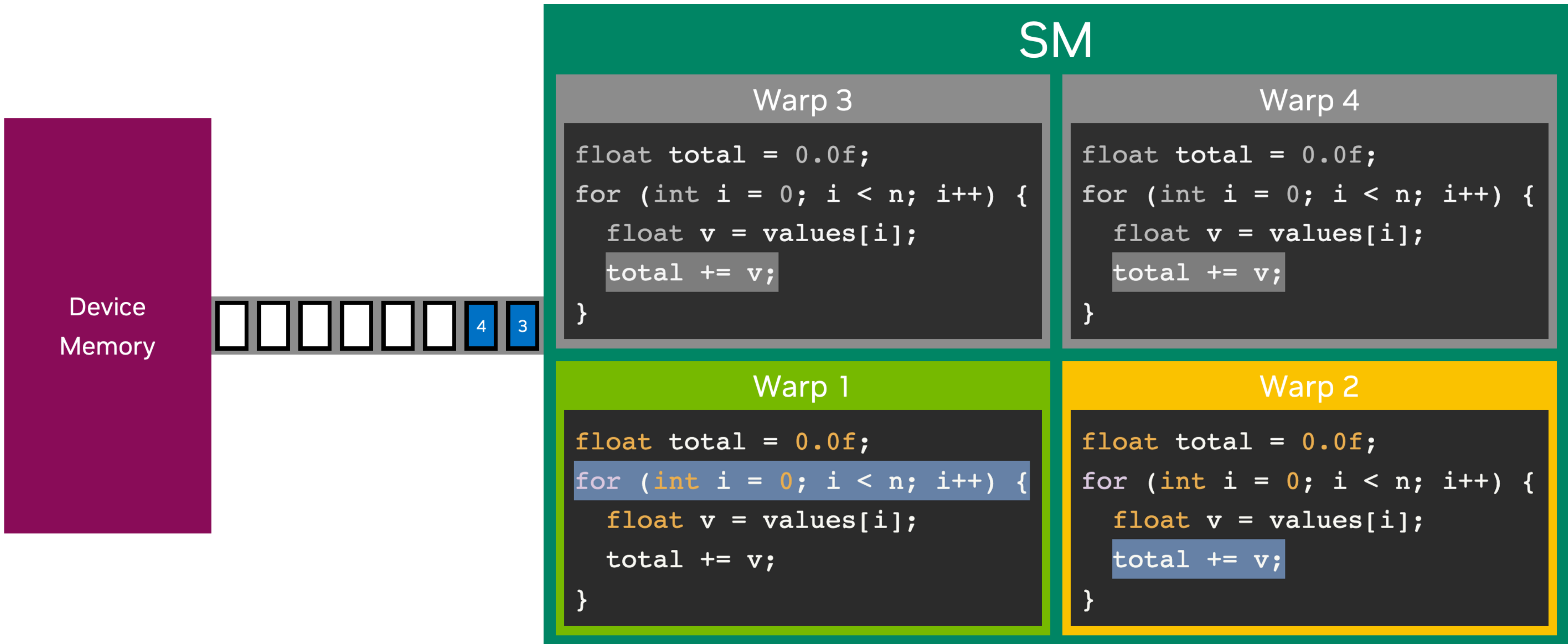
Bytes In Flight



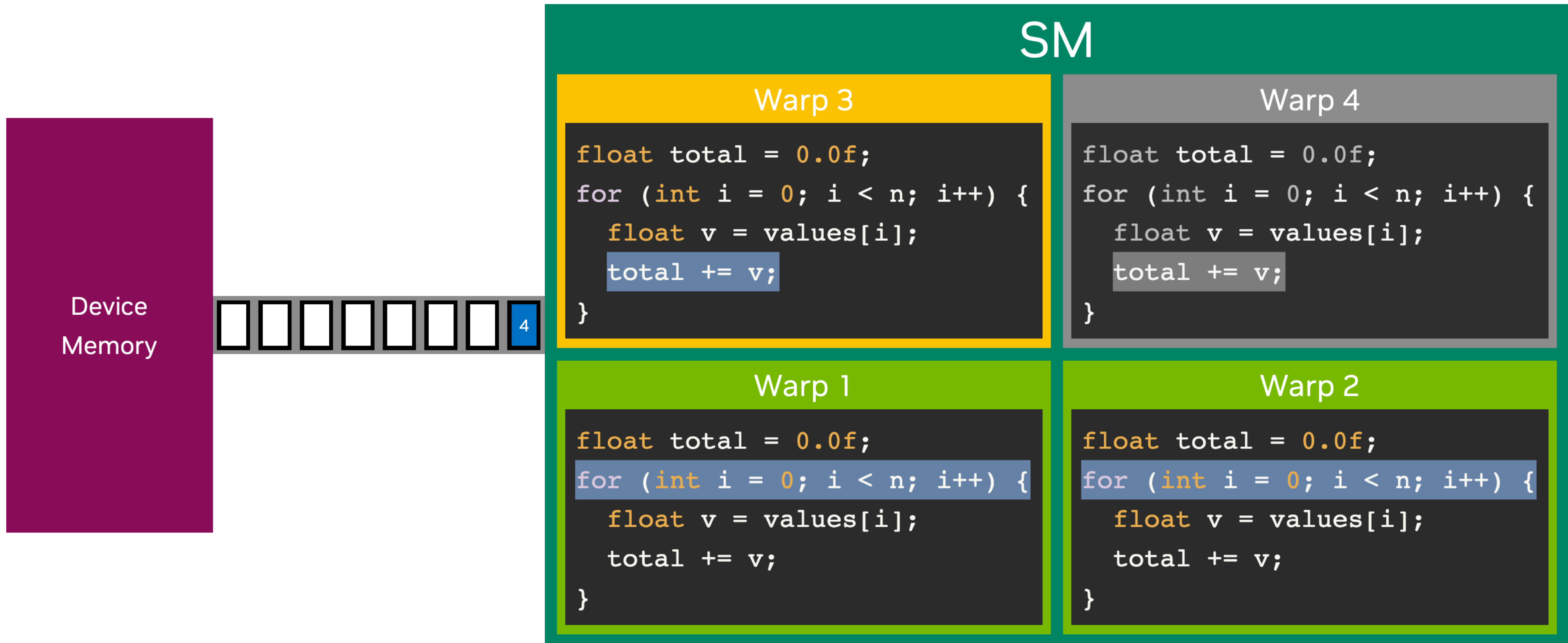
Bytes In Flight



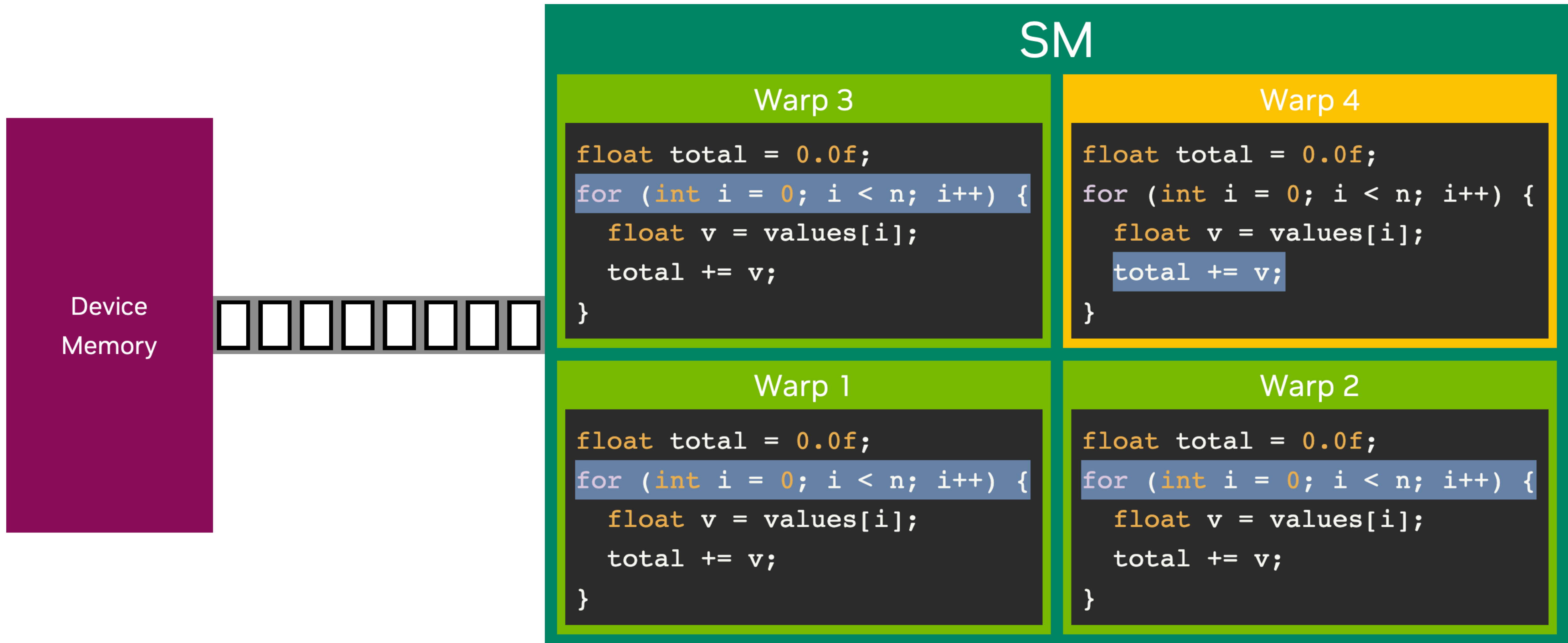
Bytes In Flight



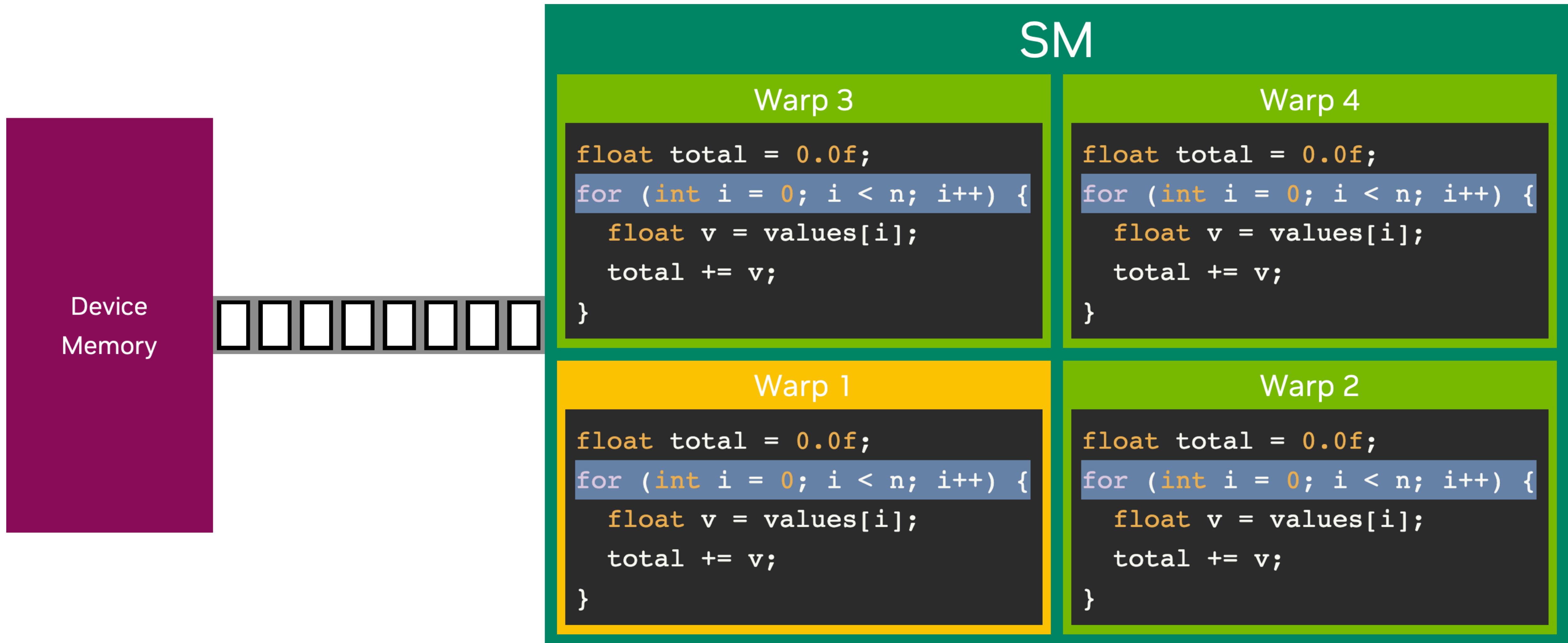
Bytes In Flight



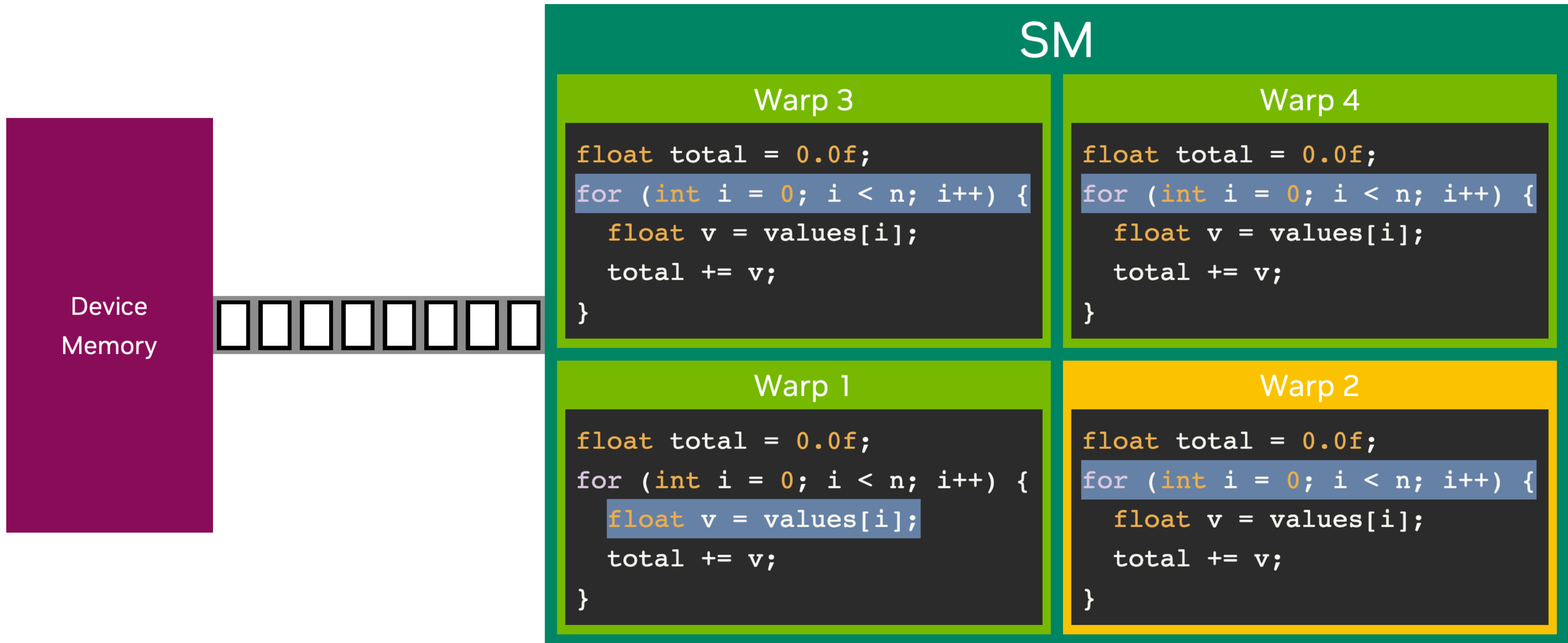
Bytes In Flight



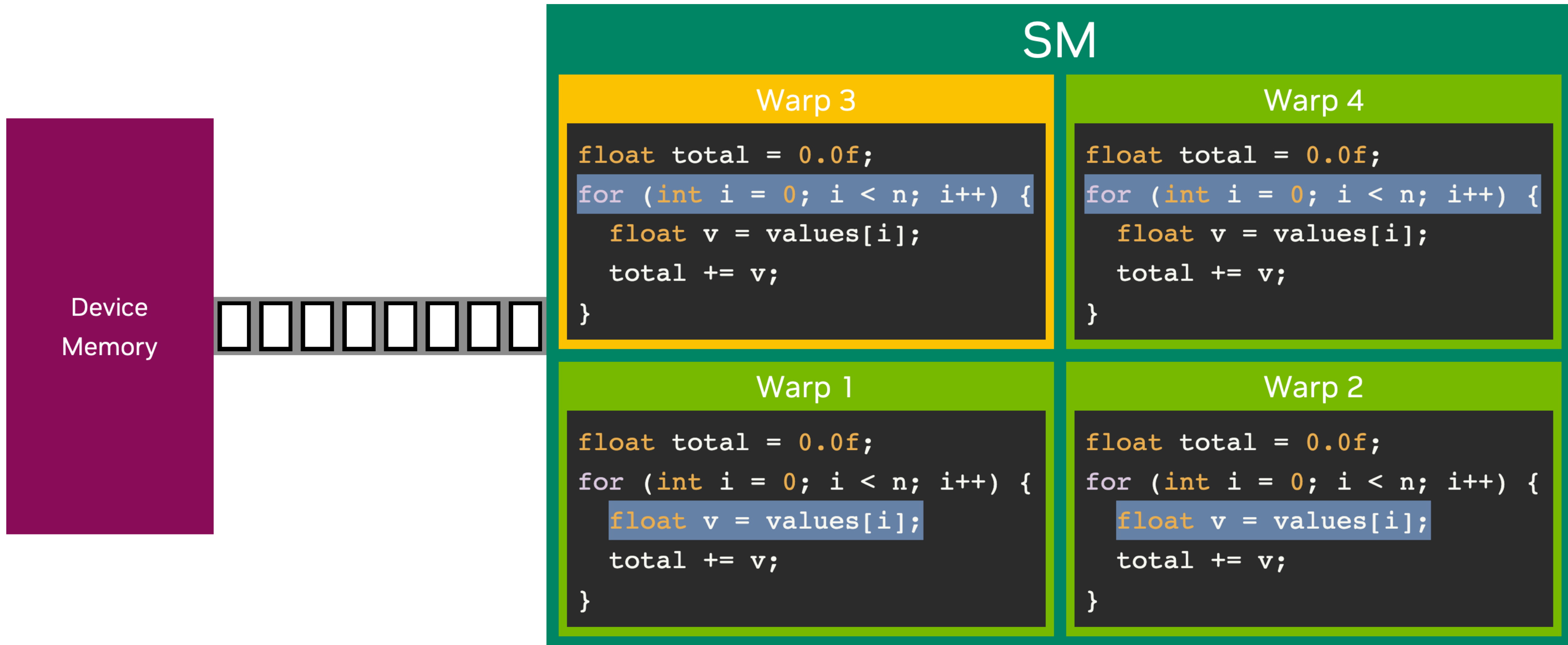
Bytes In Flight



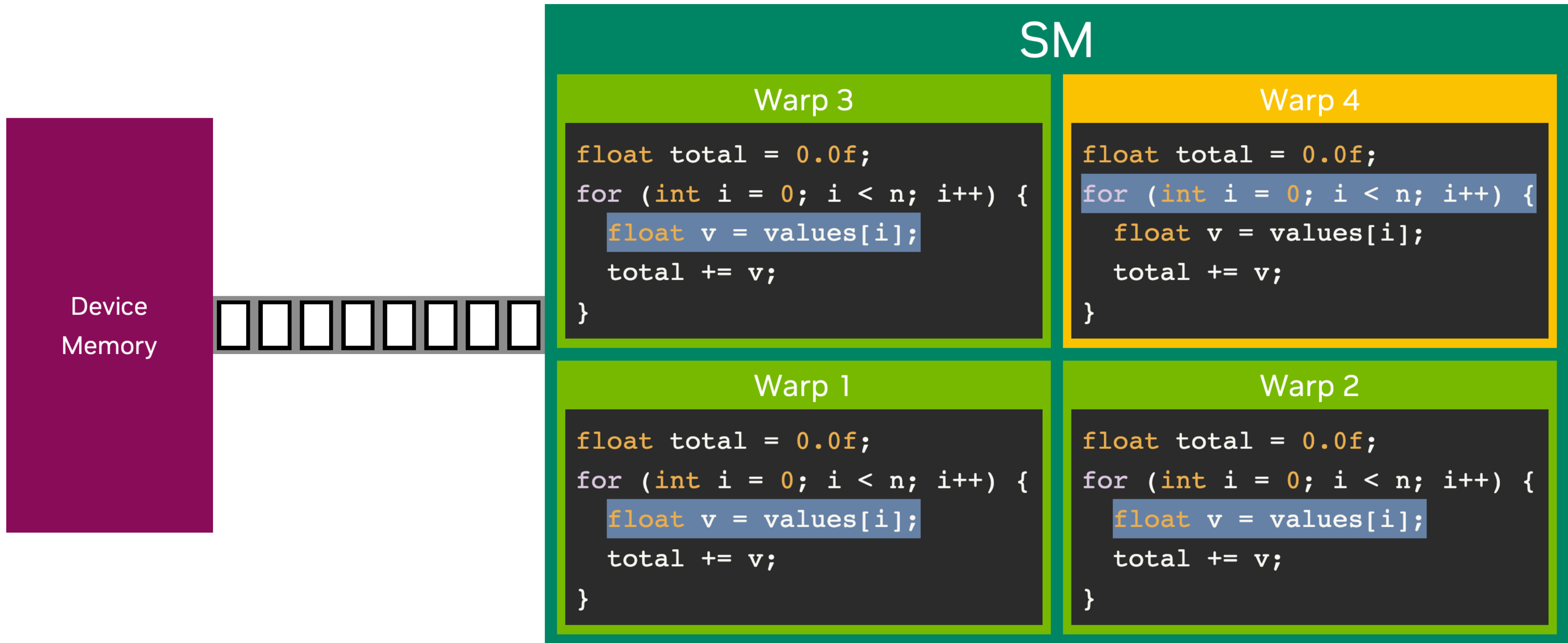
Bytes In Flight



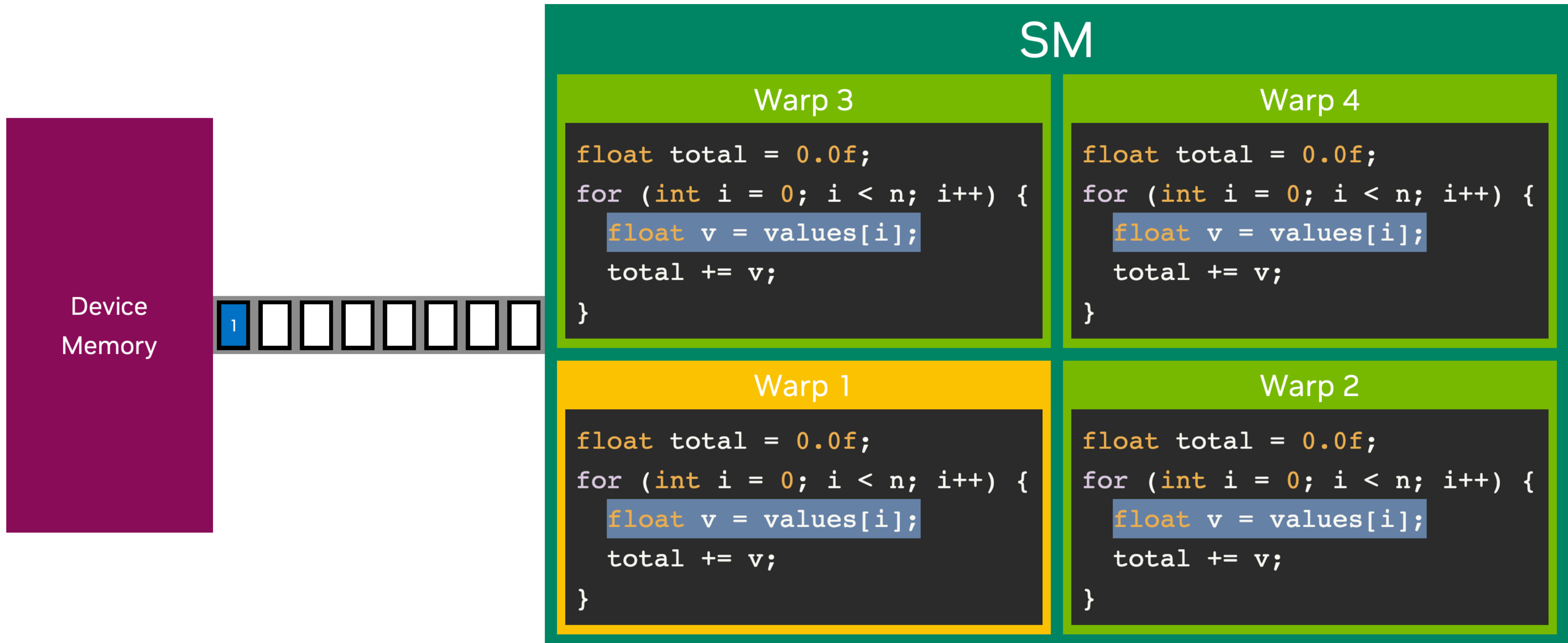
Bytes In Flight



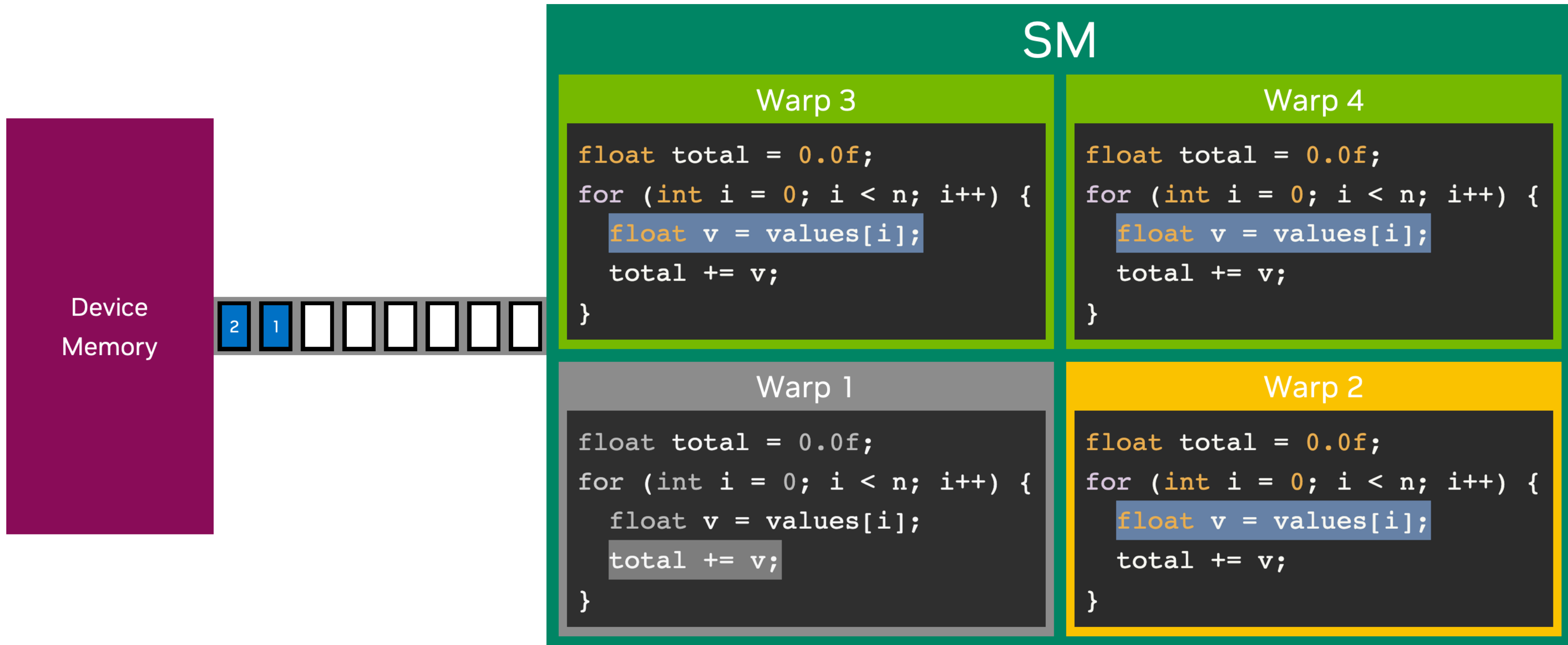
Bytes In Flight



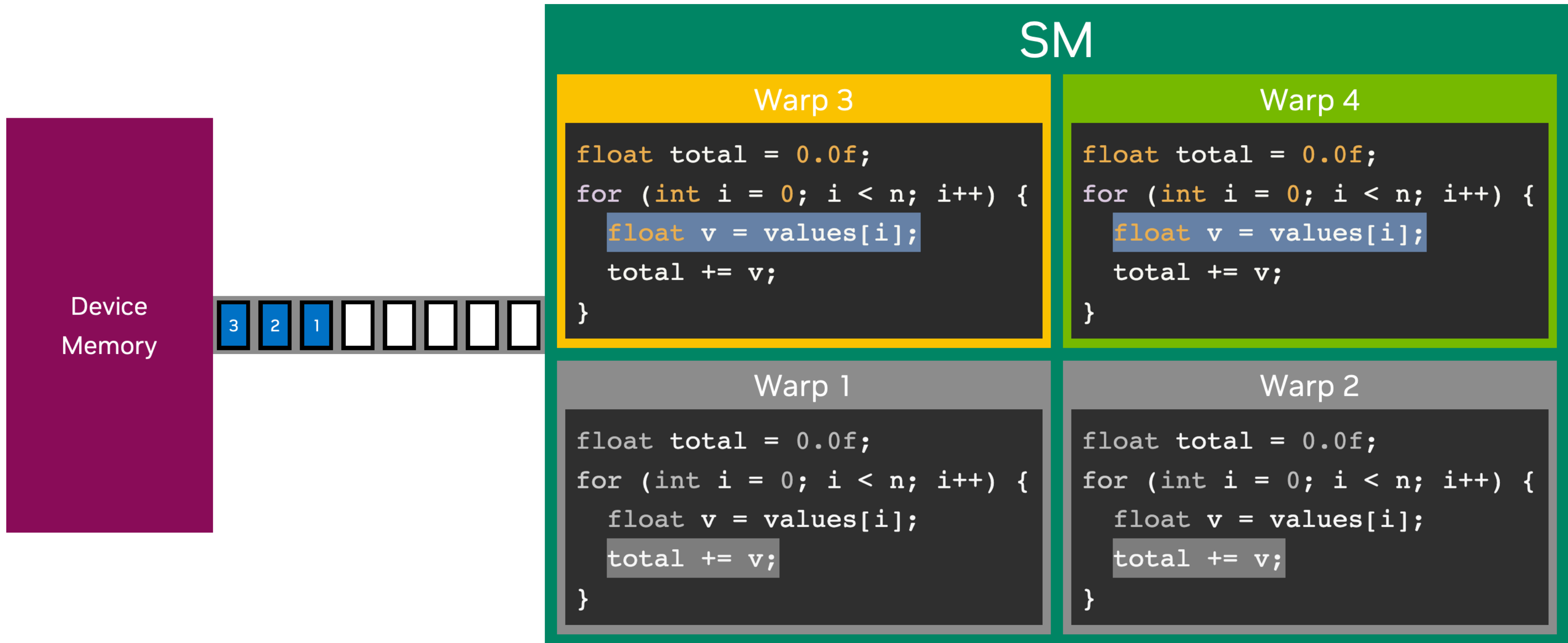
Bytes In Flight



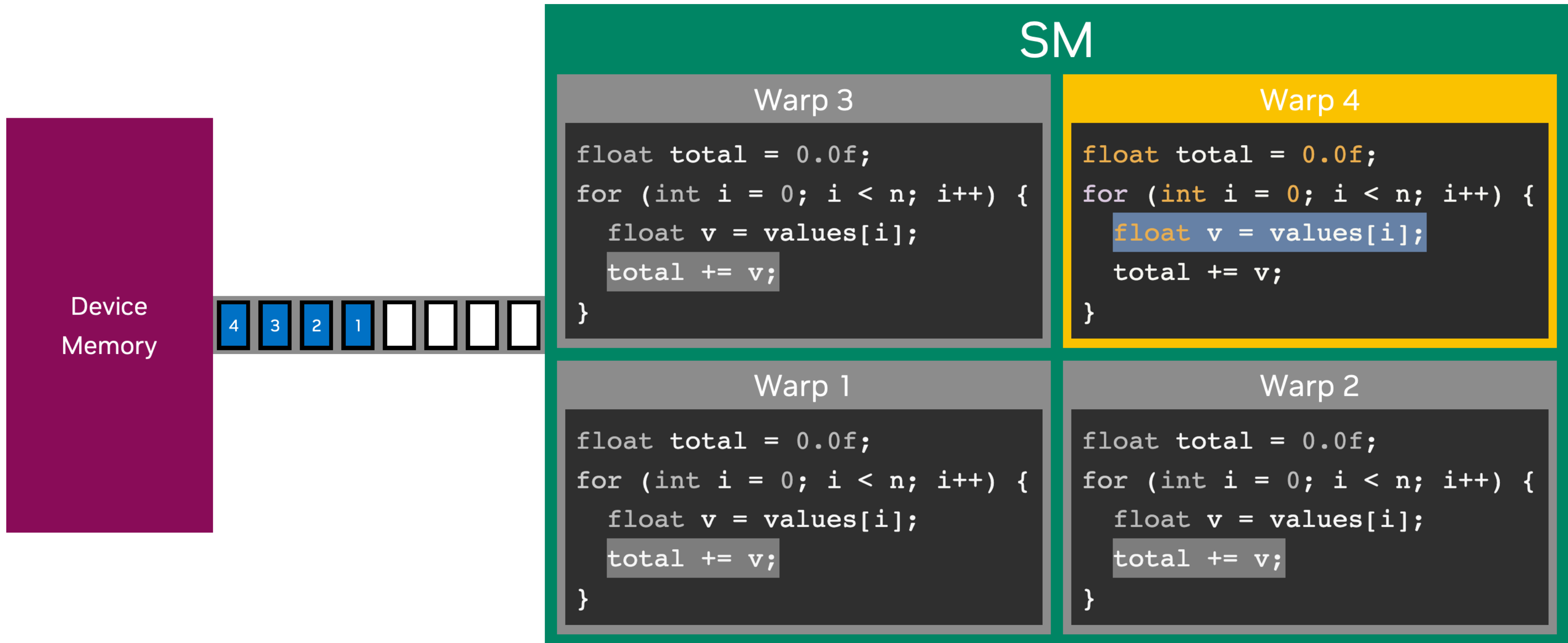
Bytes In Flight



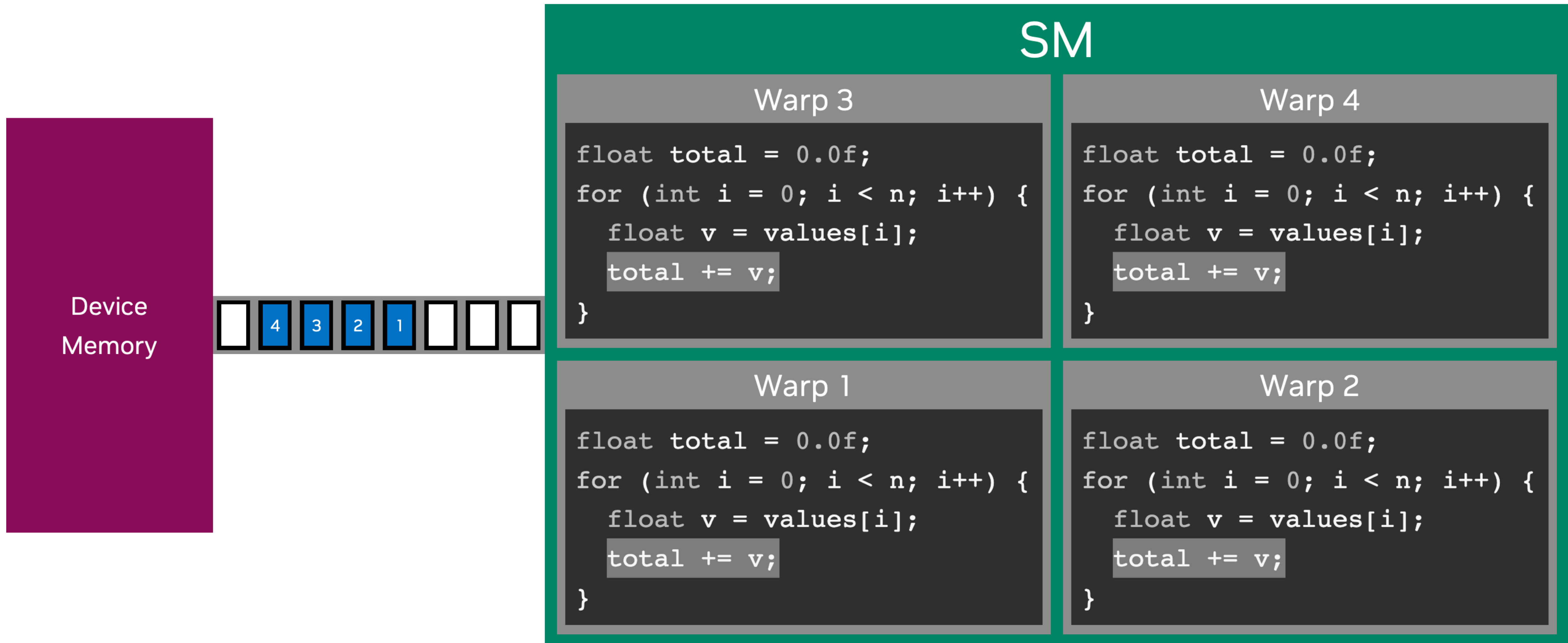
Bytes In Flight



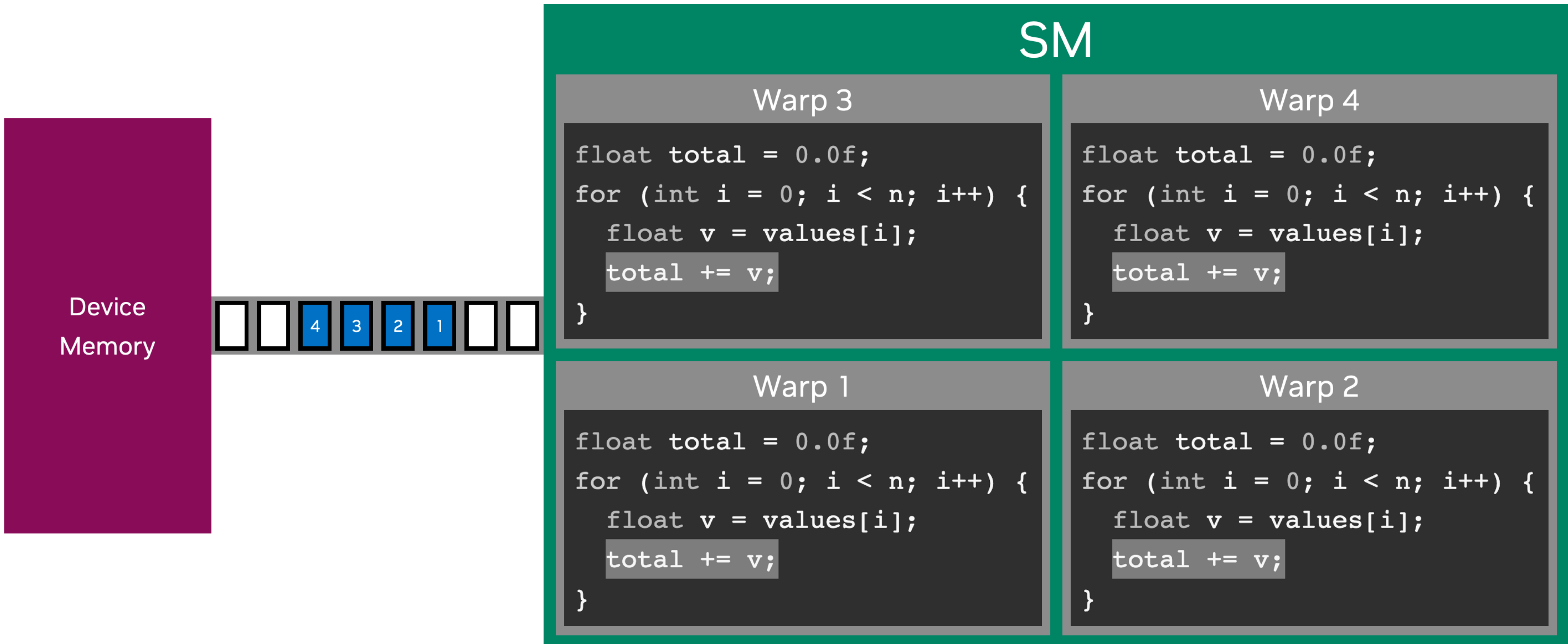
Bytes In Flight



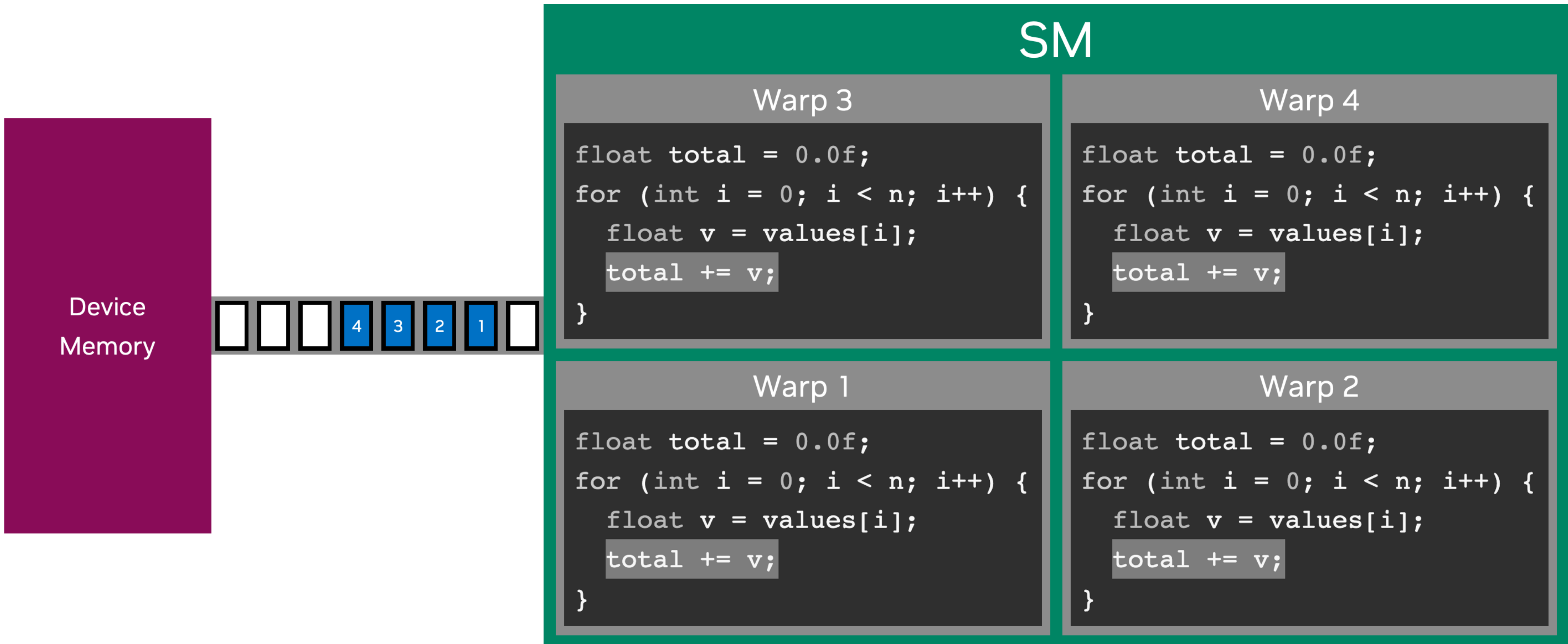
Bytes In Flight



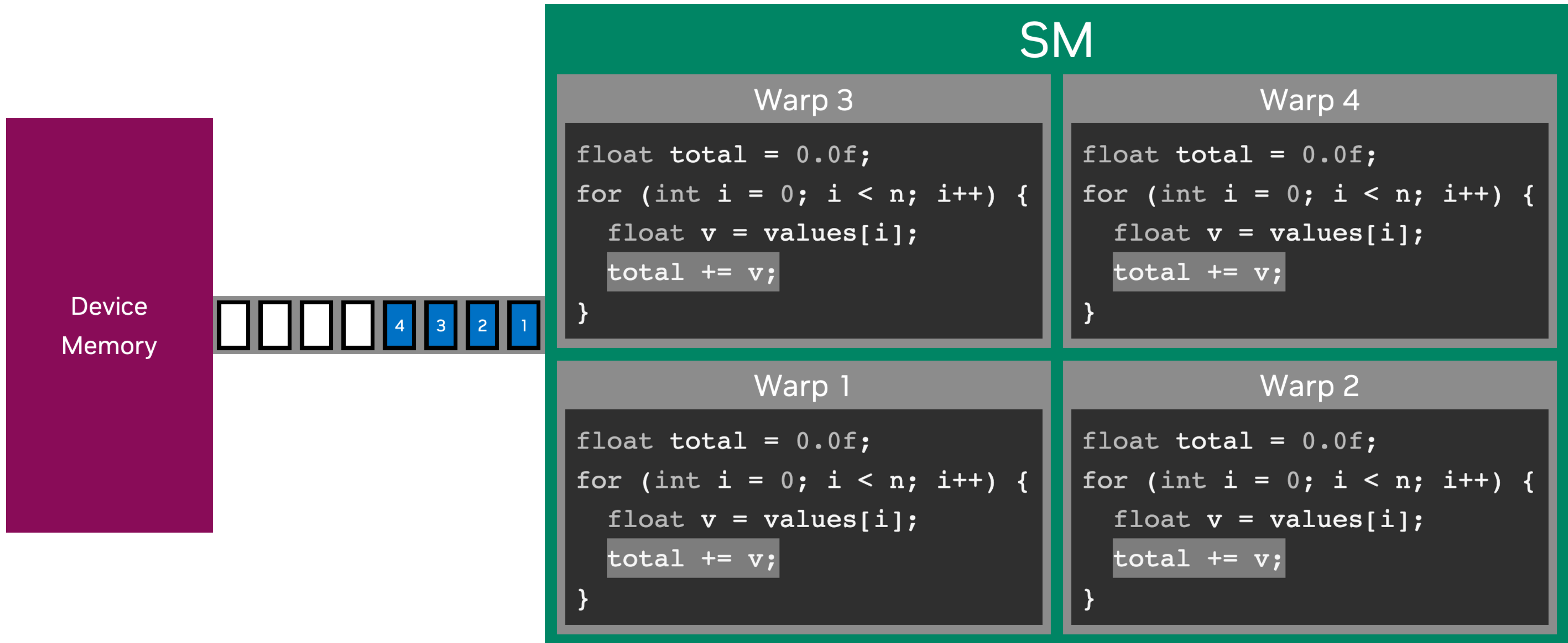
Bytes In Flight



Bytes In Flight



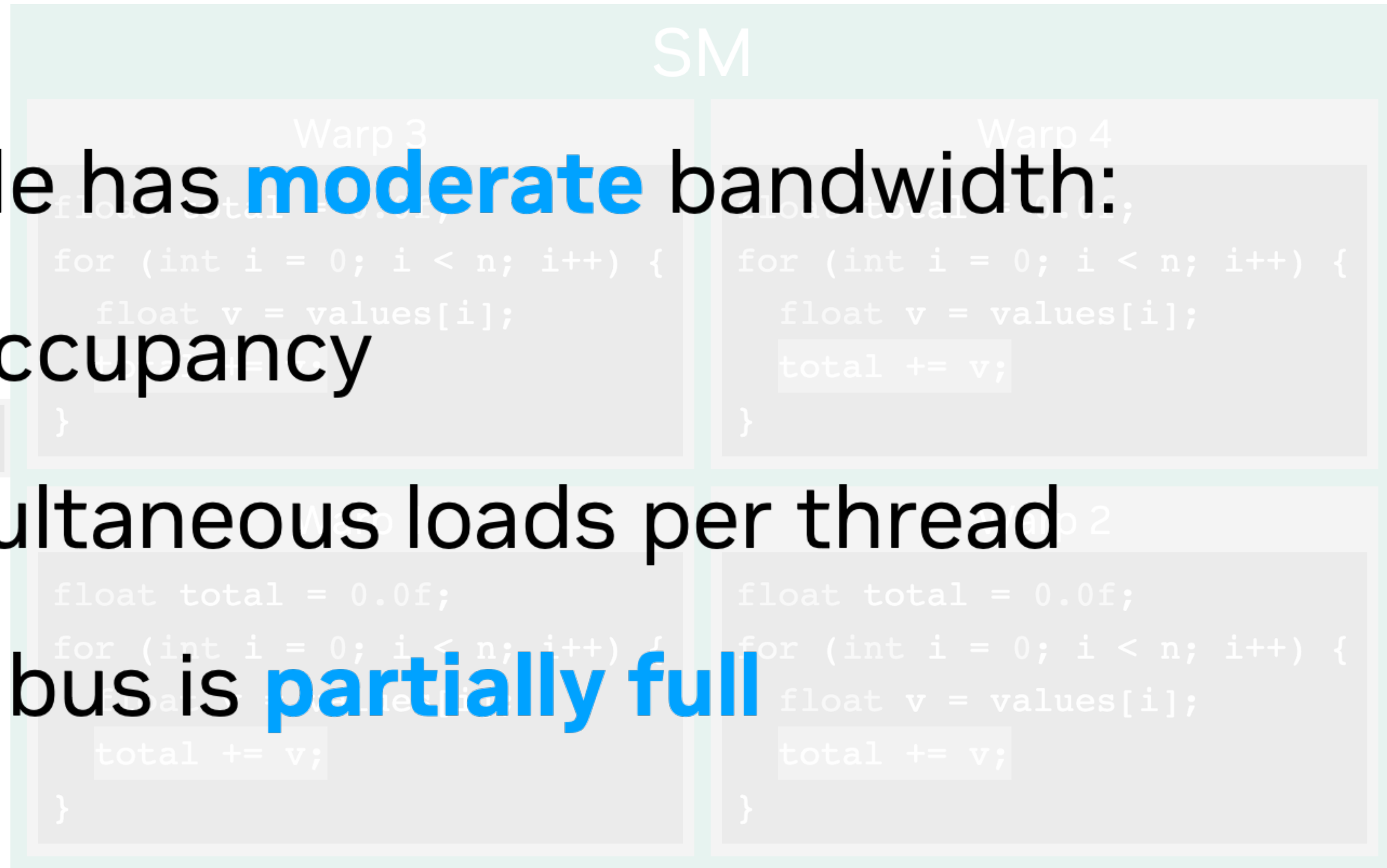
Bytes In Flight



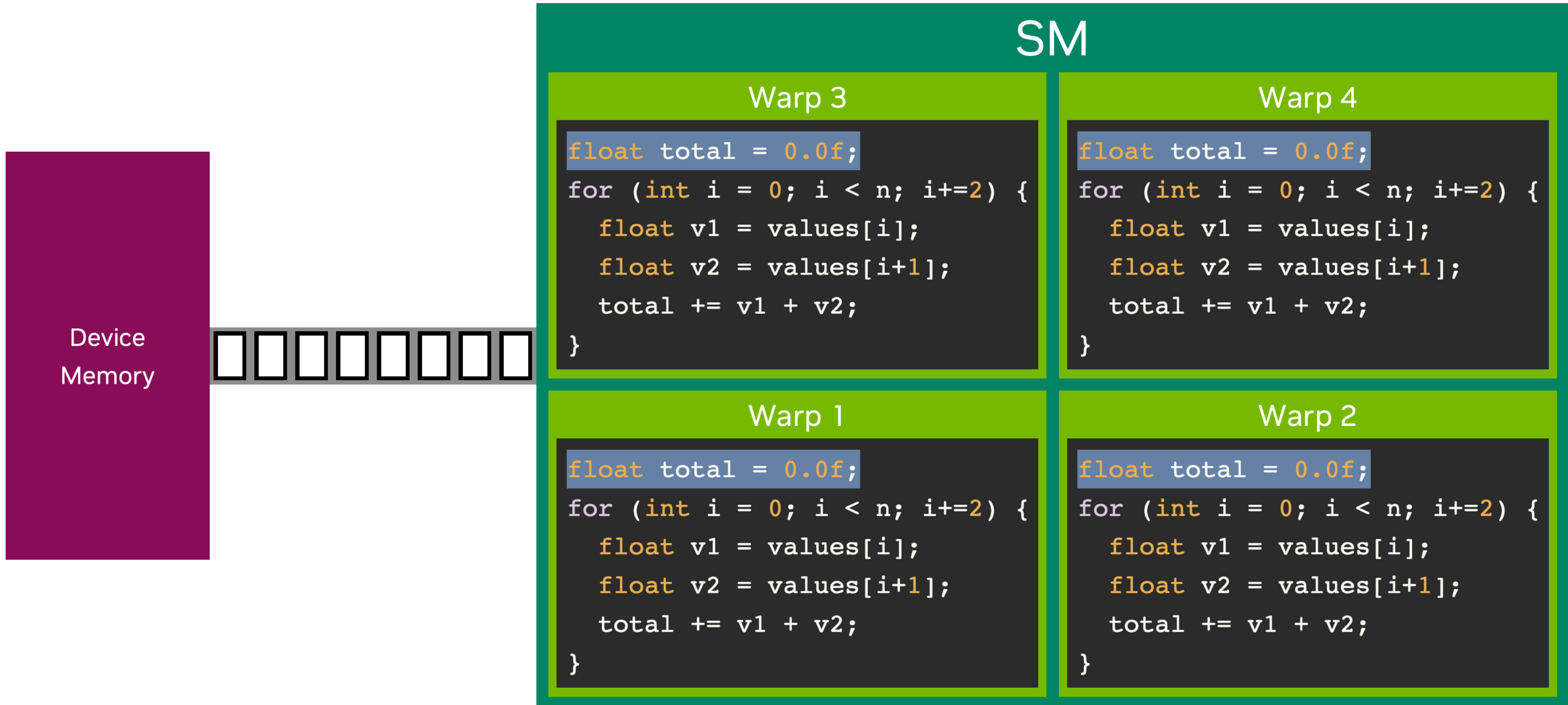
Bytes In Flight

This example has **moderate** bandwidth:

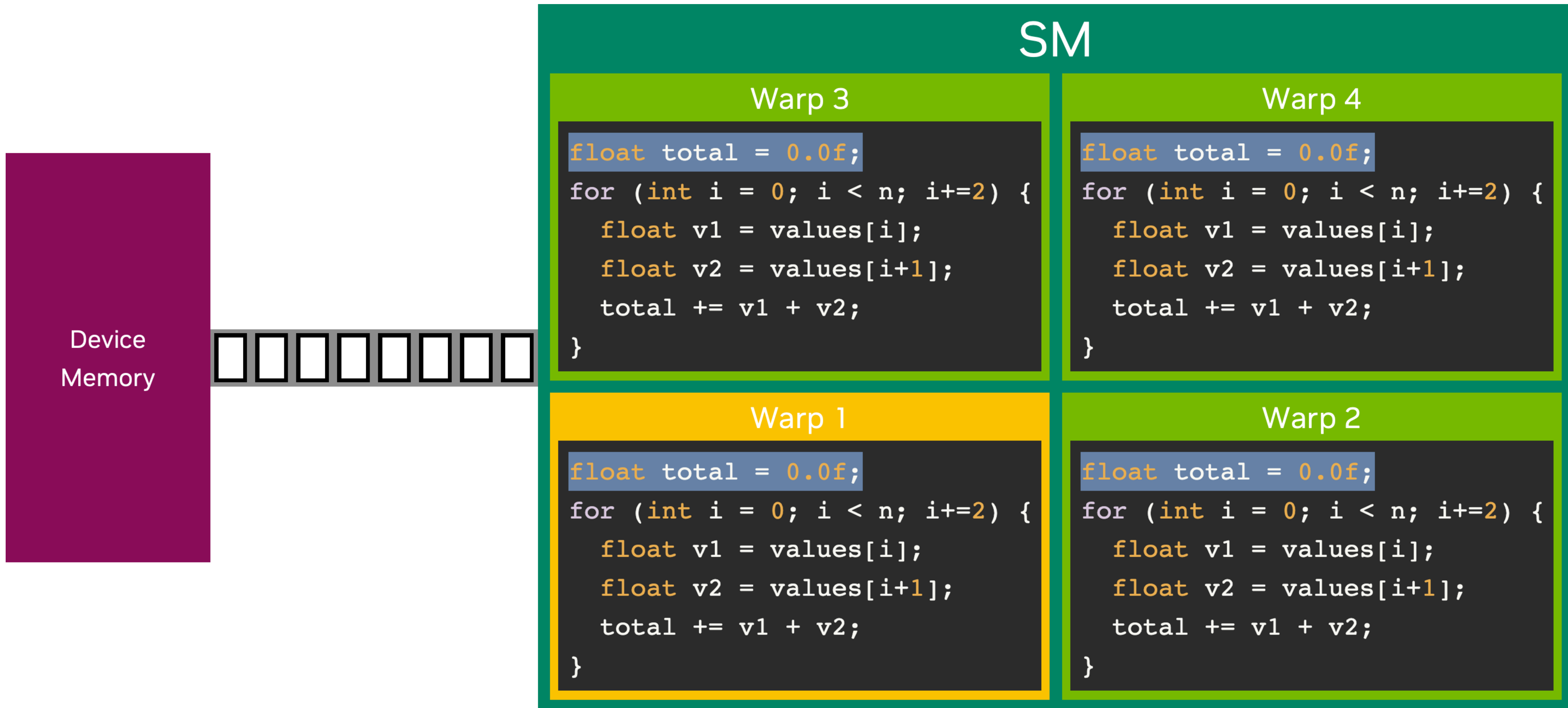
- **Better** occupancy
- **Few** simultaneous loads per thread
- Memory bus is **partially full**



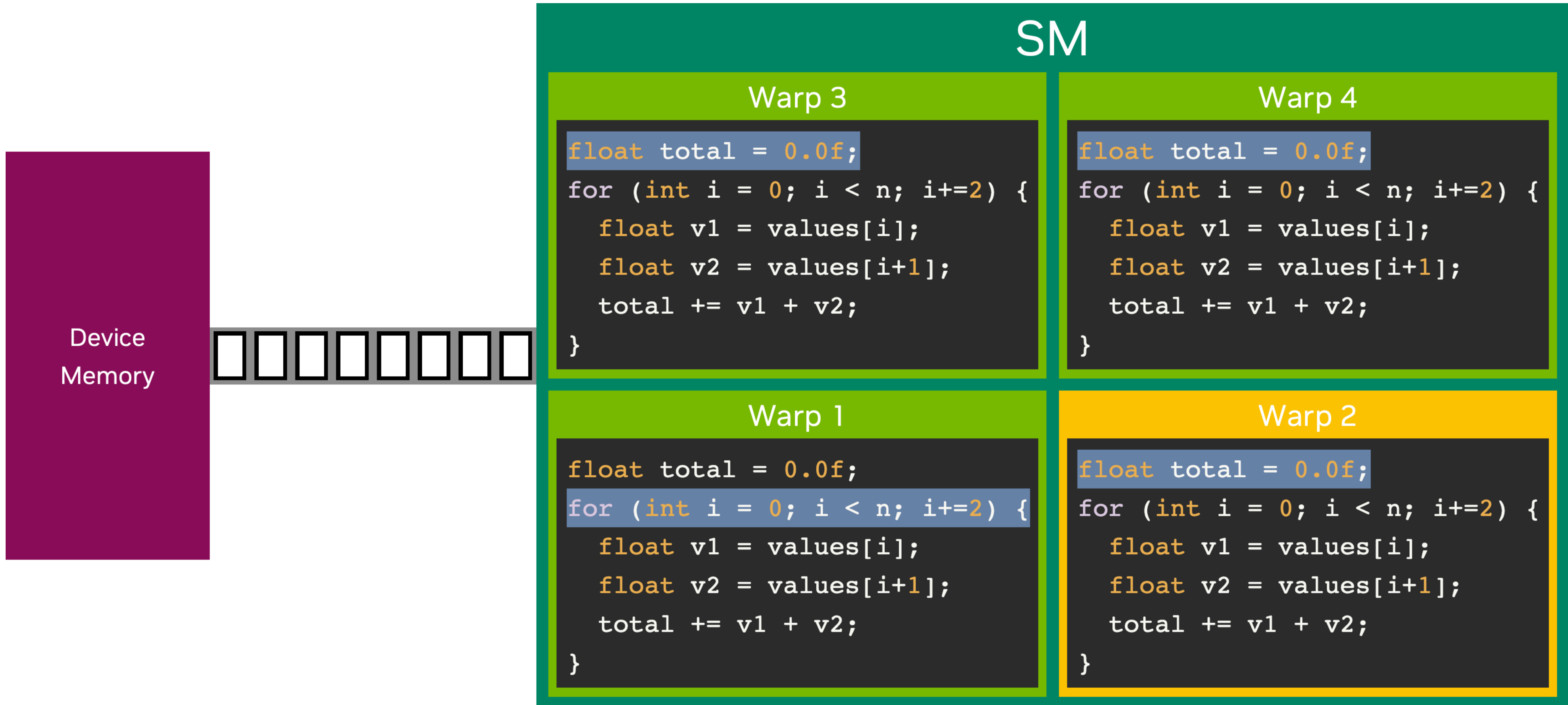
Bytes In Flight



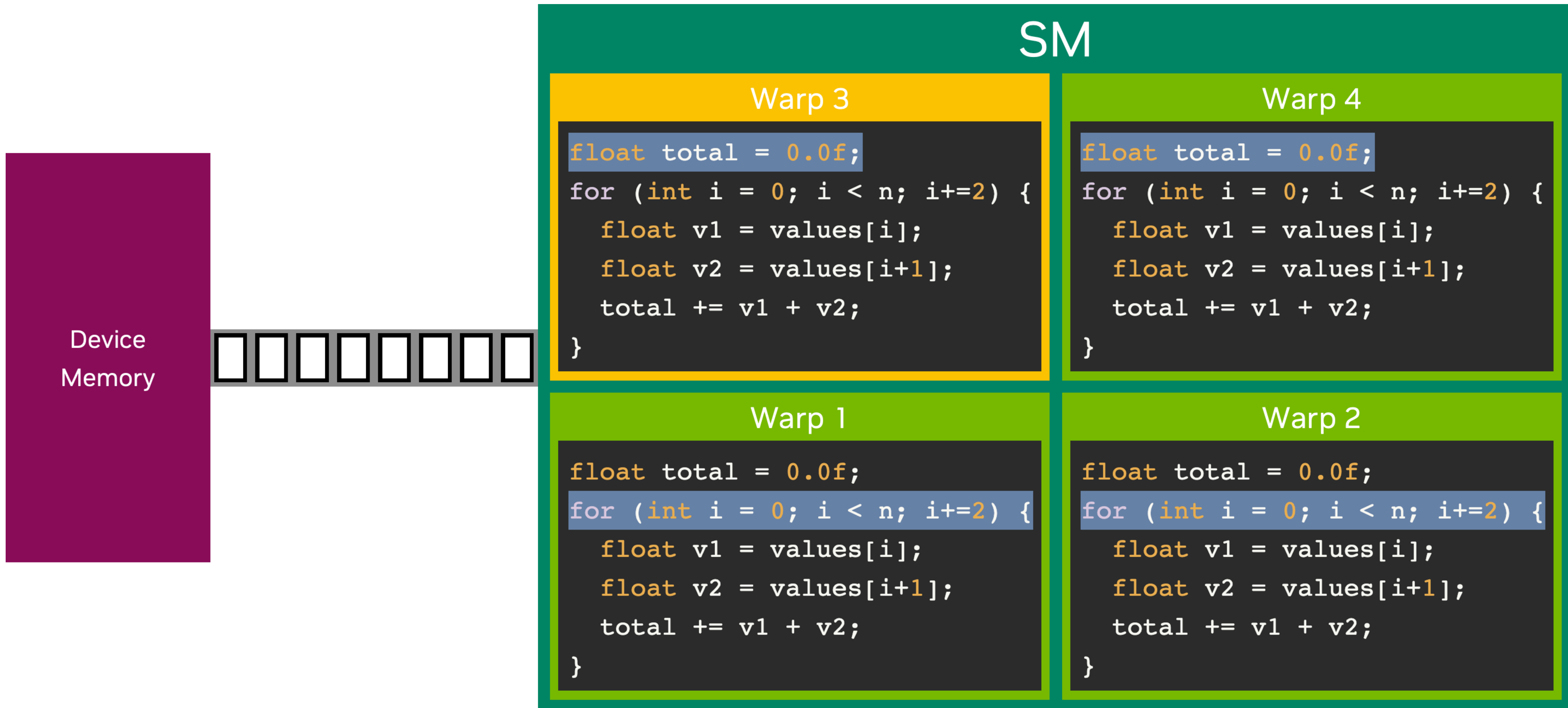
Bytes In Flight



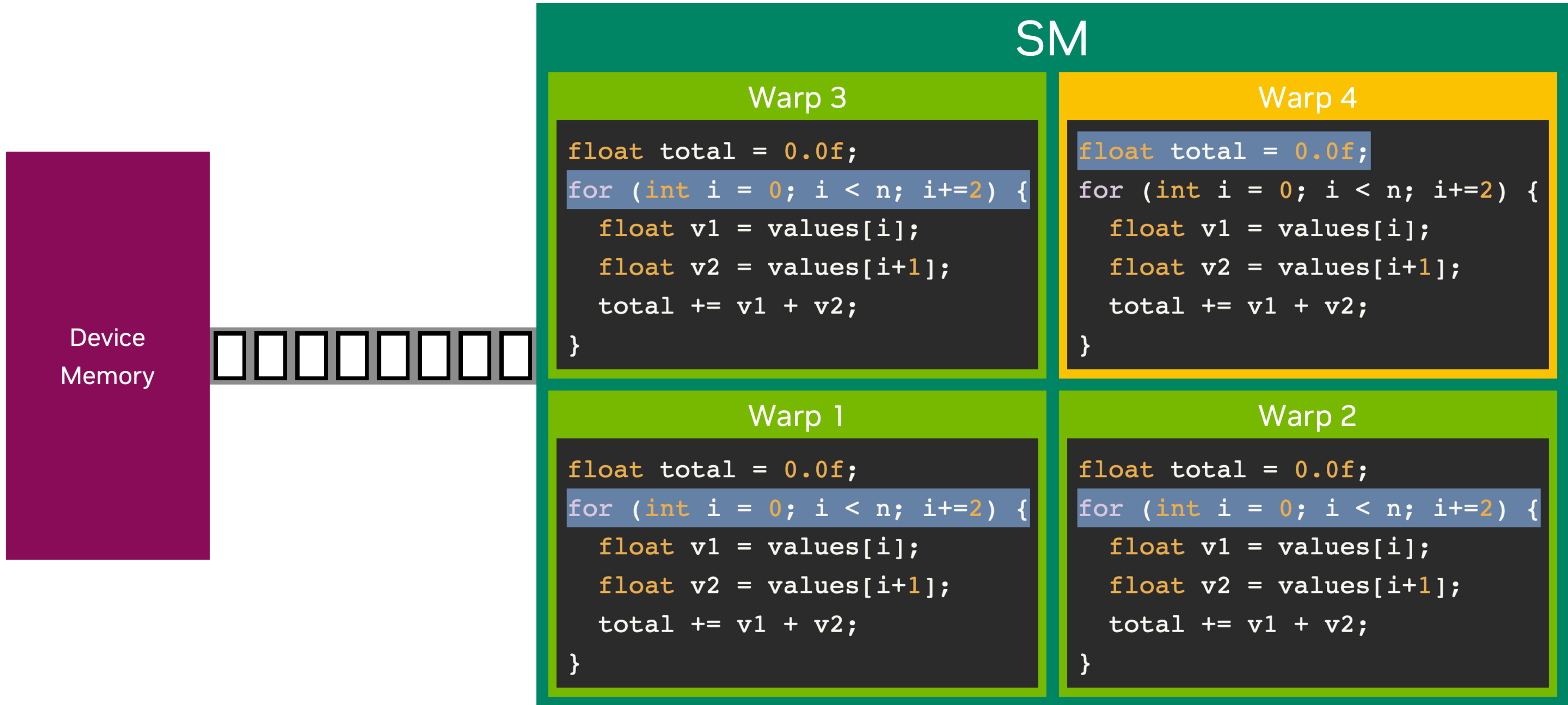
Bytes In Flight



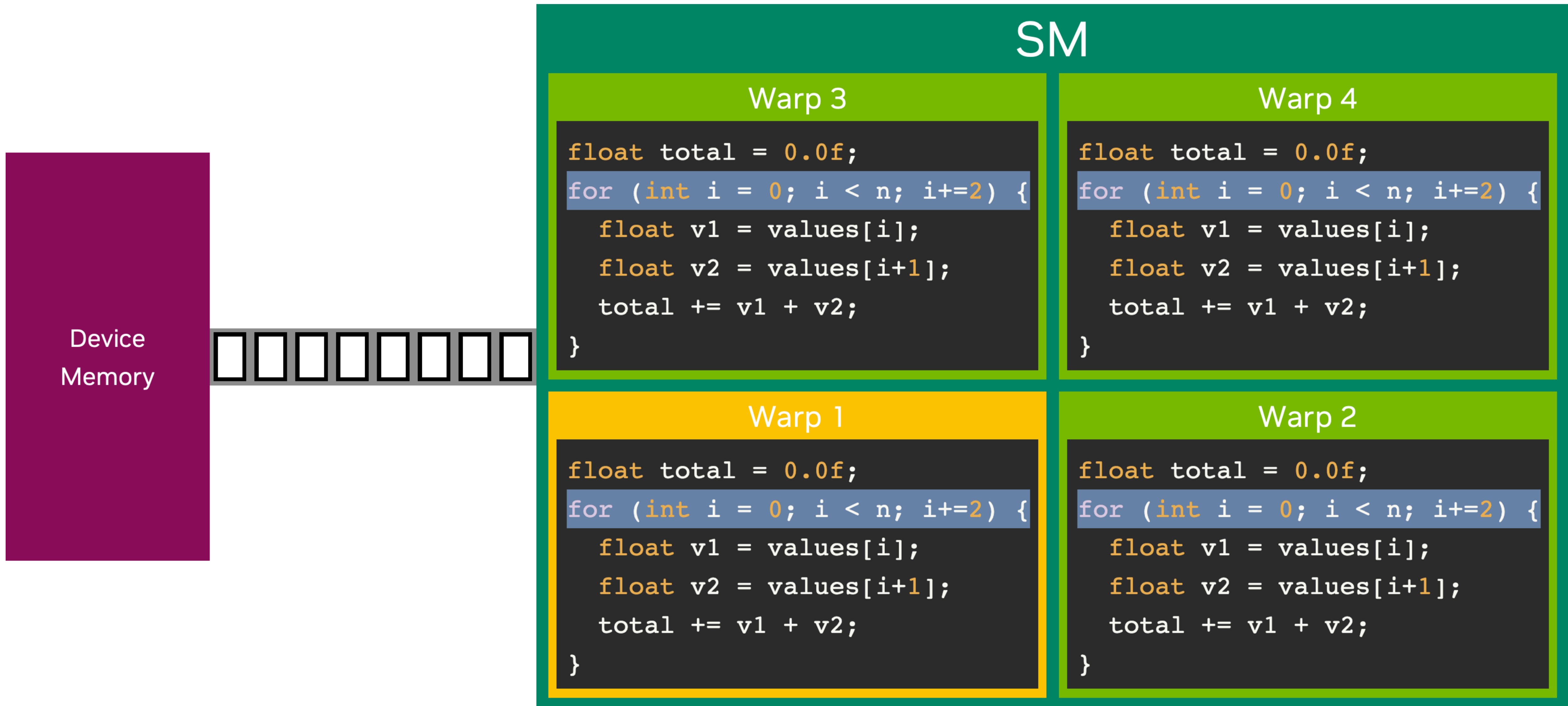
Bytes In Flight



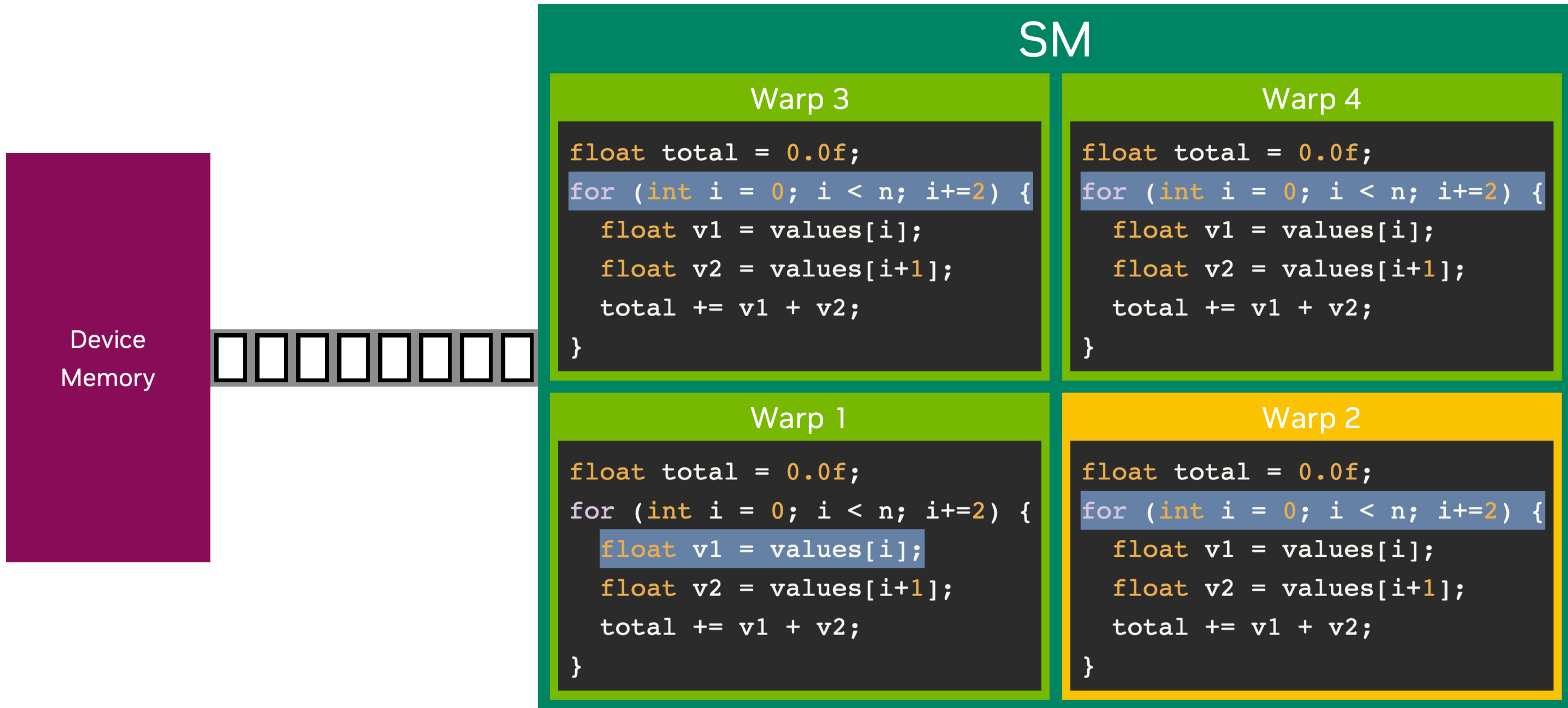
Bytes In Flight



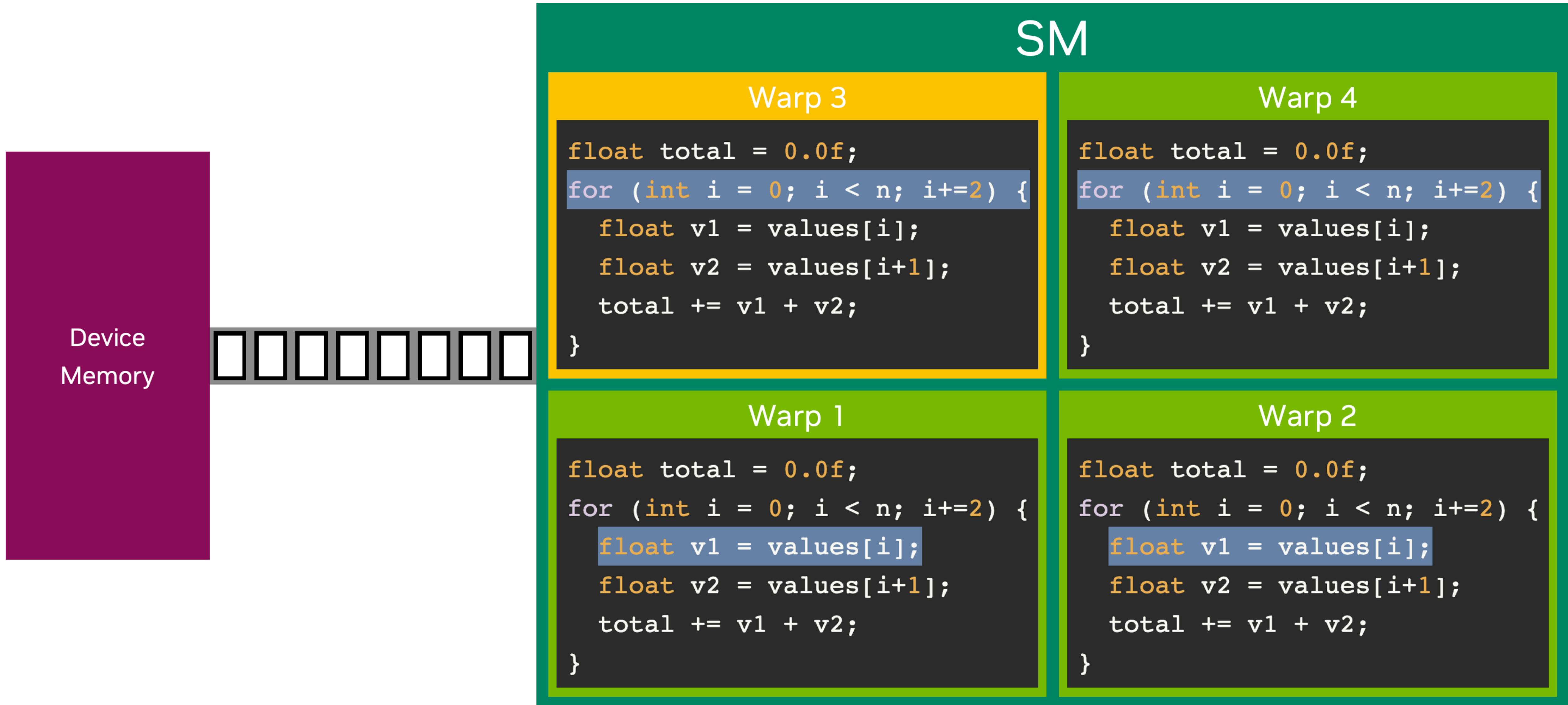
Bytes In Flight



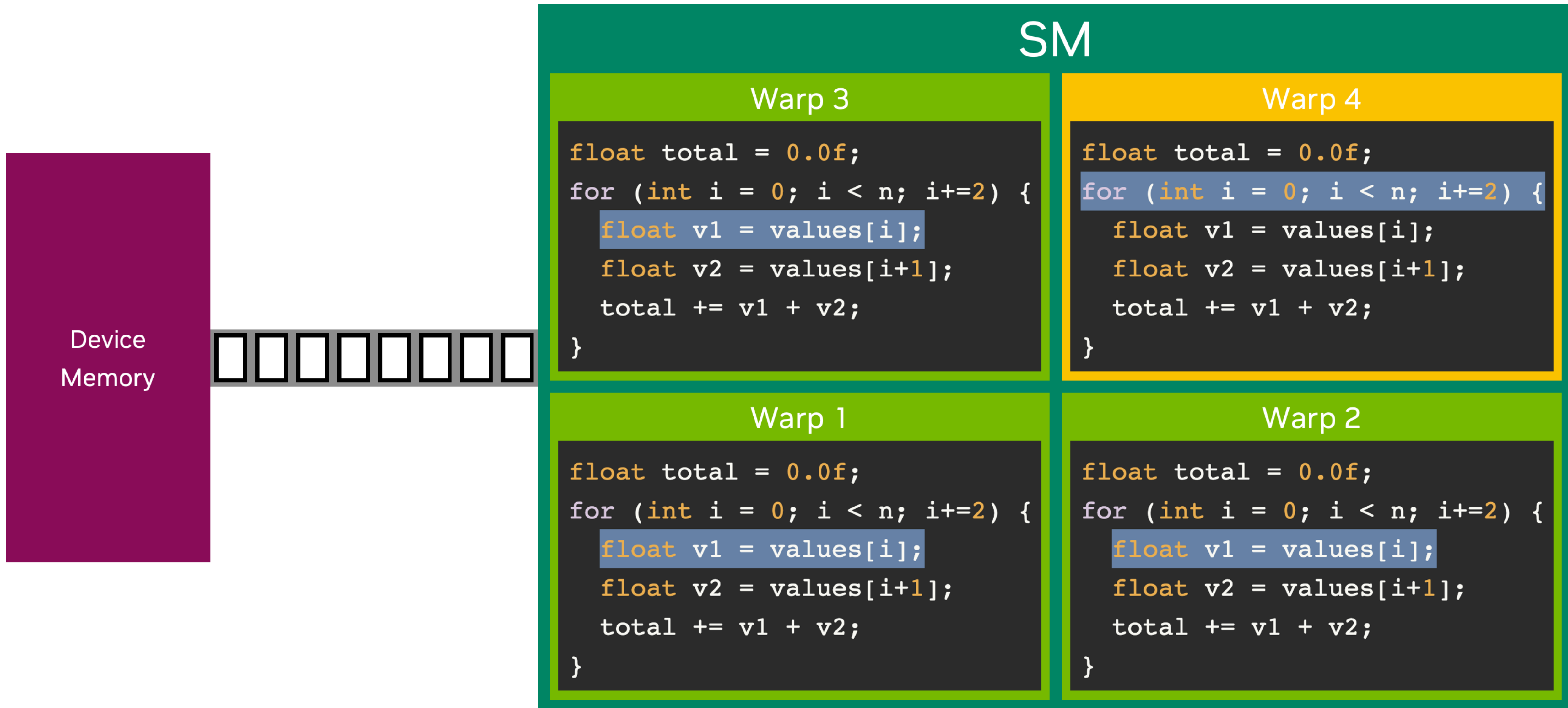
Bytes In Flight



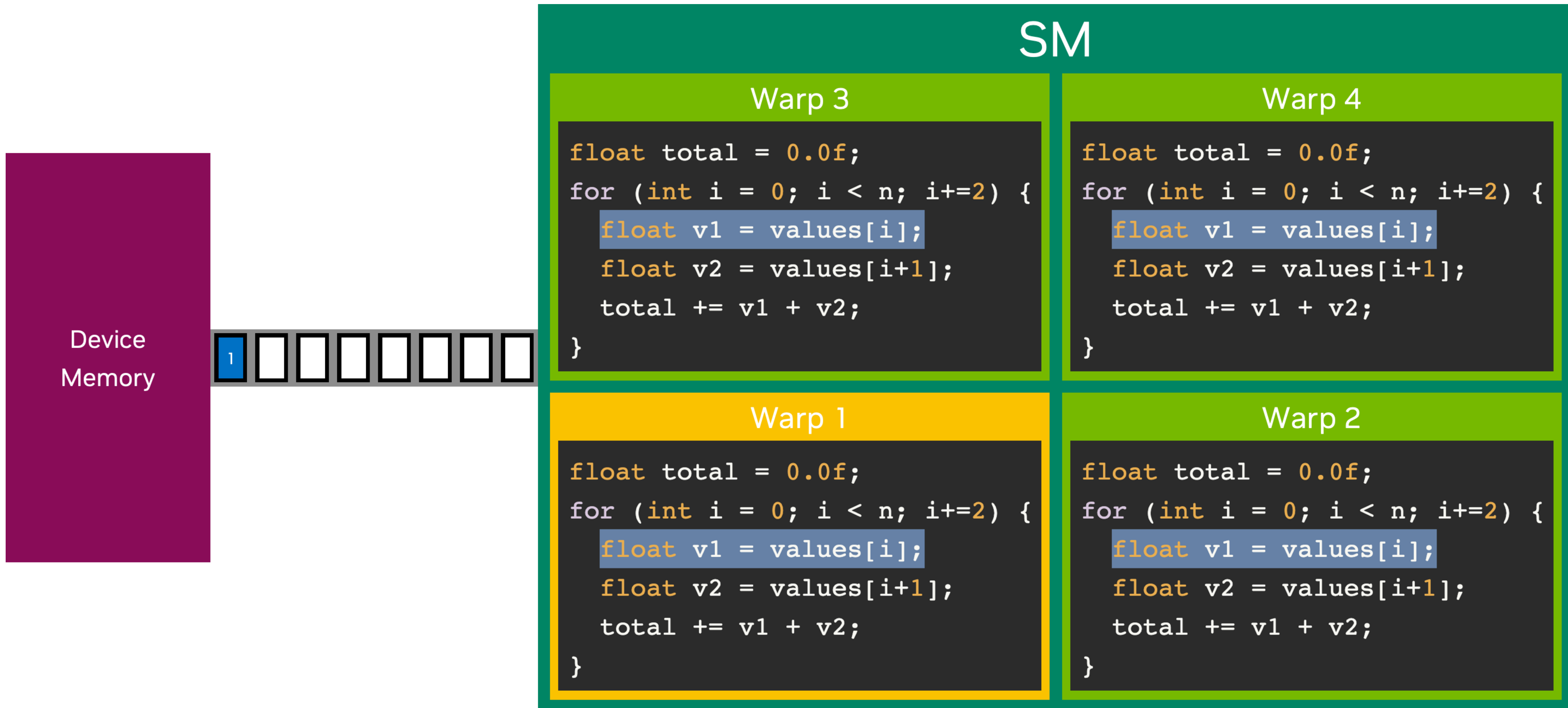
Bytes In Flight



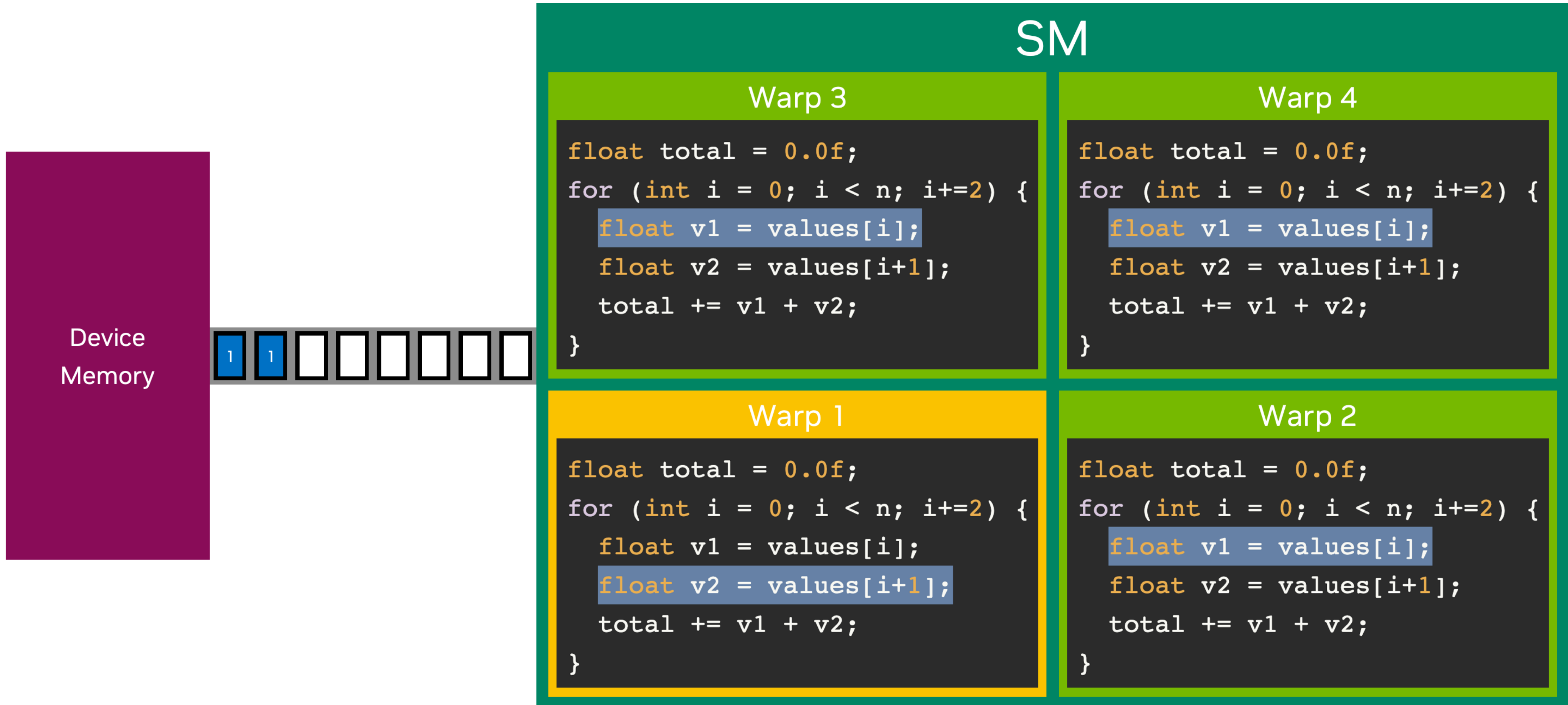
Bytes In Flight



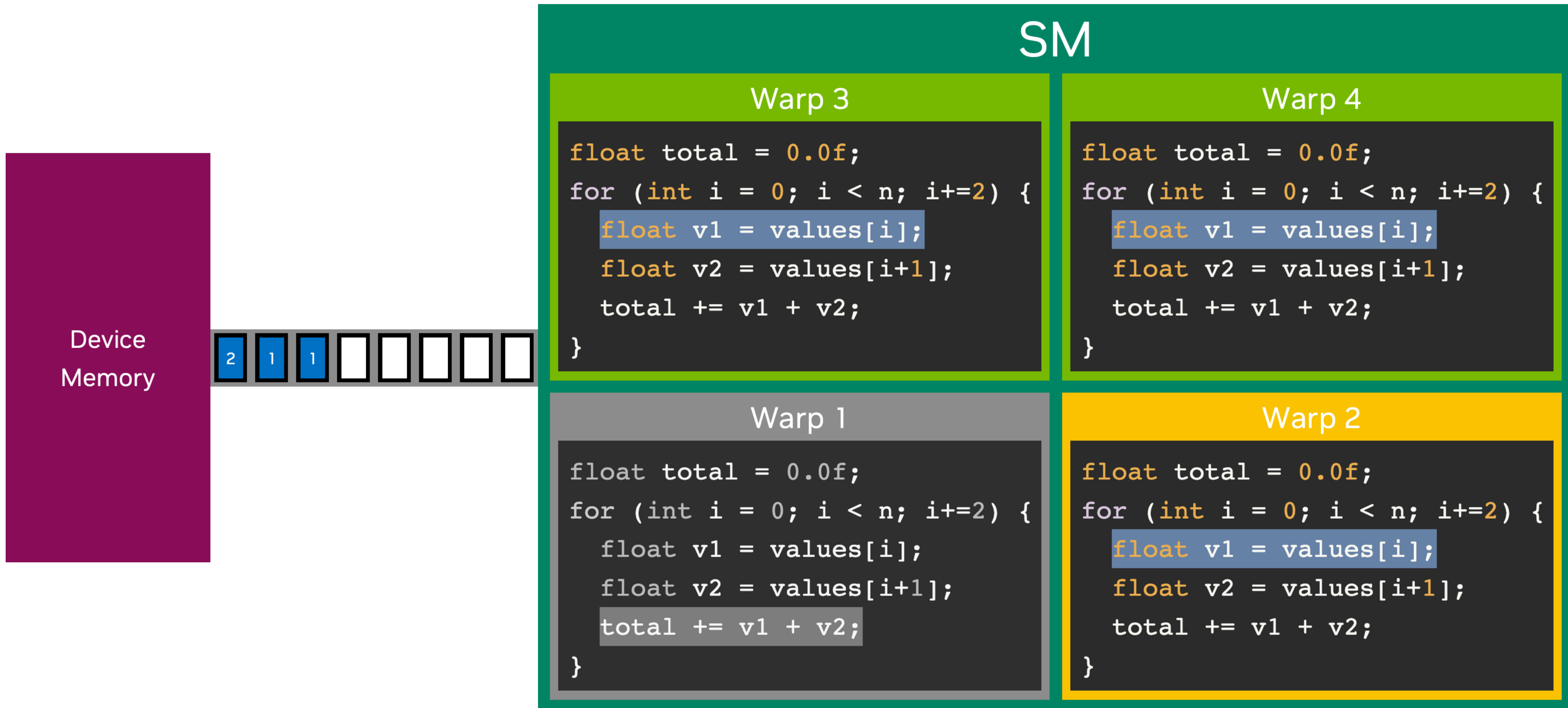
Bytes In Flight



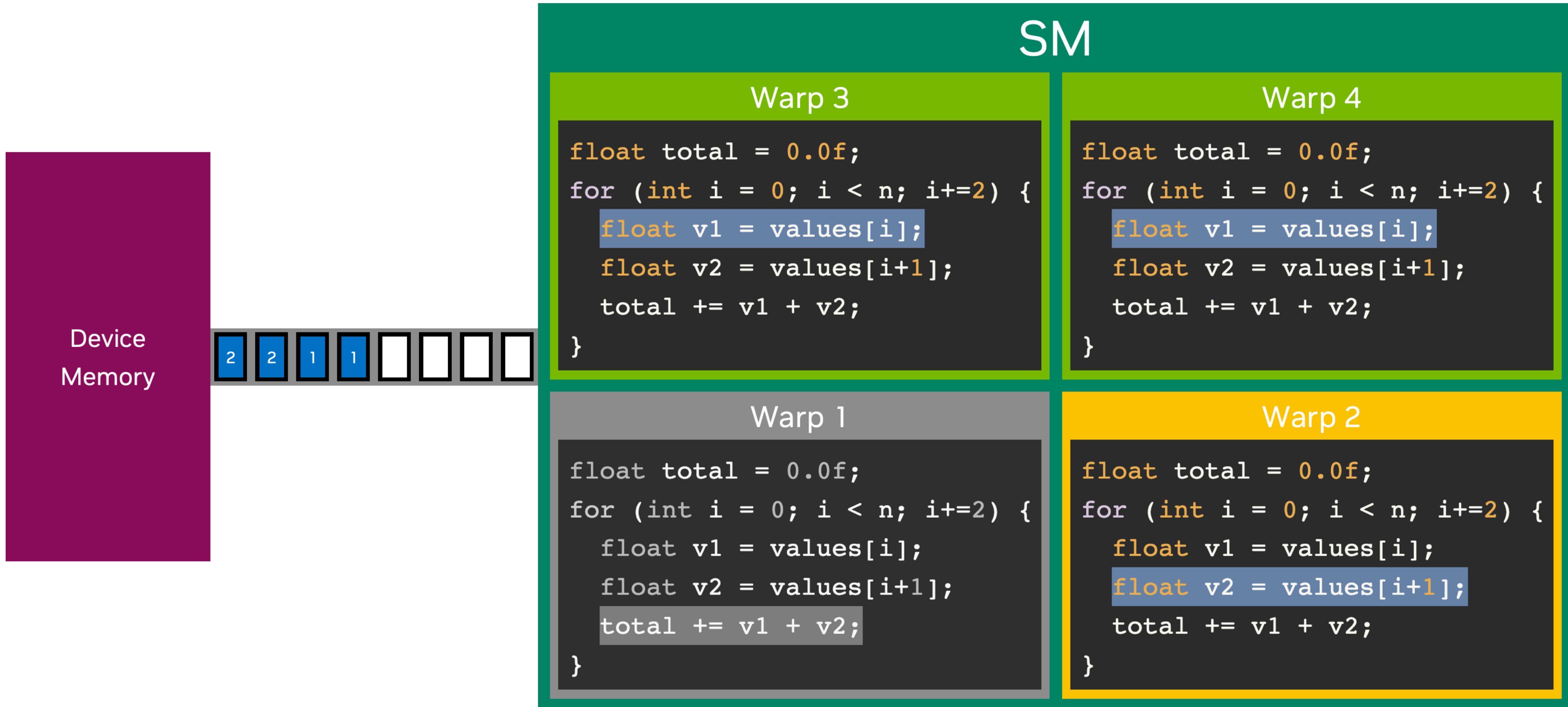
Bytes In Flight



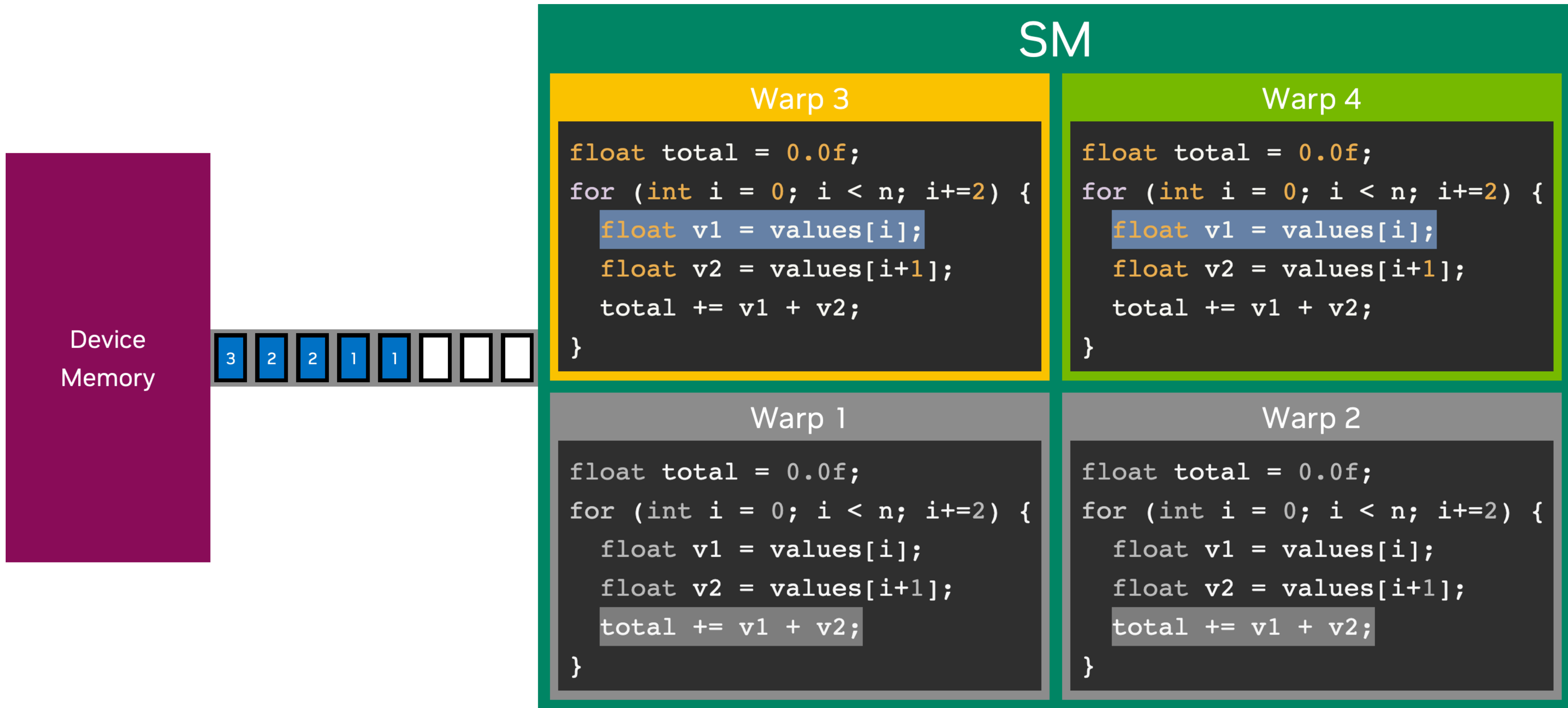
Bytes In Flight



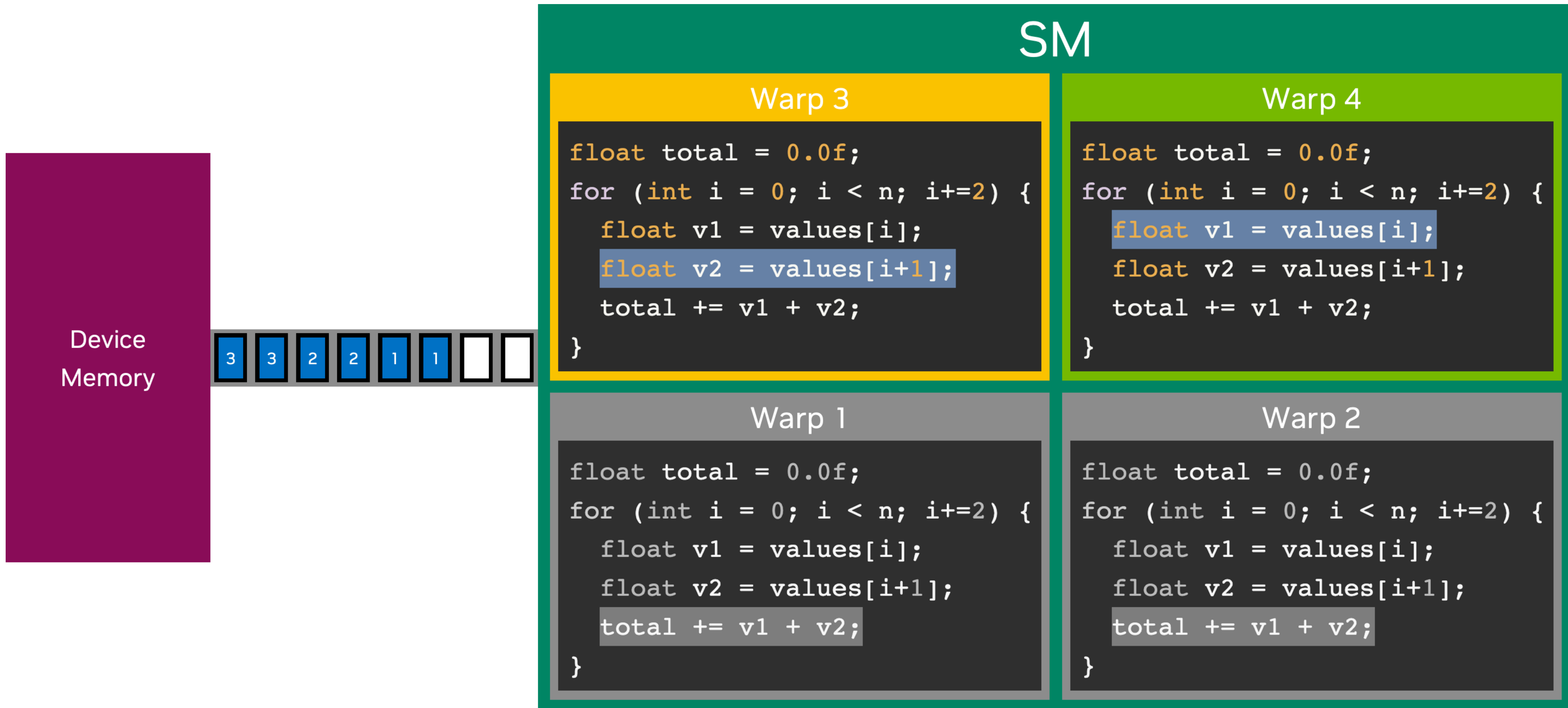
Bytes In Flight



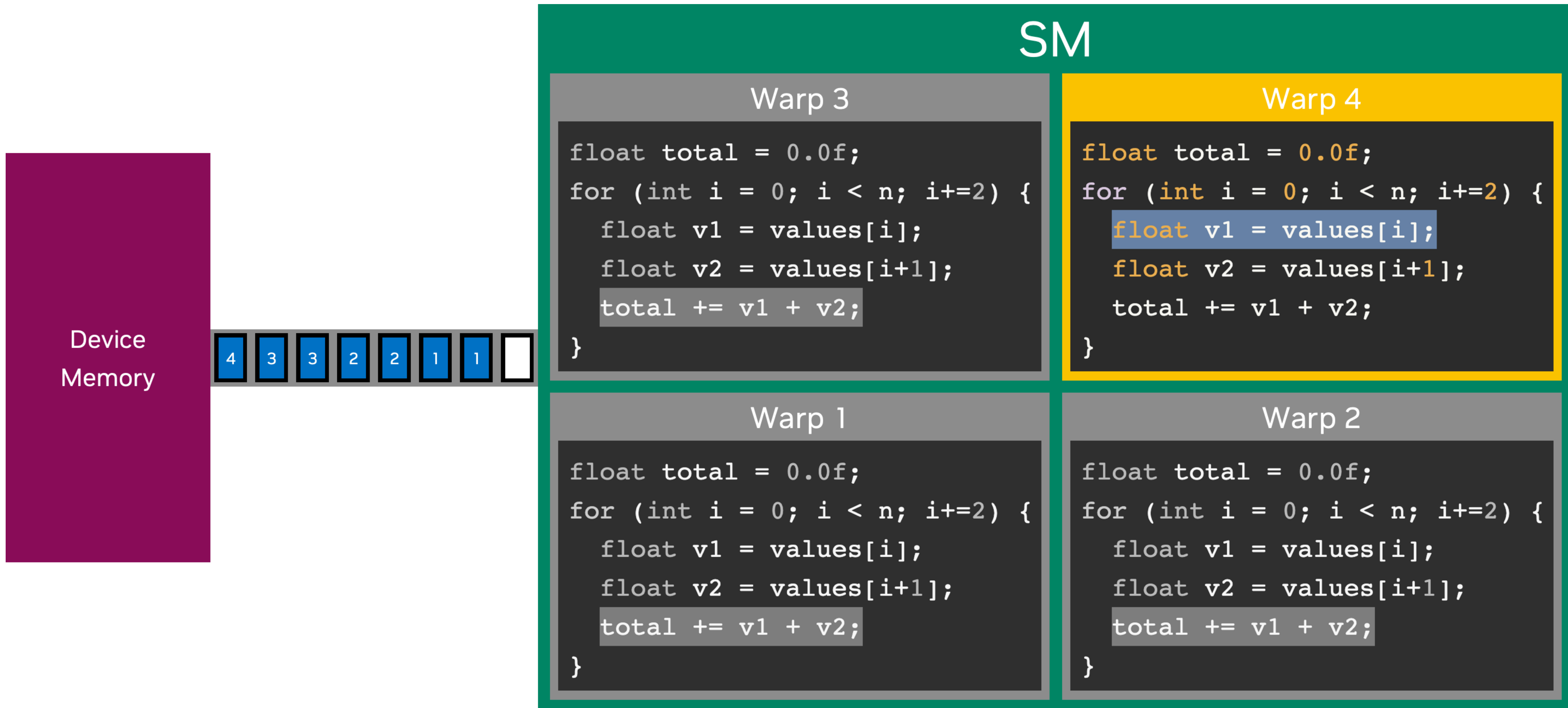
Bytes In Flight



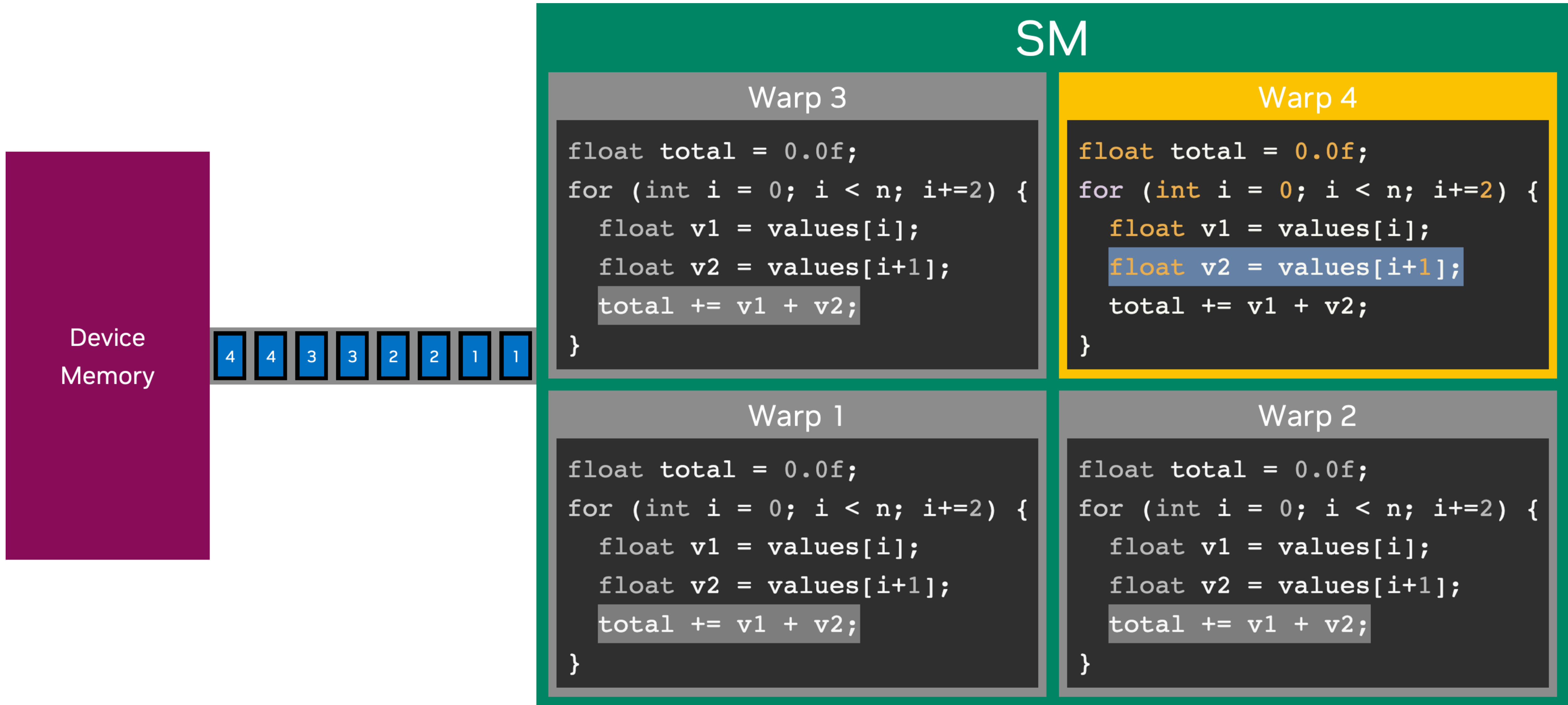
Bytes In Flight



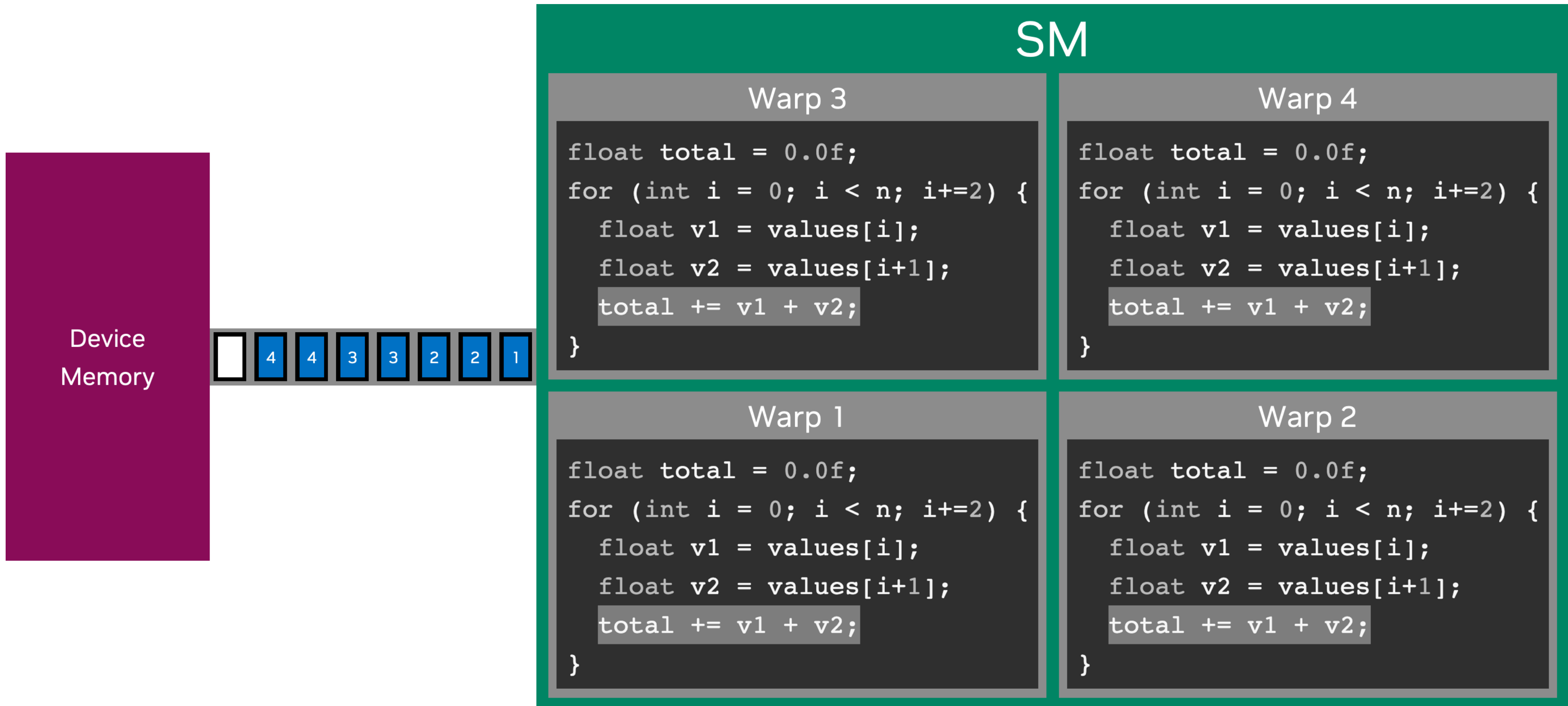
Bytes In Flight



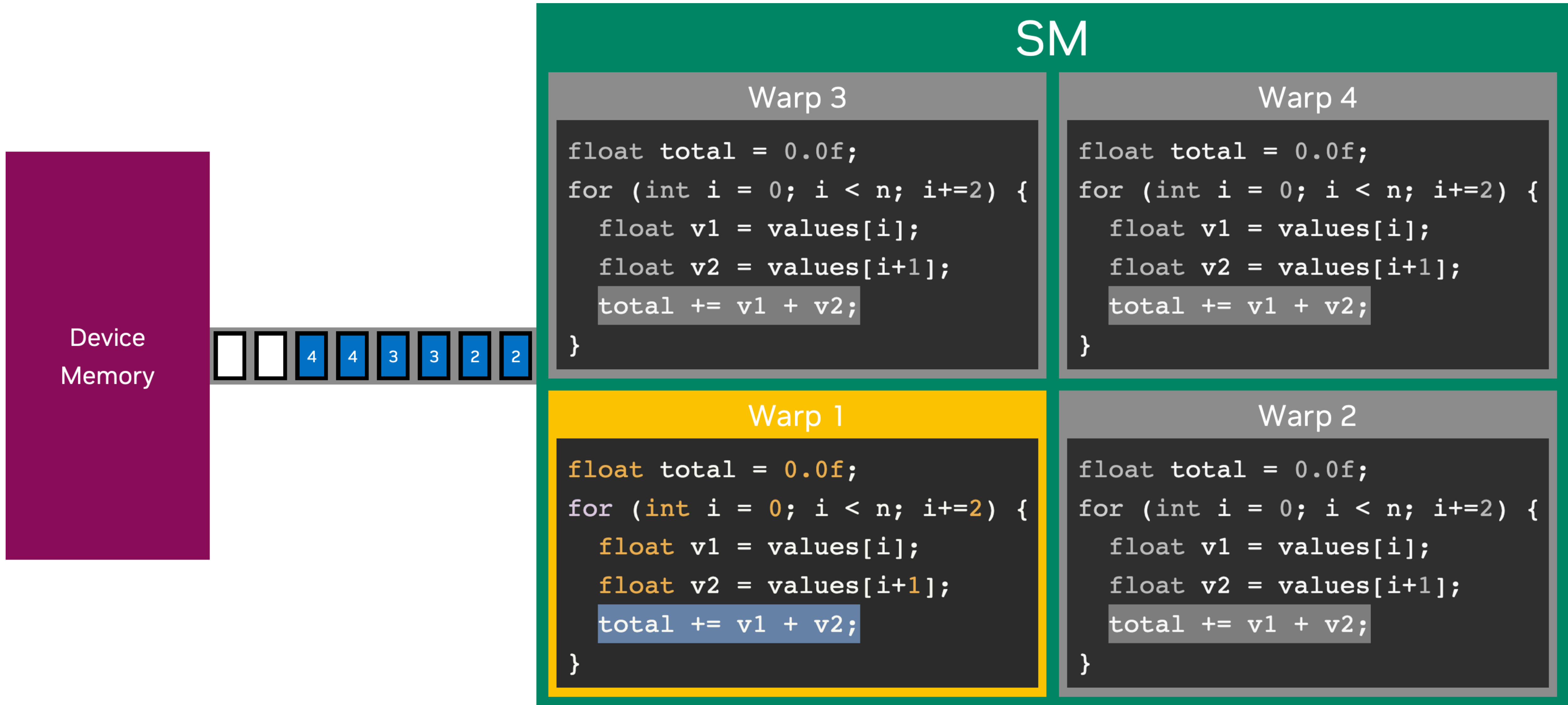
Bytes In Flight



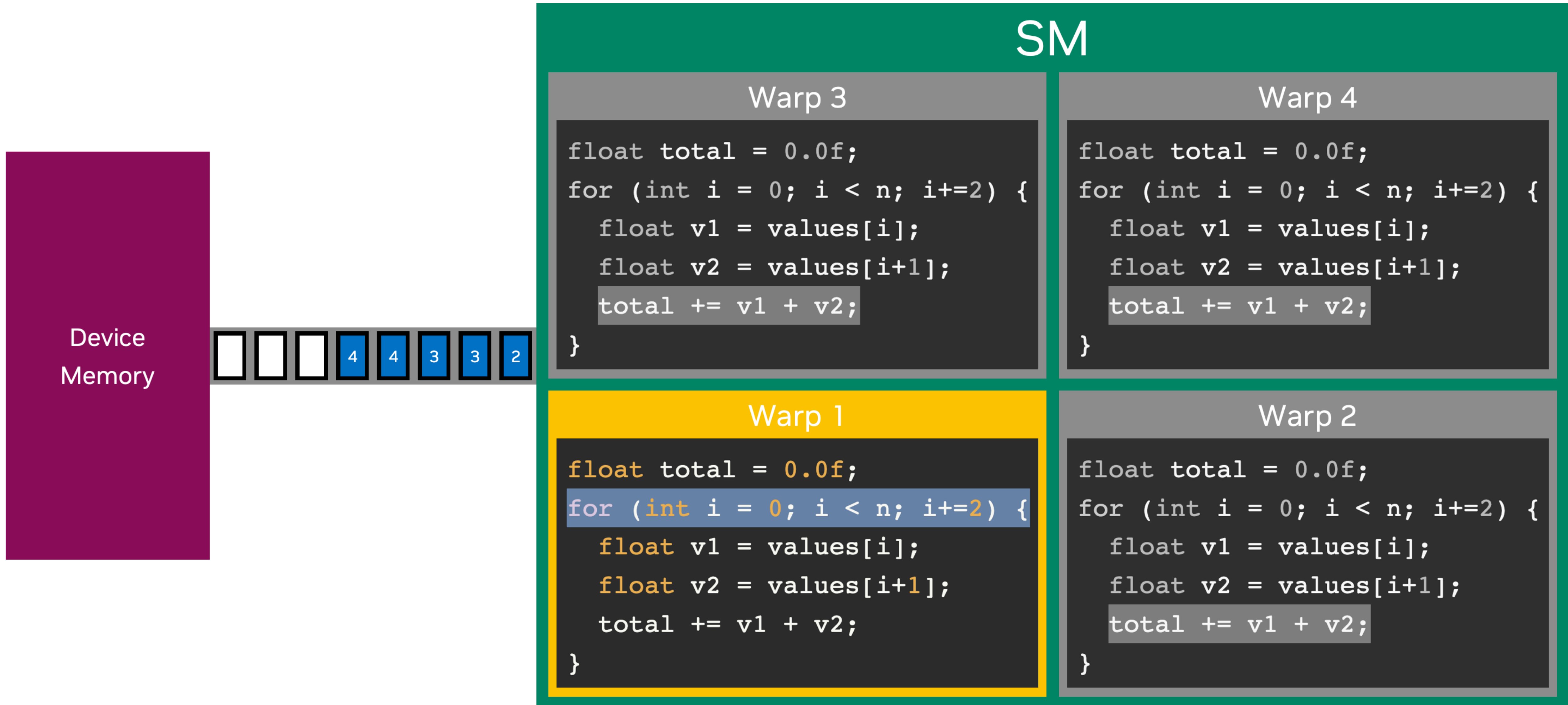
Bytes In Flight



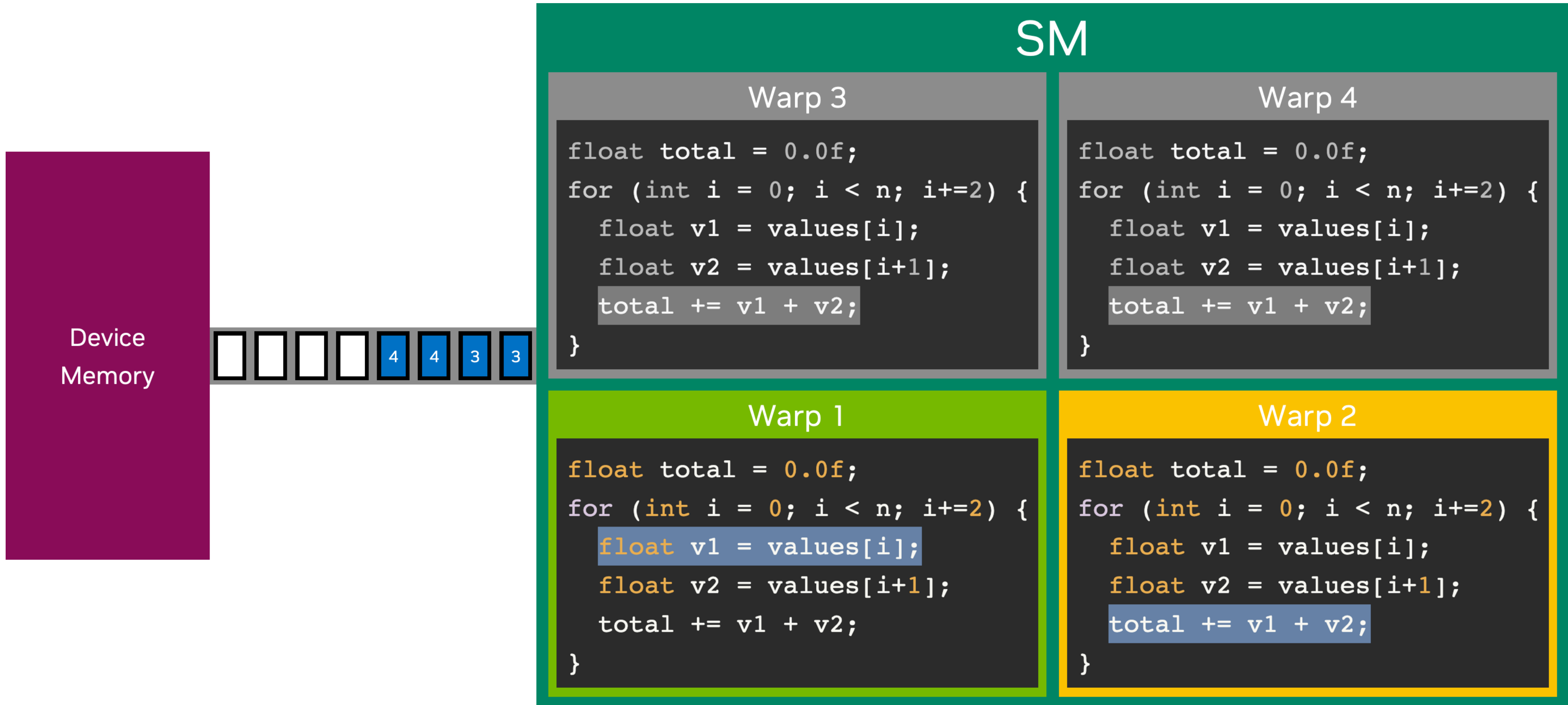
Bytes In Flight



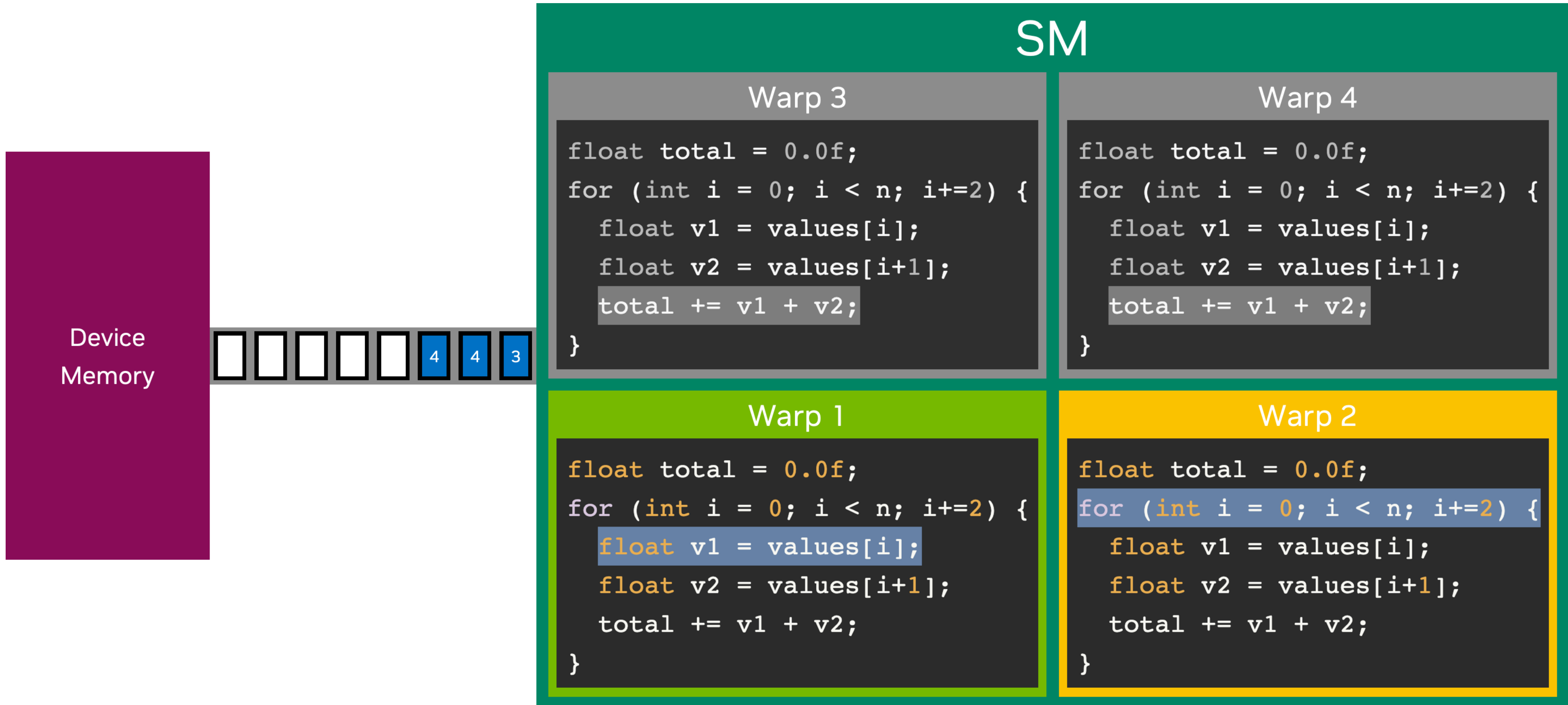
Bytes In Flight



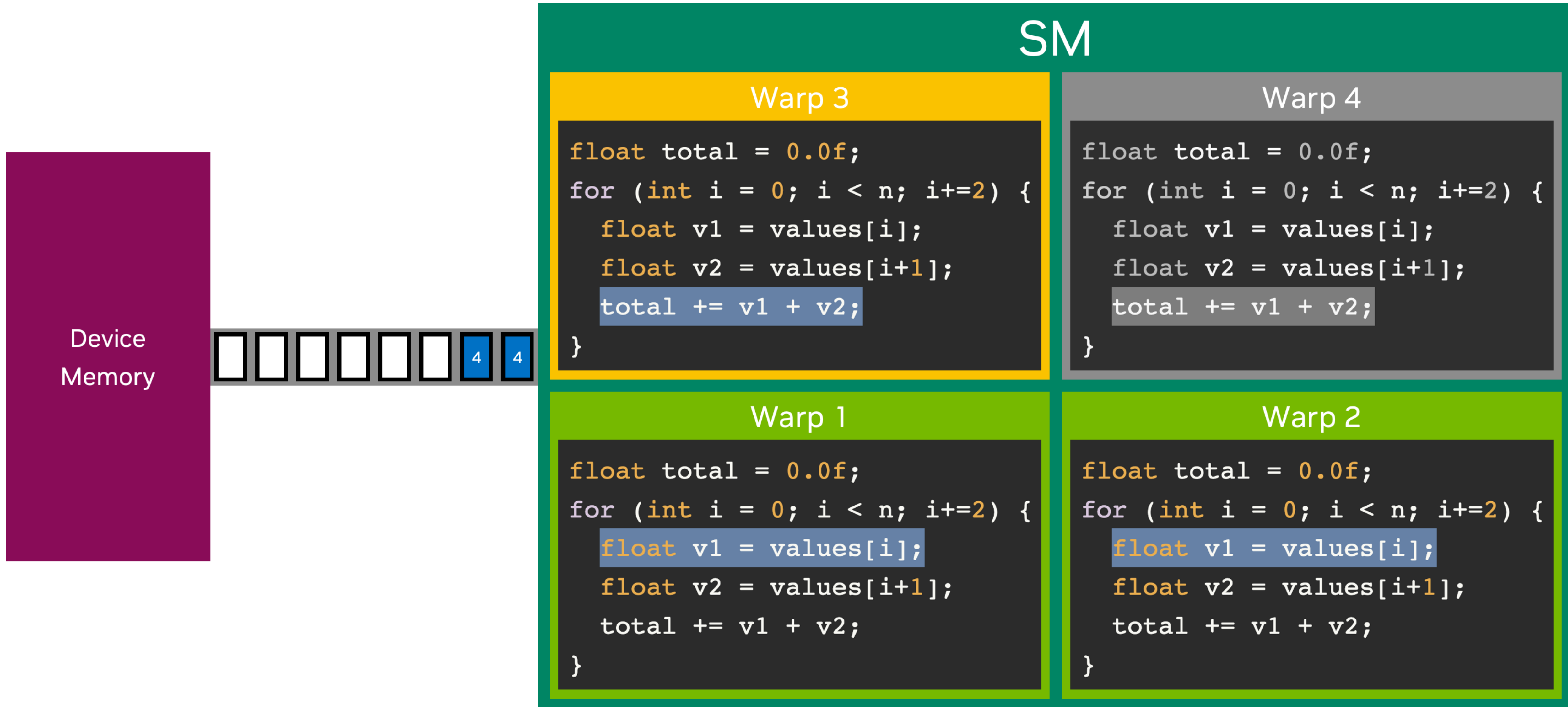
Bytes In Flight



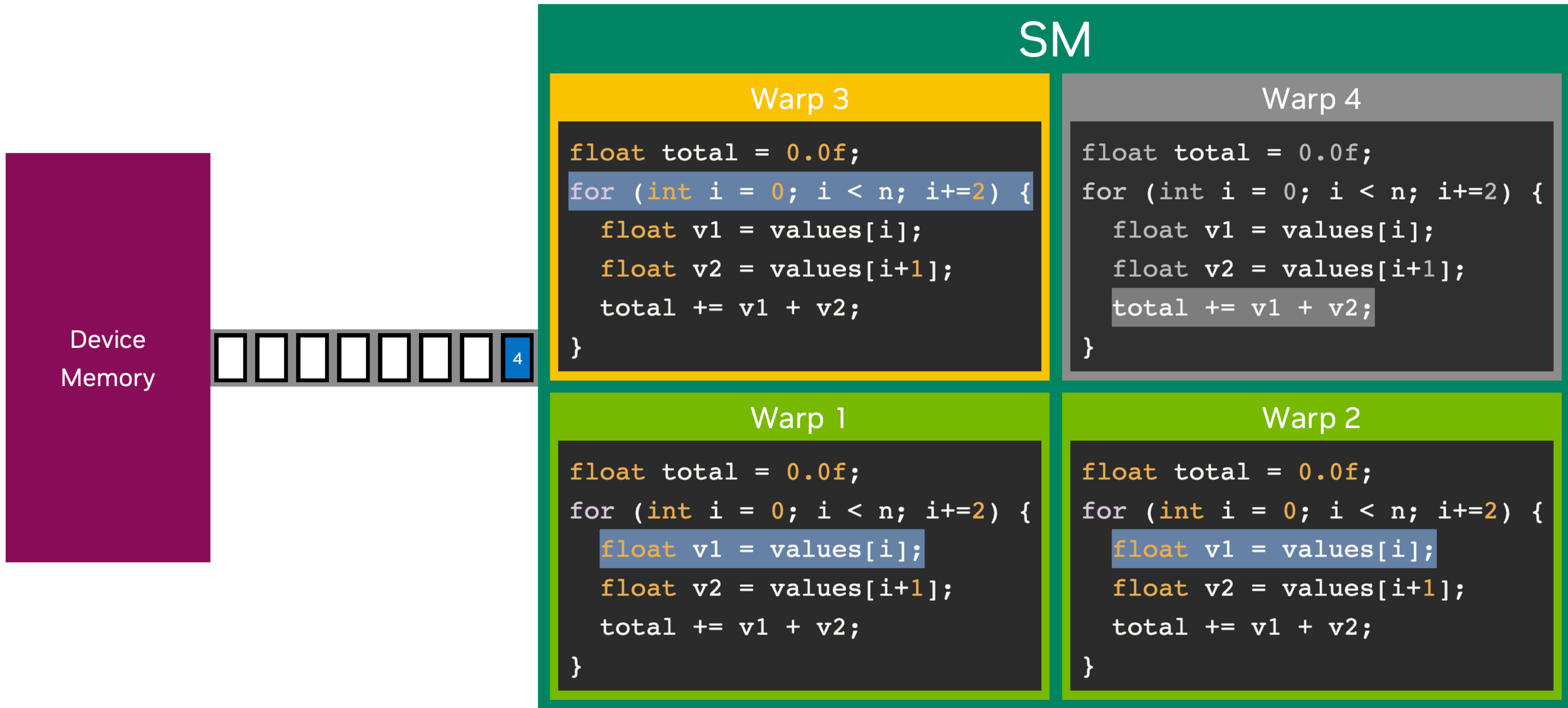
Bytes In Flight



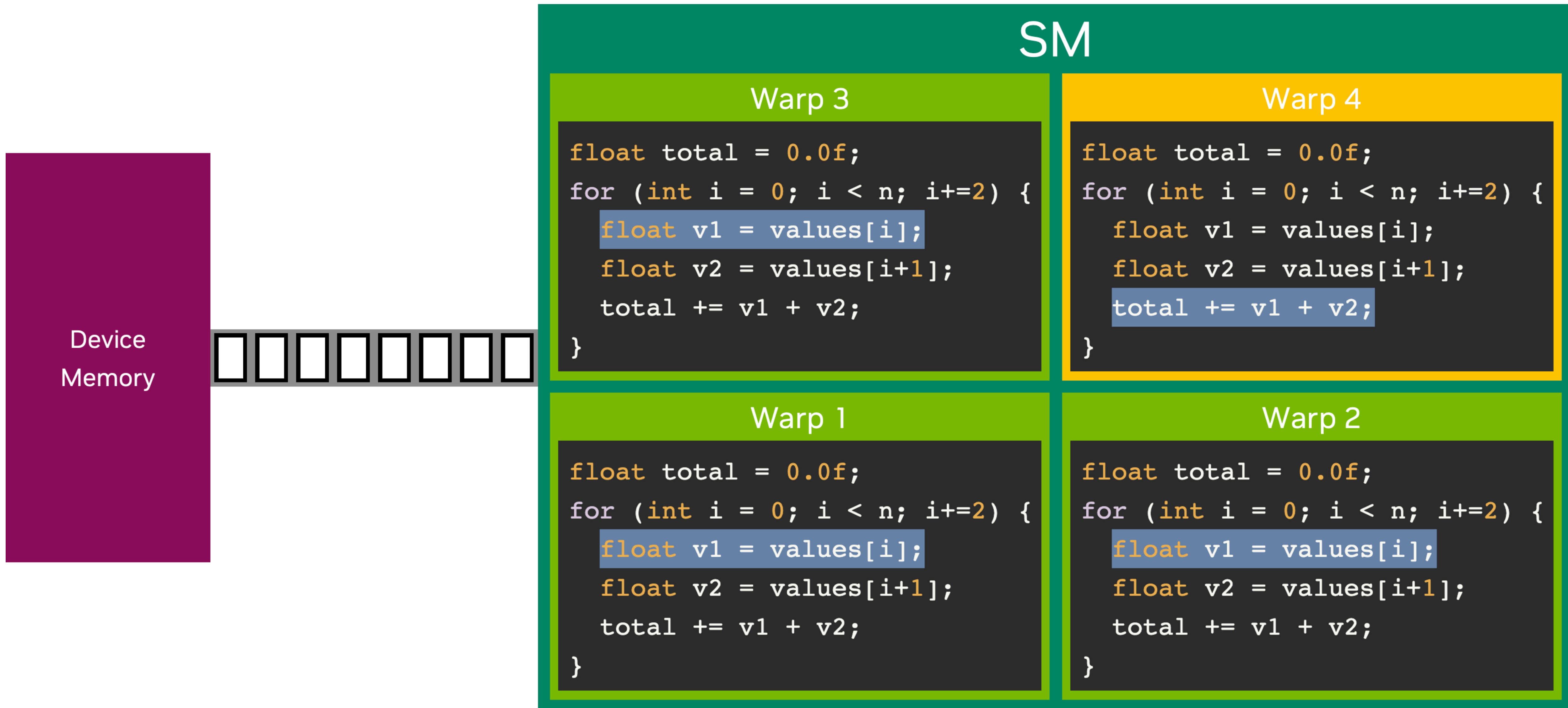
Bytes In Flight



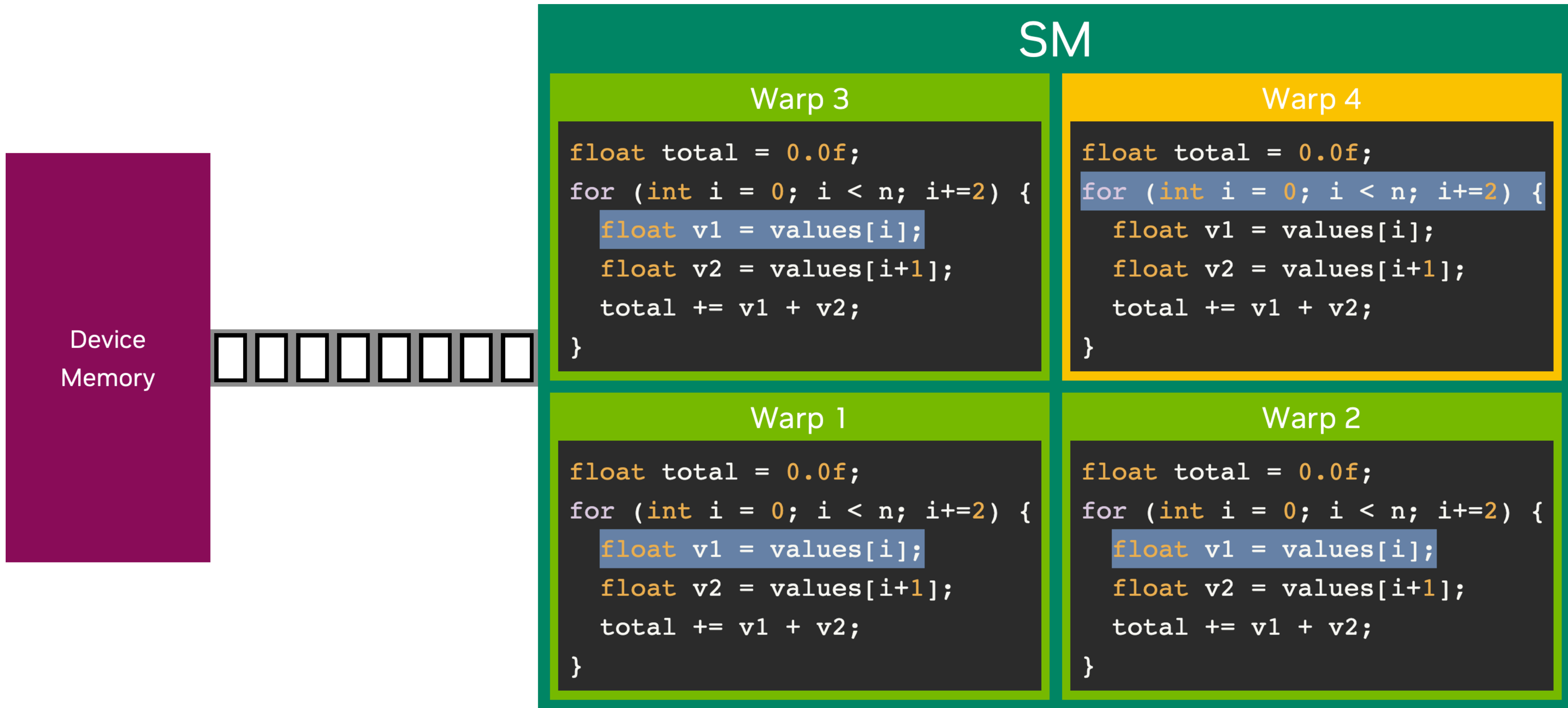
Bytes In Flight



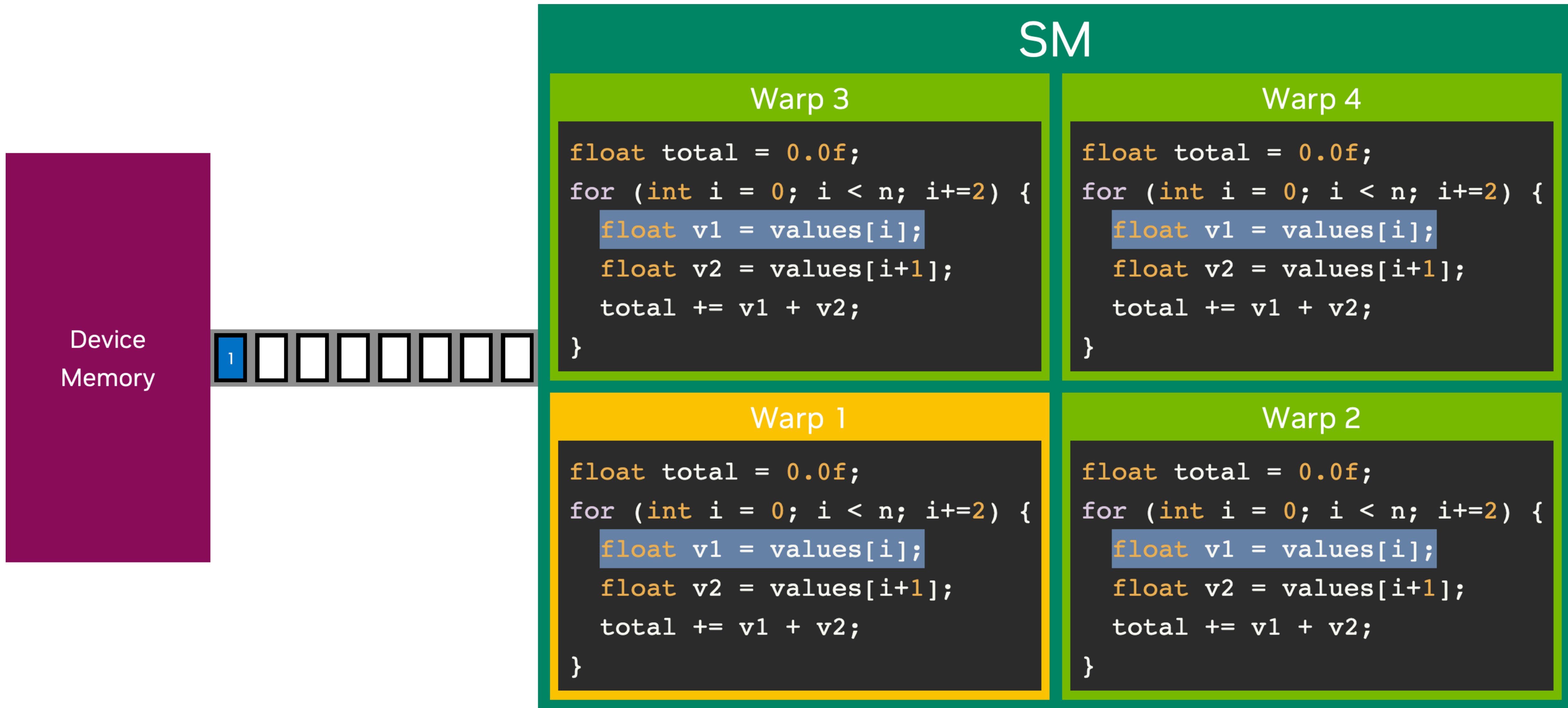
Bytes In Flight



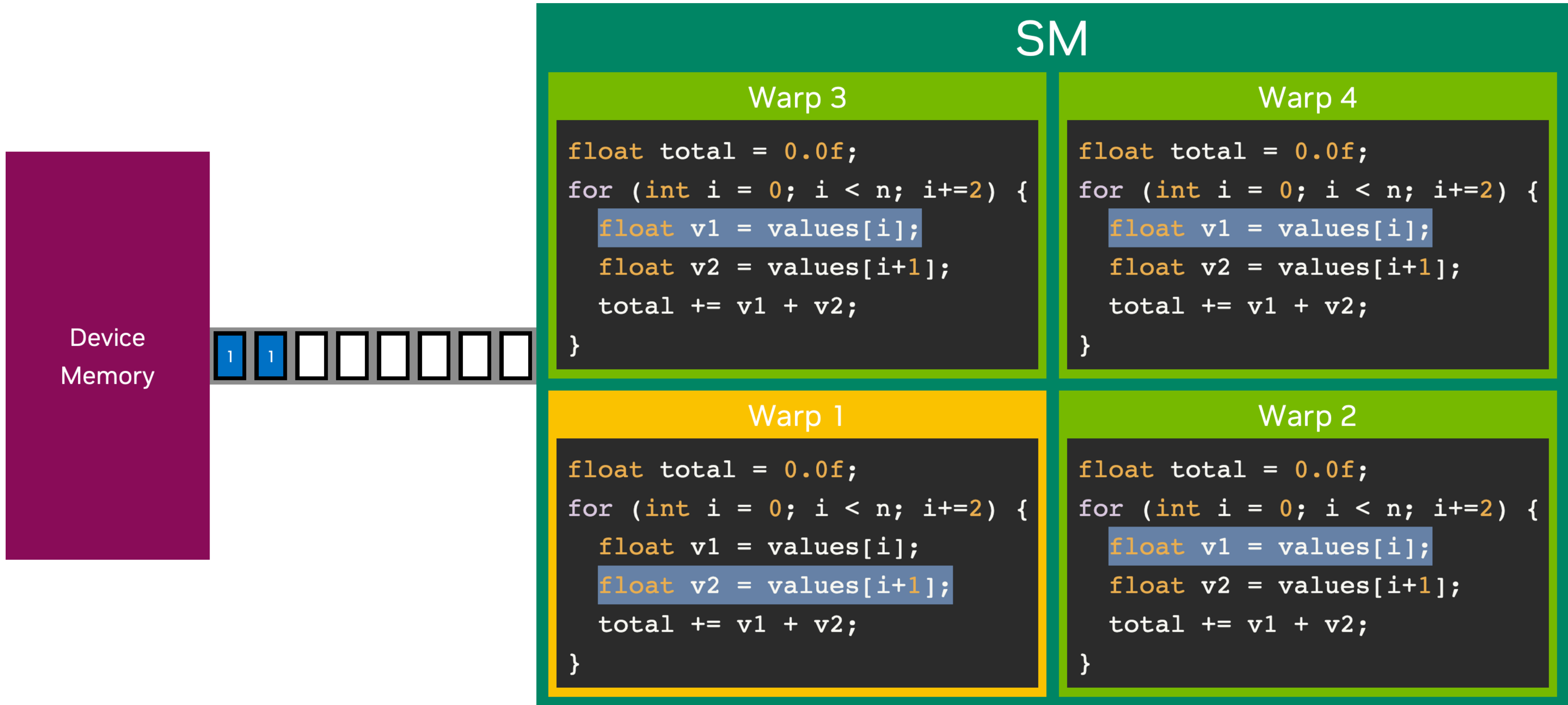
Bytes In Flight



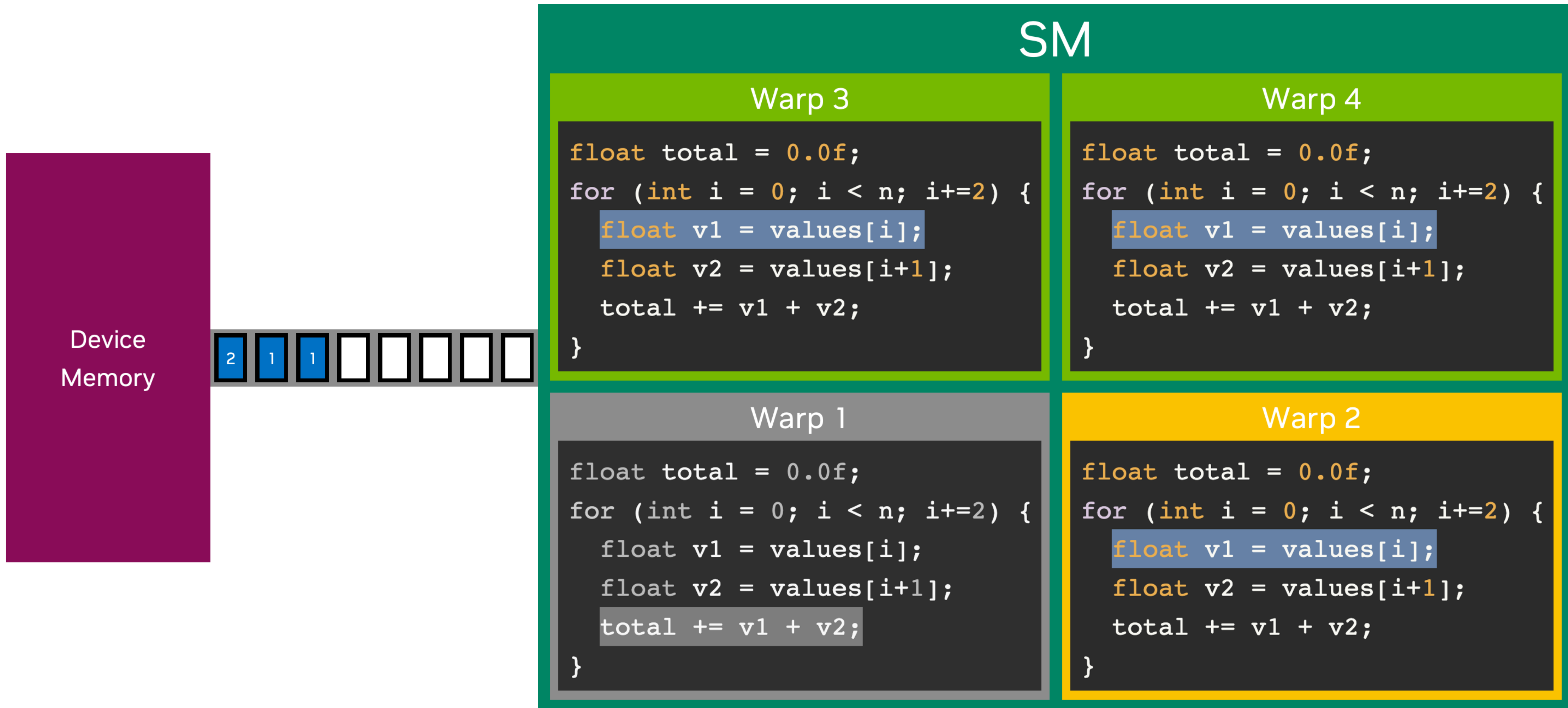
Bytes In Flight



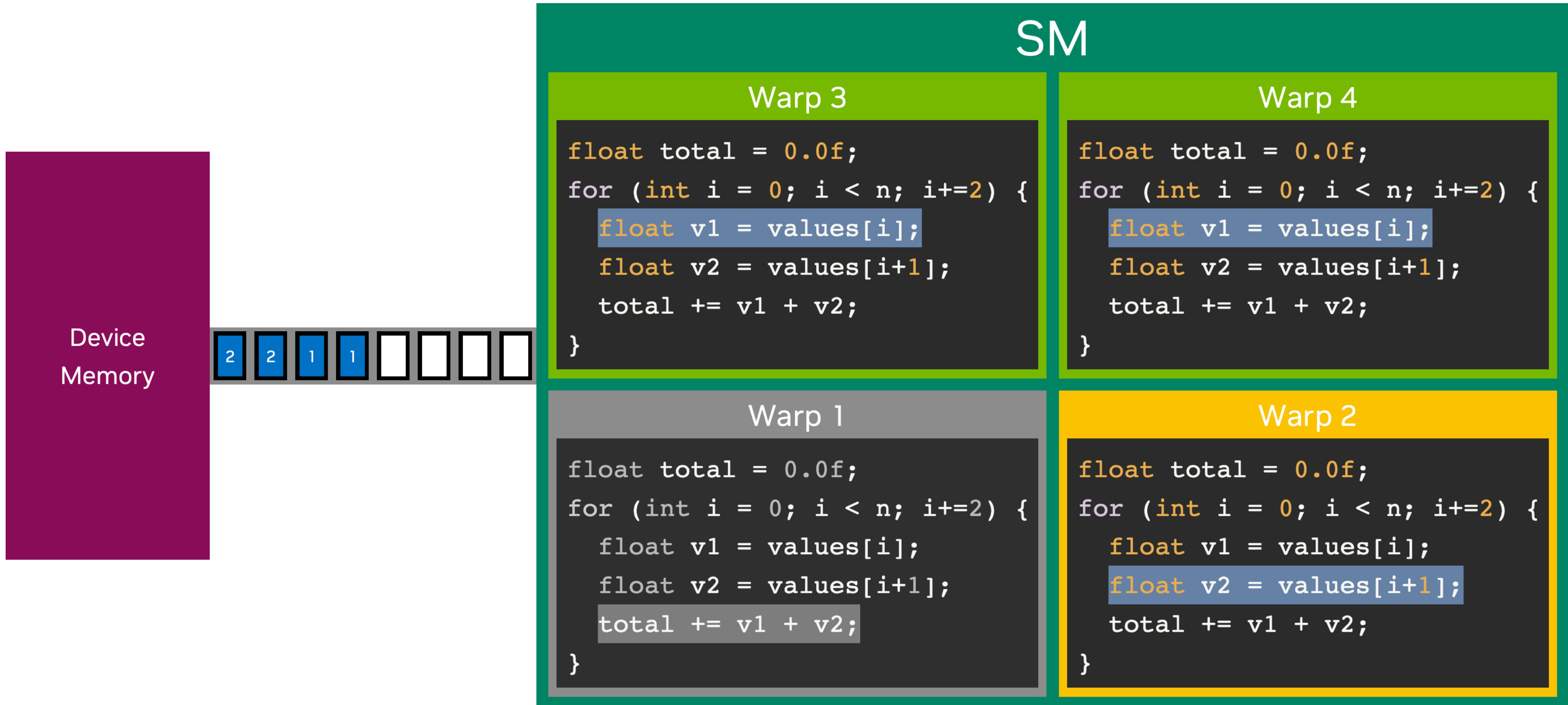
Bytes In Flight



Bytes In Flight



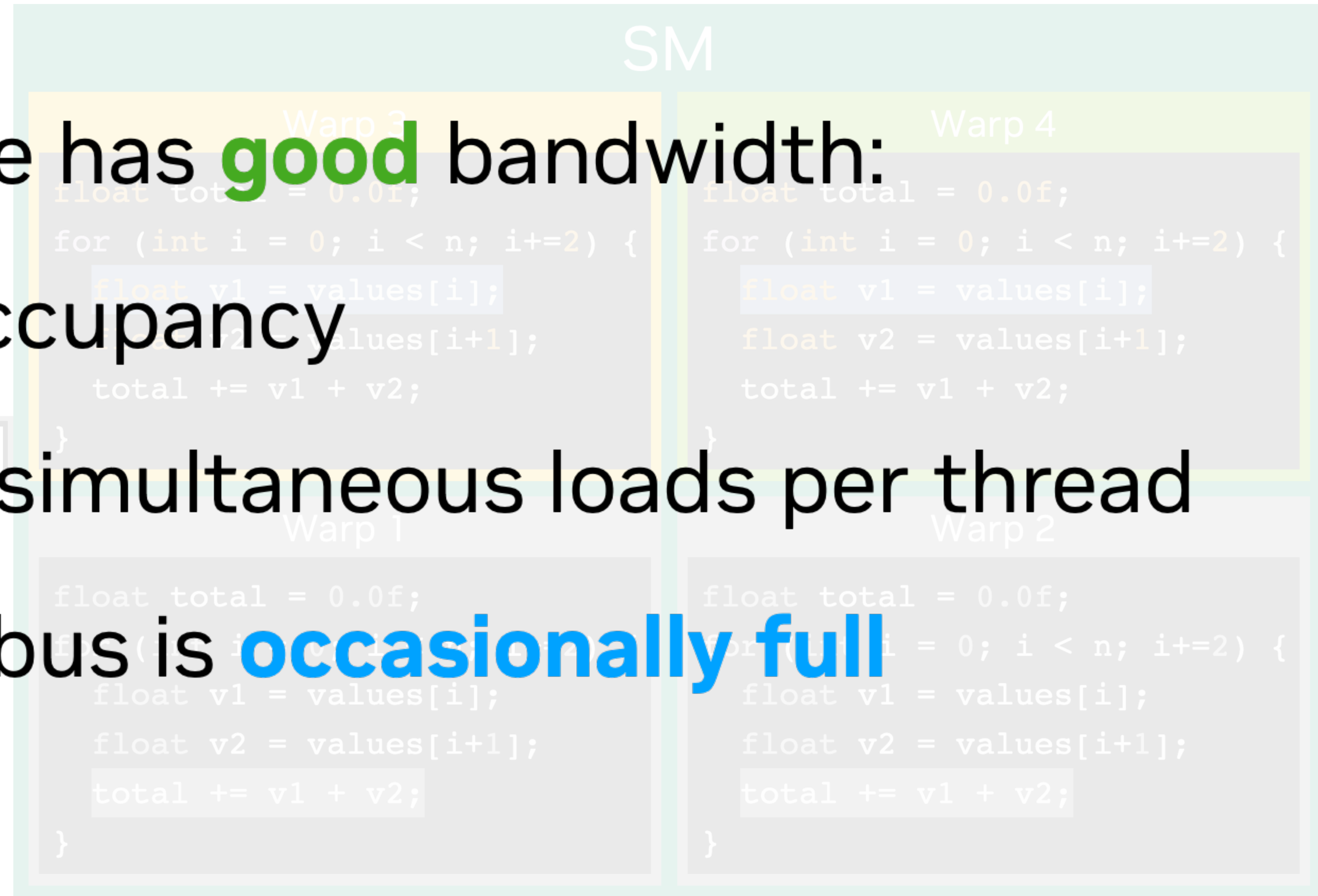
Bytes In Flight



Bytes In Flight

This example has **good** bandwidth:

- **Better** occupancy
- **Multiple** simultaneous loads per thread
- Memory bus is **occasionally full**



Device
Memory

Bytes In Flight

- Saturating the memory bus => High throughput
- Increasing the number of resident warps helps
 - ... but our original AXPY kernel already has 100% occupancy
- Initiating more loads per thread helps
 - Our original AXPY kernel only processed 1 entry / thread
 - **Let's try this idea out on our AXPY kernel**

Bytes In Flight

```
template < typename T, int n >
struct alignas(sizeof(T) * n) vec { // N.B. alignment matters!
    T values[n];
};

template < typename T, int n >
vec<T,n> operator+(const vec<T,n> & a, const vec<T,n> & b) {
    // ...
};

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

template < int k, typename T >
__global__ void axpy_vectorized(T a, const vec<T,k> * x, vec<T,k> * y, int n) {
    int tid = k * (threadIdx.x + blockIdx.x * blockDim.x);
    if (tid < n) {
        y[tid] = a * x[tid] + y[tid]; // assumes appropriate array padding
    }
}
```

axpy with vectorized loads

Bytes In Flight

```
template < int k, typename scalar_t >
__global__ void axpy_ilp(scalar_t a, const scalar_t * x, scalar_t * y, int n) {

    int tid = threadIdx.x + blockIdx.x * blockDim.x;

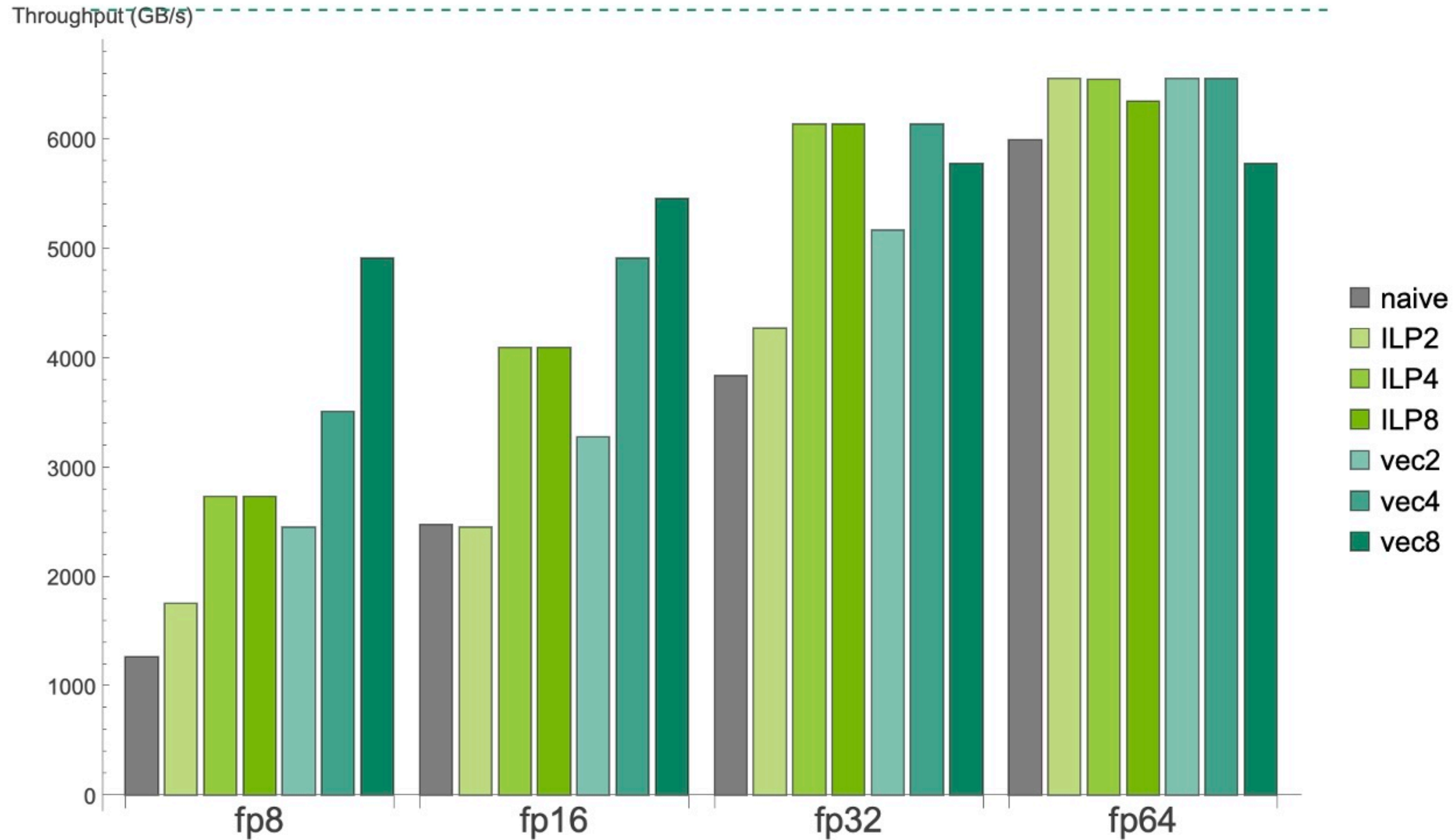
    scalar_t x_tmp[k];
    scalar_t y_tmp[k];
    for (int i = 0; i < k; i++) {
        int j = tid + i * gridDim.x * blockDim.x;
        if (j < n) {
            x_tmp[i] = x[j];
            y_tmp[i] = y[j];
        }
    }

    for (int i = 0; i < k; i++) {
        int j = tid + i * gridDim.x * blockDim.x;
        if (j < n) {
            y[j] = a * x_tmp[i] + y_tmp[i];
        }
    }

}
```

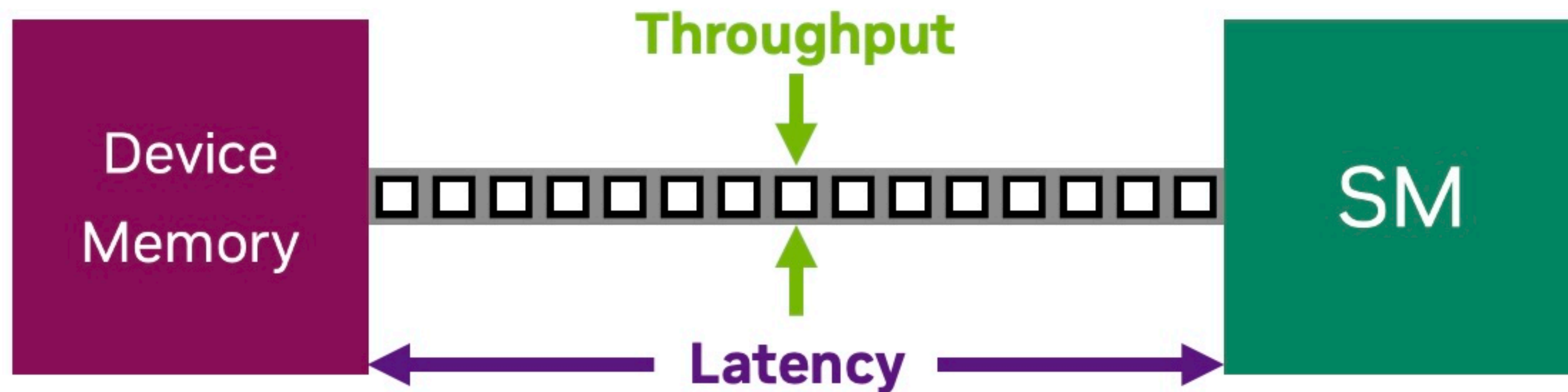
axpy with instruction-level parallelism (ILP)

Bytes In Flight



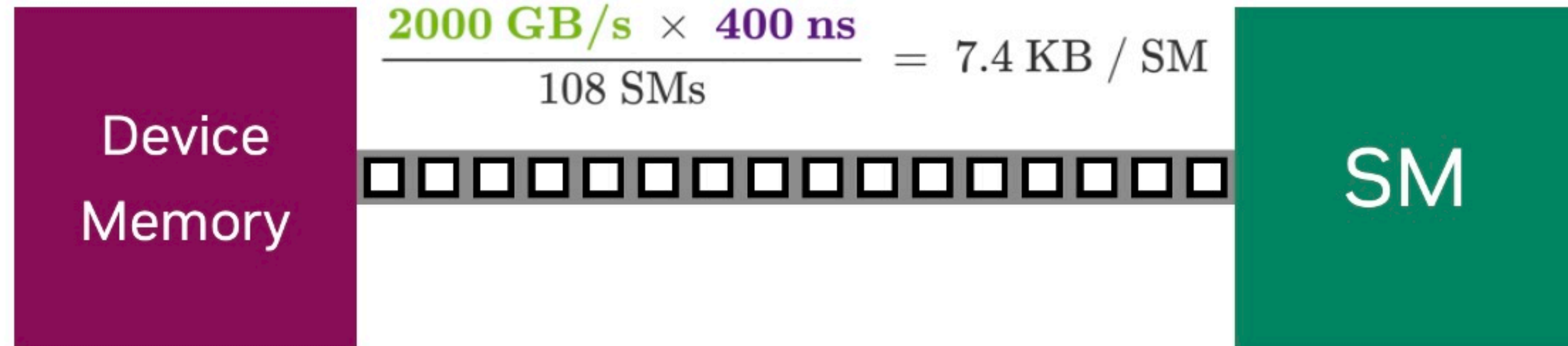
Bytes In Flight

$$\text{Bytes In Flight} = \text{Throughput} \times \text{Latency}$$

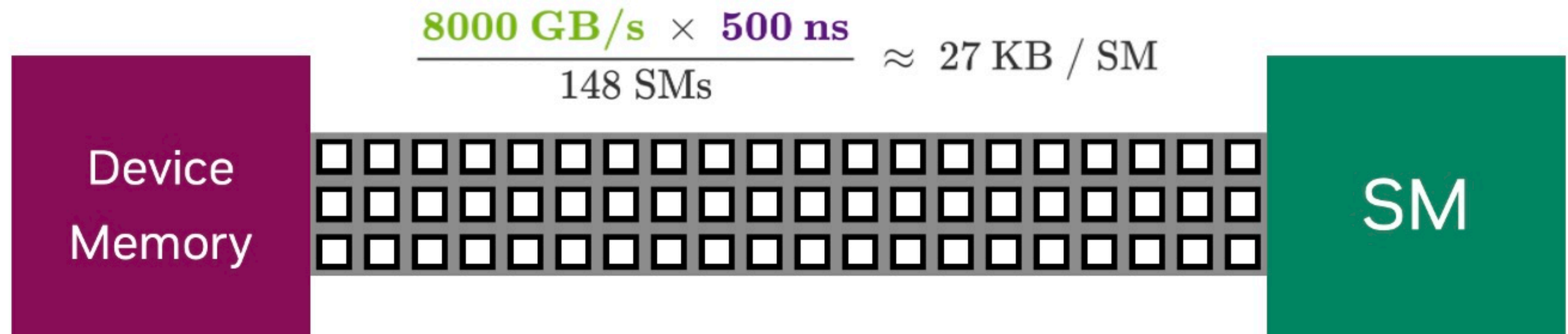


Bytes In Flight

A100

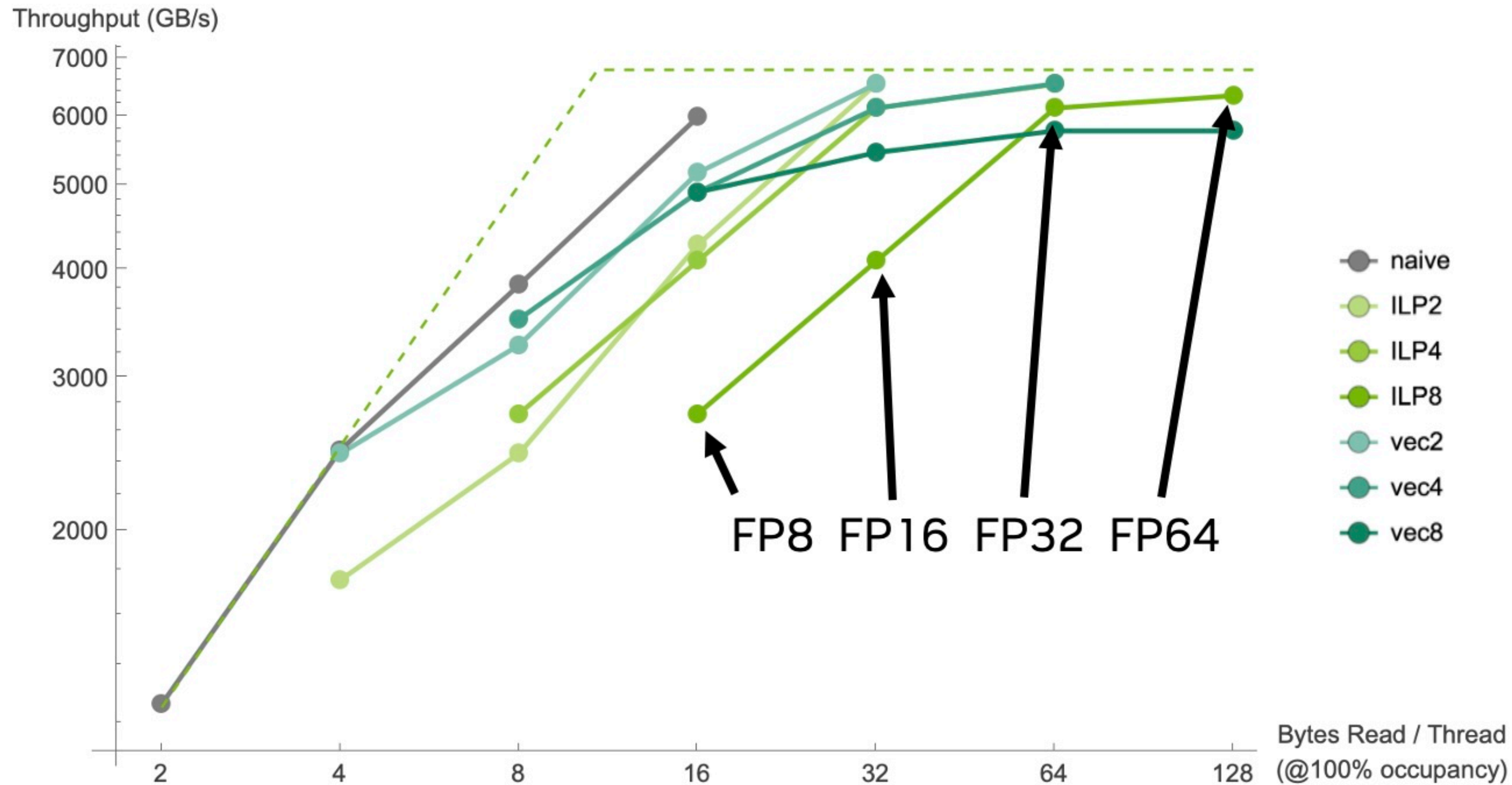


B200

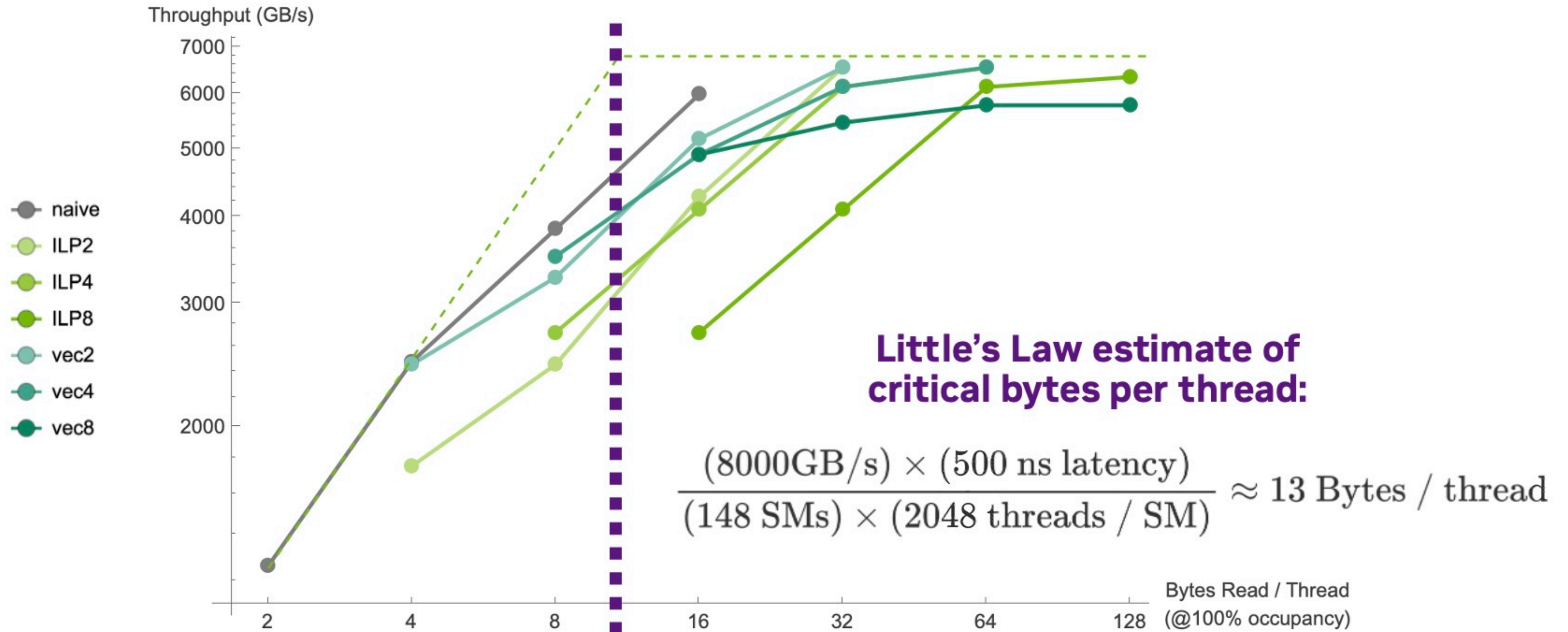


Bytes In Flight

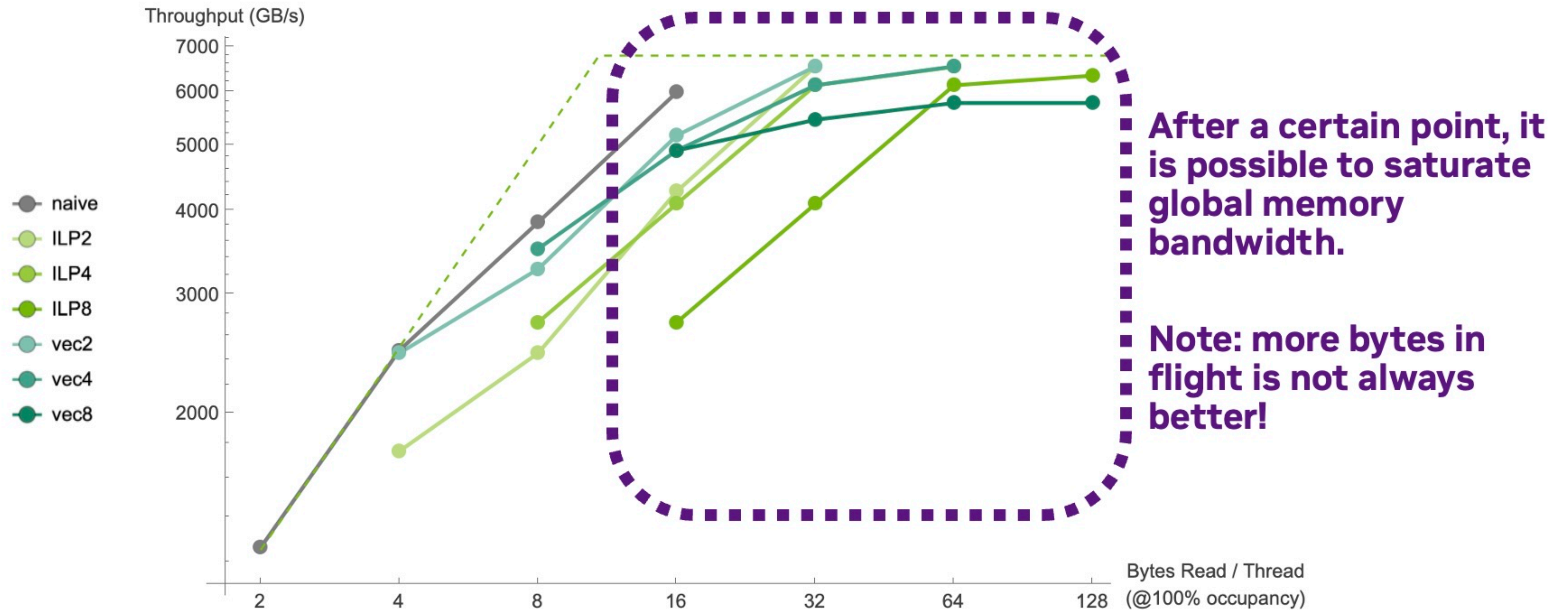
If we show the B200 results from the previous bar chart as a scatter plot, we see the following pattern:



Bytes In Flight



Bytes In Flight



Bytes In Flight

High memory throughput **requires** transactions to saturate the memory bus

- Warps **stall** when reaching an instruction that depends on data that has not arrived yet
- **bytes in flight** / SM requirements have increased for modern GPUs

Recommendations:

- Consider loading data **ahead-of-time**, rather than just-in-time
 - Leads to fewer stalls, and more average bytes-in-flight per thread
 - Also increases resource usage
- Try **parameterizing the kernel** and tune to find the "sweet spot"
 - Little's law can help us get a "ballpark" estimate of required bytes / thread

Optimization #6: AXPY / Dot Kernels

```
$ ./dot_product
```

```
fp32: 3.35544e+07: 0.350425 ms 383.014 GB/s  
fp64: 3.35544e+07: 0.424249 ms 632.731 GB/s  
  u8: 3.35544e+07: 0.34963 ms 95.9713 GB/s  
  u16: 3.35544e+07: 0.350156 ms 191.654 GB/s  
  u32: 3.35544e+07: 0.351249 ms 382.116 GB/s  
  u64: 3.35544e+07: 0.424565 ms 632.26 GB/s
```

```
$ ./dot_product_solution
```

```
fp32: 3.35544e+07: 0.220432 ms 608.885 GB/s  
fp64: 3.35544e+07: 0.424716 ms 632.035 GB/s  
  u8: 3.35544e+07: 0.095009 ms 353.171 GB/s  
  u16: 3.35544e+07: 0.128328 ms 522.948 GB/s  
  u32: 3.35544e+07: 0.220693 ms 608.165 GB/s  
  u64: 3.35544e+07: 0.424951 ms 631.686 GB/s
```


Optimization #6: AXPY / Dot Kernels

```
$ ./axpy
fp32: 0.476215 ms, 422.764 GB/s
fp64: 0.656838 ms, 613.017 GB/s
  u8: 0.475705 ms, 105.804 GB/s
  u16: 0.47584 ms, 211.549 GB/s
  u32: 0.476785 ms, 422.259 GB/s
  u64: 0.653189 ms, 616.442 GB/s
```

```
$ ./axpy_solution
fp32: 0.351281 ms, 573.121 GB/s
fp64: 0.672836 ms, 598.442 GB/s
  u8: 0.088873 ms, 566.332 GB/s
  u16: 0.169367 ms, 594.35 GB/s
  u32: 0.331918 ms, 606.555 GB/s
  u64: 0.66582 ms, 604.748 GB/s
```

Optional: Laplace Comparison

- See: 06_profiling/laplace_comparison directory

Another example of a performance study on stencil-type kernels.
Some similar optimizations were used: vectorized loads, ILP, blocksize tuning

Aside: Laplace Comparison

This table shows all results in the report. Use the column headers to sort the results in this report. Double click a result to see detailed metrics. Double click on demangled names to rename it.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [us] (8.53 us)	Runtime Improvement [us] (2.67 us)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thr]	Grid Size
0	61.51	laplace_original	..laplace_original..	2.22	1.37	87.39	26.66	23	16, 64, ...
1	15.88	laplace_vectoriz...	..laplace_vectori...	0.69	0.11	21.79	84.12	28	8, 64, ...
2	19.62	laplace_vectoriz...	..laplace_vectori...	0.68	0.13	10.92	84.45	32	4, 64, ...
3	15.95	laplace_tiled_3D	..laplace_tiled_3...	0.69	0.11	20.18	84.05	24	8, 64, ...
4	19.68	laplace_tiled_3D	..laplace_tiled_3...	0.69	0.13	10.43	84.45	32	4, 64, ...
5	16.79	laplace_tiled_3D	..laplace_tiled_3...	0.69	0.12	16.55	84.22	27	8, 32, ...
6	22.39	laplace_tiled_3D	..laplace_tiled_3...	0.69	0.15	9.77	84.06	34	4, 32, ...
7	24.09	laplace_tiled_3D	..laplace_tiled_3...	0.70	0.17	8.41	83.18	42	4, 16, ...
8	19.59	laplace_tiled_3D	..laplace_tiled_3...	0.69	0.14	14.52	84.07	32	8, 32, ...
9	31.07	laplace_tiled_3D	..laplace_tiled_3...	0.78	0.24	6.49	77.50	54	4, 16, ...

Aside: Numbering / Unstructured Access

Demo: 06_profiling/unstructured_mesh directory

Aside: Numbering / Unstructured Access

```
$ ./unstructured_mesh_solution
```

```
(cpu) z_rotation calculation took 14.7848 ms
```

```
(cpu) normal calculation took 59.6735 ms
```

```
(gpu) z_rotation calculation took 0.938733 ms
```

```
(gpu) normal calculation took 2.02847 ms
```

```
randomizing mesh numbering ... done
```

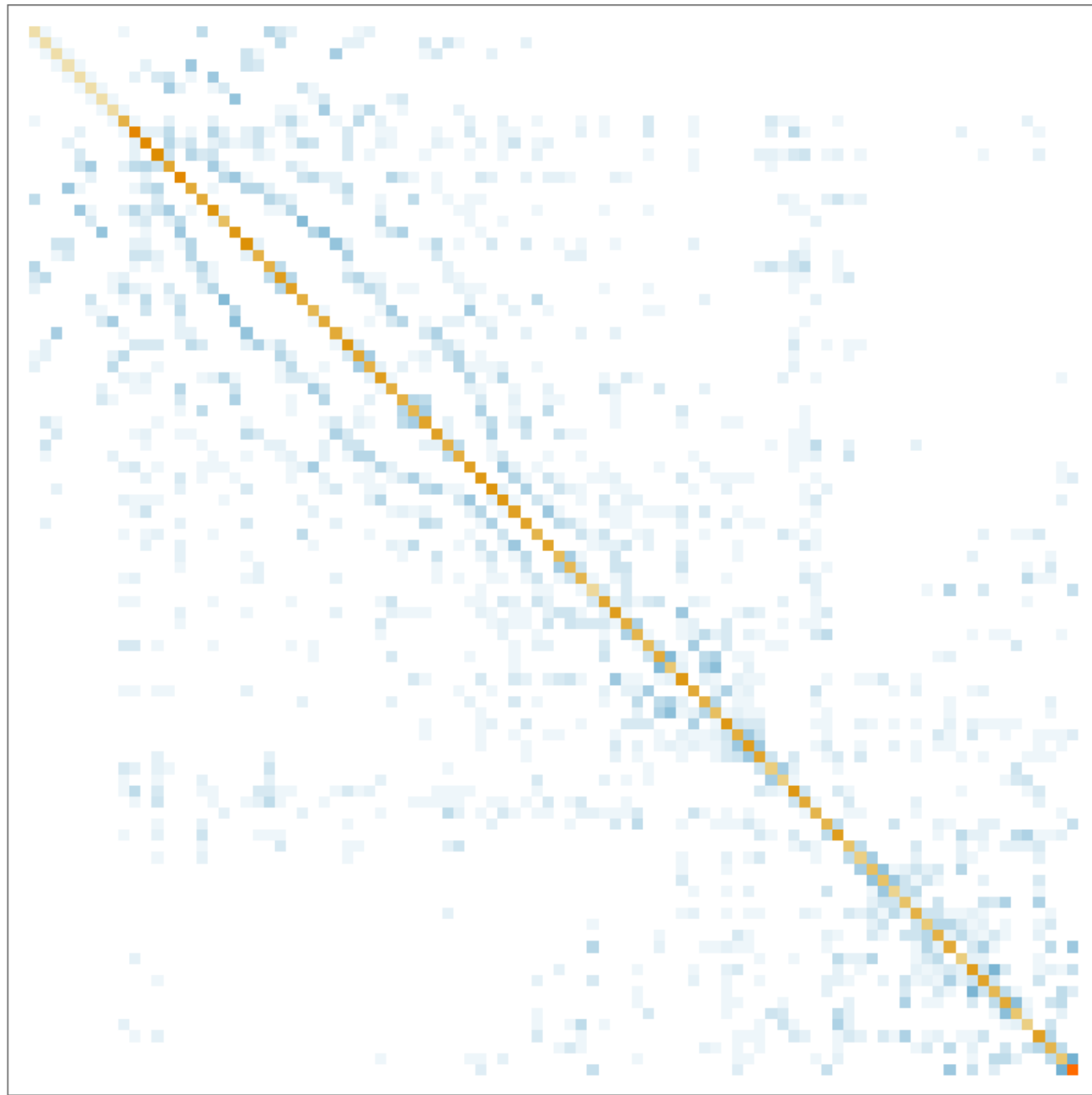
```
(cpu) z_rotation calculation took 14.7069 ms
```

```
(cpu) normal calculation took 752.328 ms
```

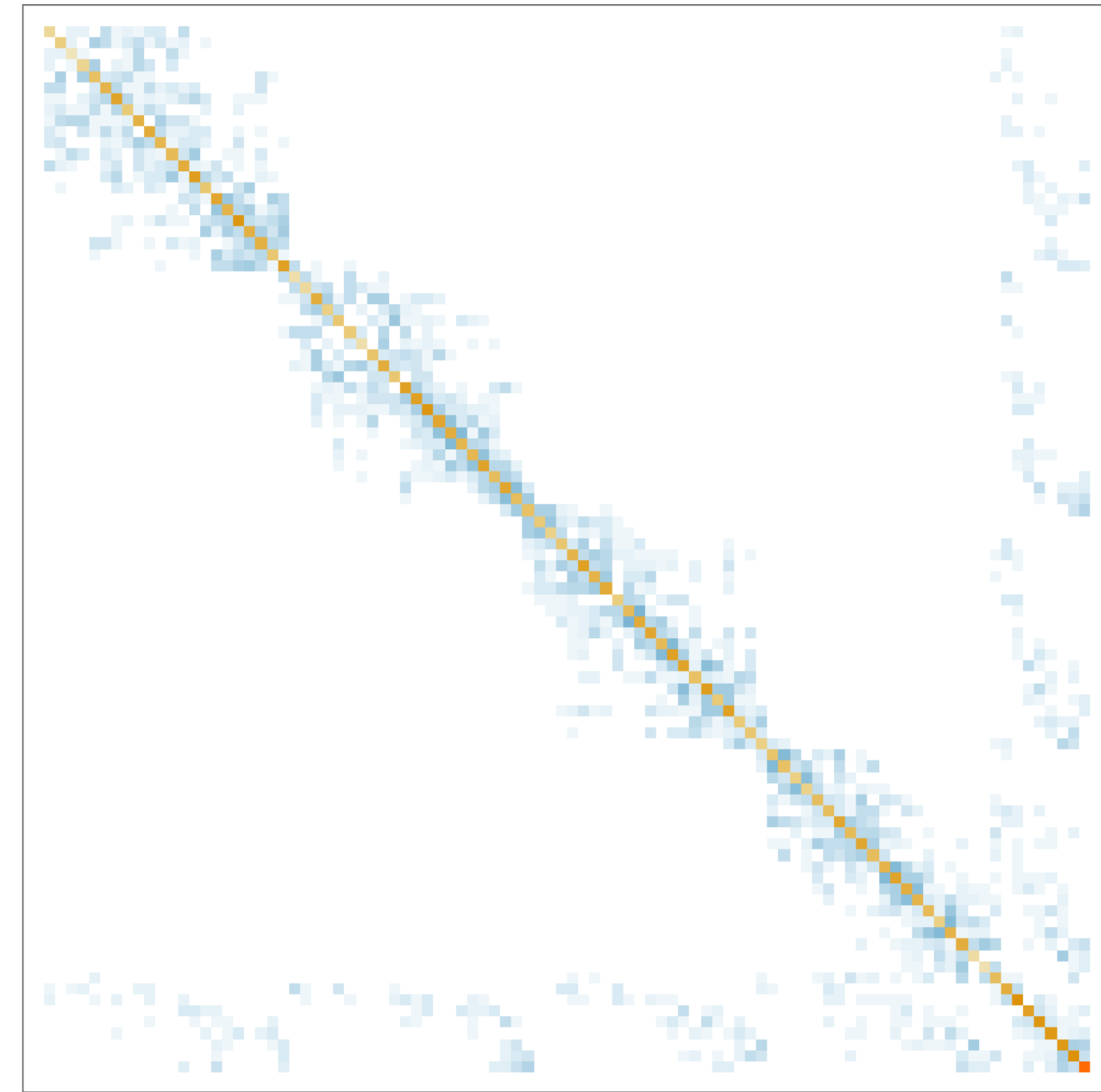
```
(gpu) z_rotation calculation took 0.936833 ms
```

```
(gpu) normal calculation took 38.2973 ms
```


Aside: Numbering / Unstructured Access



original



reordered

Summary: Optimization Process

1. run / profile application
2. identify aspects that could use improvement
3. select one that has high (estimated) return on investment
4. implement the change
5. accept changes if helpful, else discard.
6. GOTO 1

Summary: Optimization Process

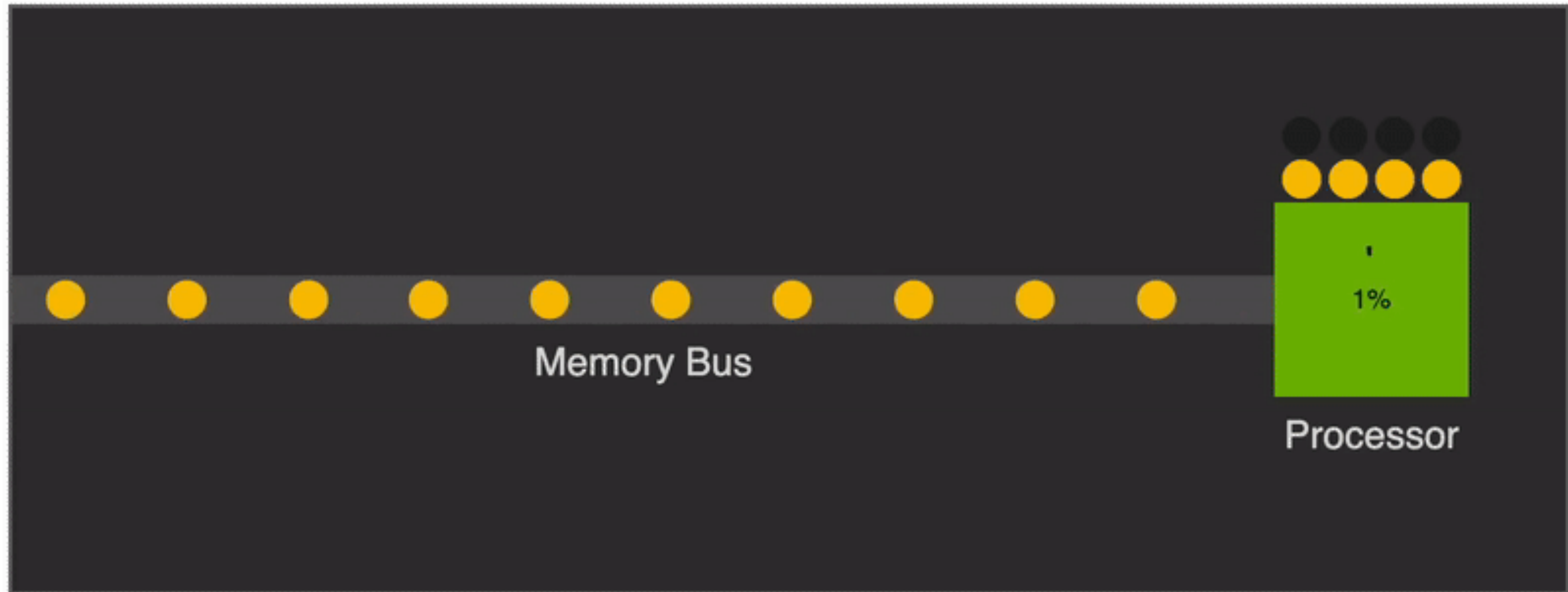
1. run / profile application
2. identify aspects that could use improvement
3. select one that has high (estimated) return on investment
4. implement the change
5. accept changes if helpful, else discard.
6. GOTO 1

Software development is a constrained optimization problem!

Summary: Kinds of Bottlenecks

- Compute Bound: **High** Compute Throughput, **Low** Memory Throughput
 - Time to do computation outweighs time to move data
- Memory Bound: **Low** Compute Throughput, **High** Memory Throughput
 - Time to move data outweighs time to do computation
- Latency Bound: **Low** Compute Throughput, **Low** Memory Throughput
 - Not enough concurrency to hide latency (typically)

Summary: Memory-Bound Kernels



Summary: Memory-Bound Kernels

- Which memory subsystem is the bottleneck?
 - L1, L2, DRAM
- What are my data access patterns like?
 - coalesced / random, structured / unstructured
- How is my data laid out?
 - flat, tree-like, AoS, SoA, AoSoA
- Do I have enough bytes-in-flight?
 - occupancy, vector loads, ILP, identify stalls
- Are multiple memory-bound kernels executed in sequence?
 - consider kernel fusion, or sizing problems to fit in L2\$
- Are all the data accesses necessary?
 - some quantities could be recalculated, rather than loaded

Summary: Compute-Bound Kernels



Summary: Compute-Bound Kernels

- Which kind of instructions are the bottleneck?
 - integer, floating point, LSU
- How accurate do my answers need to be?
 - FP32/FP64, numerical approximations
- Are all the calculations necessary?
 - save/load expensive quantities
 - table-lookup
 - transcendental functions
 - integer division and modulo
- Am I multiplying lots of dense matrices?
 - tensor cores
- See also: compiler intrinsics / inline PTX